



媒体与认知

Media and Cognition

清华大学电子工程系

王生进 李亚利

Email: liyali13@tsinghua.edu.cn

第4讲 深度学习

4.1 深度学习概述

4.2 人工神经元模型

4.3 全连接前馈神经网络

4.1 深度学习概述

➤ 浅层学习(Shallow Learning)

- ◆ 传统机器学习和信号处理仅含单层非线性变换，称为浅层学习结构
- ◆ 例：感知器模型、线性判别分析、支持向量机
- ◆ 例：隐马尔可夫模型、条件随机场
- ◆ 对复杂函数表示能力有限，对复杂分类问题性能受限

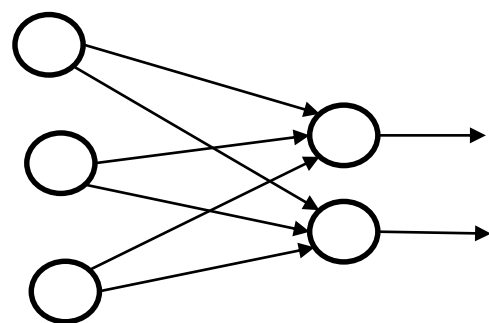
➤ 深度学习(Deep Learning)

- ◆ 受大脑的分层结构启发，利用多个隐层的人工神经网络赋予特征学习能力；
- ◆ 通过学习深层非线性网络结构，实现复杂函数的逼近，从数据中学习本质特征。

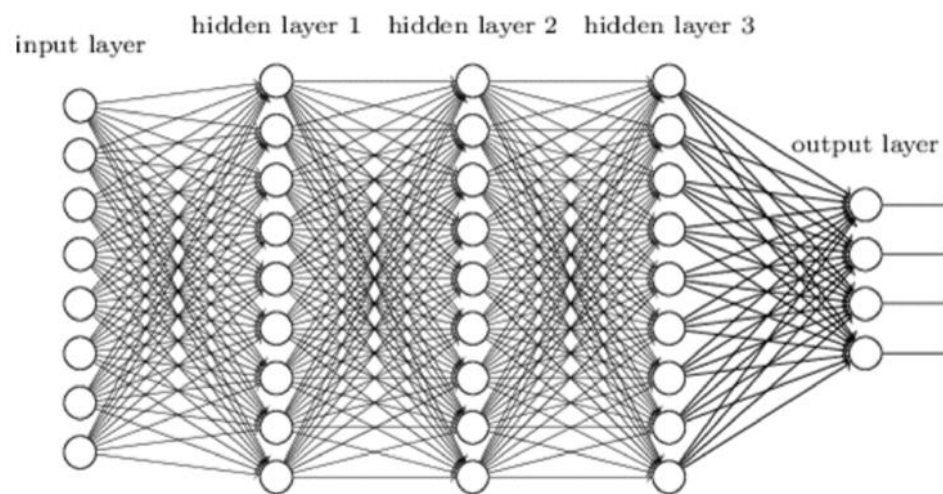
4.1 深度学习概述

➤ 深度学习与浅层学习

- ◆ 深度学习：强调模型结构的深度，通常有5层以上的隐层节点；
- ◆ 深度学习通过逐层特征变换，将样本从原特征空间变换到新的特征空间，从而使分类预测更加容易；
- ◆ 利用大数据来学习特征，实现对模式本质特征的自动学习。



$$f(\mathbf{x})$$

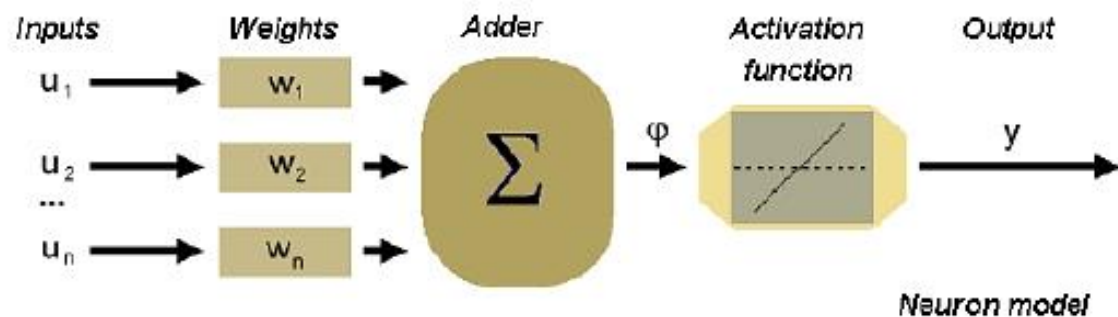
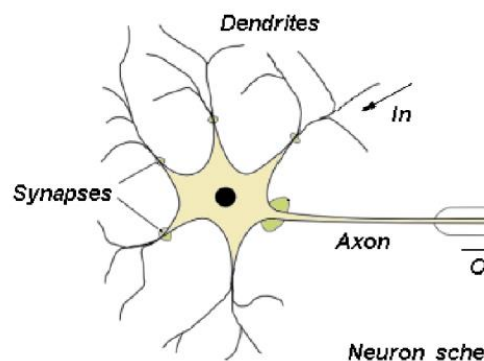


$$f_l(\cdots f_3(f_2(f_1(\mathbf{x}))))$$

4.1 深度学习概述：发展历史

➤ 神经网络概念及人工神经元的数学模型 (McCulloch, Pitts, 1943)

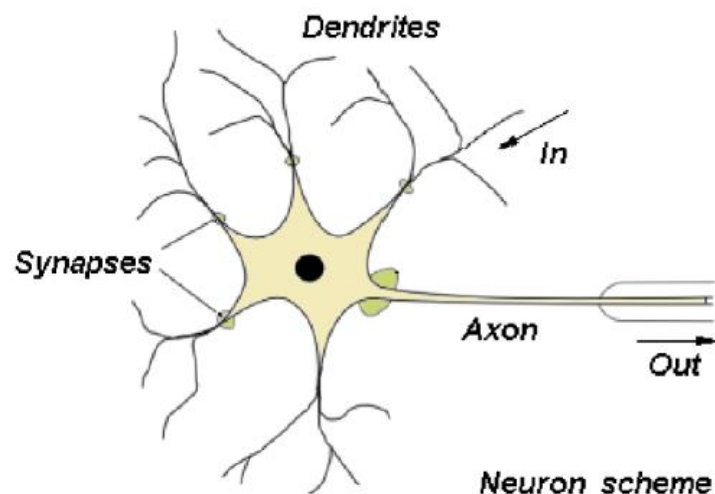
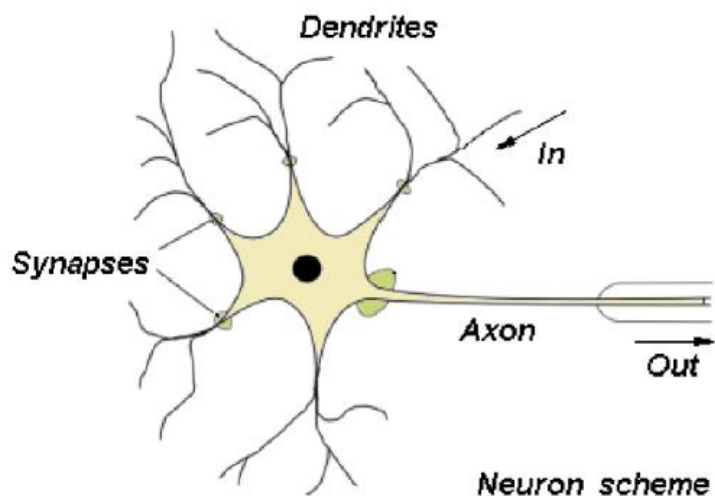
- ◆ 神经元的“兴奋”和“抑制”
- ◆ 神经元具备空间整合特征和阈值特征
- ◆ 神经元输入与输出间有固定时滞



4.1 深度学习概述：发展历史

➤ 神经集合体假设 (Hebb, 1949)

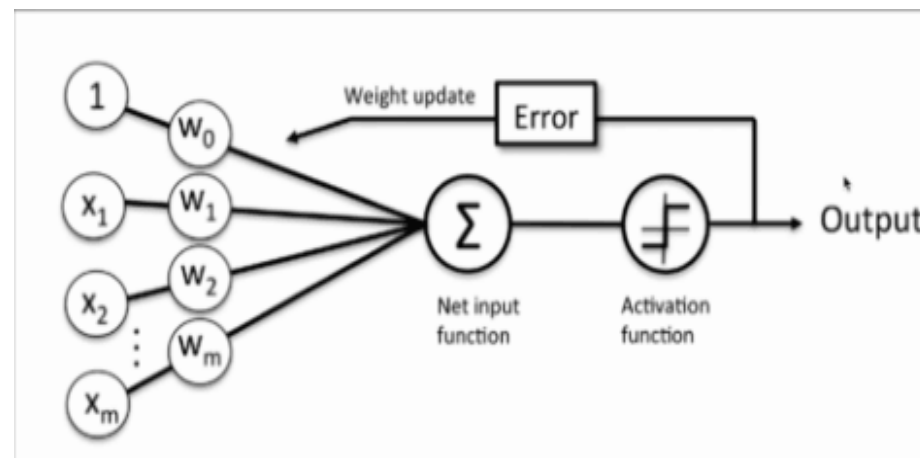
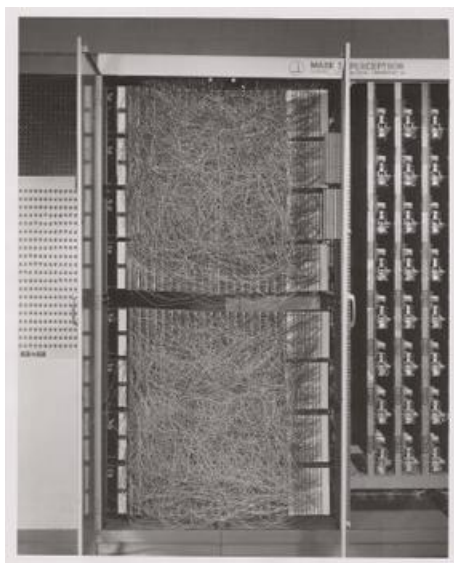
- ◆ 细胞集合由成百个用突触形成大规模连接的神经元细胞组成的；
- ◆ Hebb规则：神经网络的学习过程最终是发生在神经元之间的突触部位，突触的联结强度随着突触前后神经元的活动而变化，变化的量与两个神经元的活性之和成正比；



4.1 深度学习概述：发展历史

➤ 感知器模型 Perceptron (Rosenblatt, 1957)

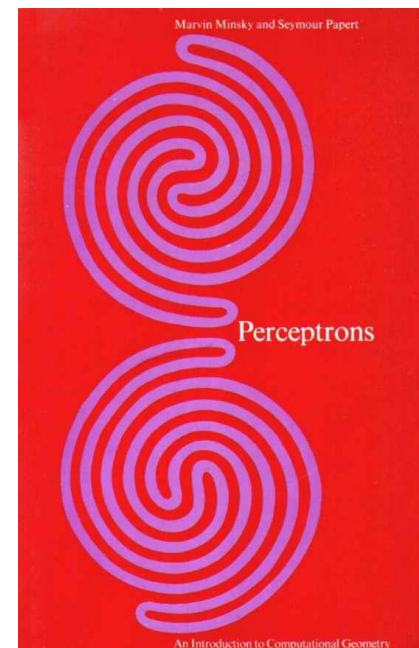
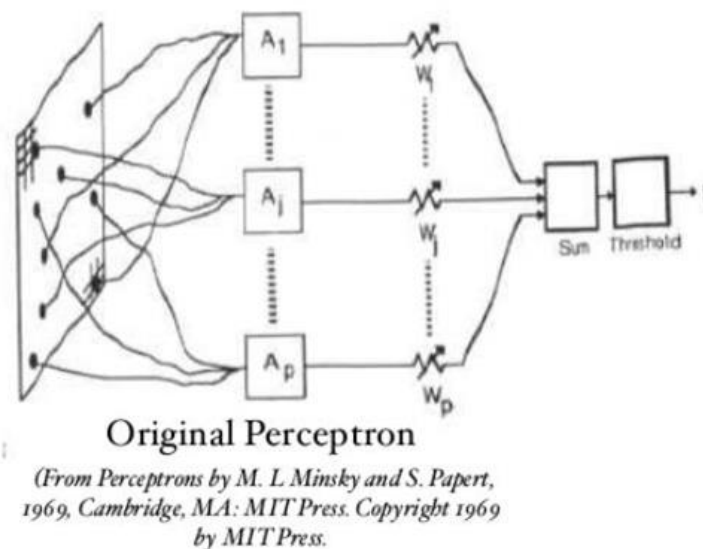
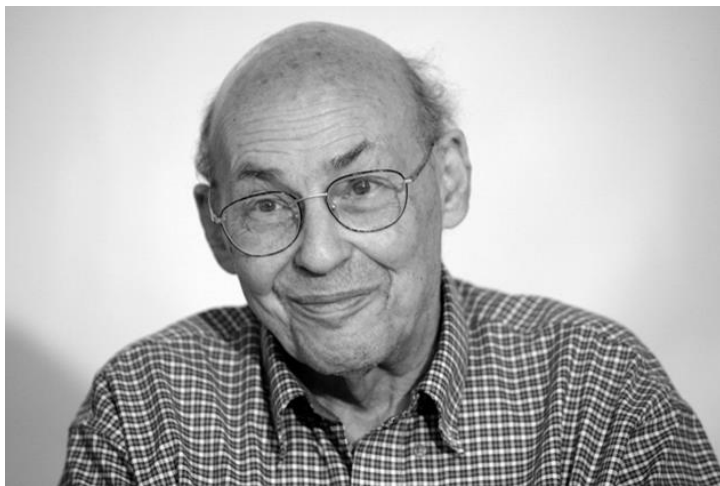
- ◆ 阶跃函数作为非线性激活函数
- ◆ 利用链式法则对误差进行传播以学习模型参数



4.1 深度学习概述：发展历史

➤ 感知器模型的理论研究 (Minsky, Papert, 1969)

- ◆ 感知器模型的局限性，例如不能解决简单异或（XOR）等线性不可分问题；发表了《Perceptron》一书，AI的第一个寒冬；



4.1 深度学习概述：发展历史

➤ BP神经网络算法(Werbos 1974; Rumelhart, Hinton, Williams, LeCun 1986)

◆ 利用链式法则对梯度进行求导

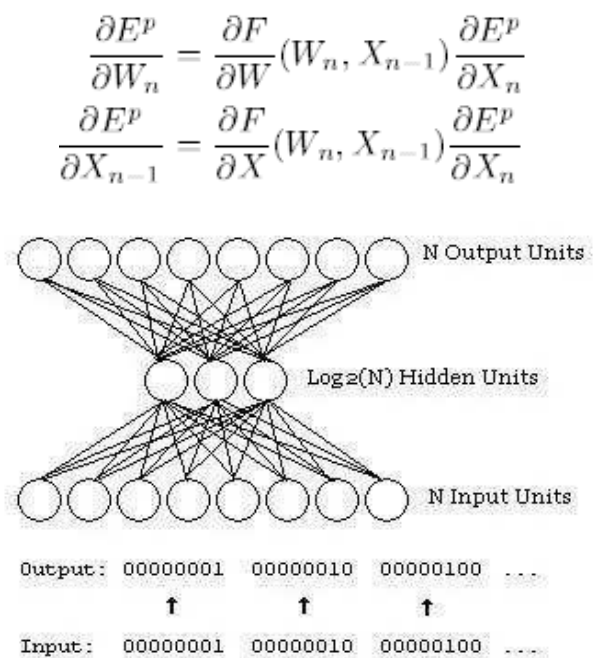
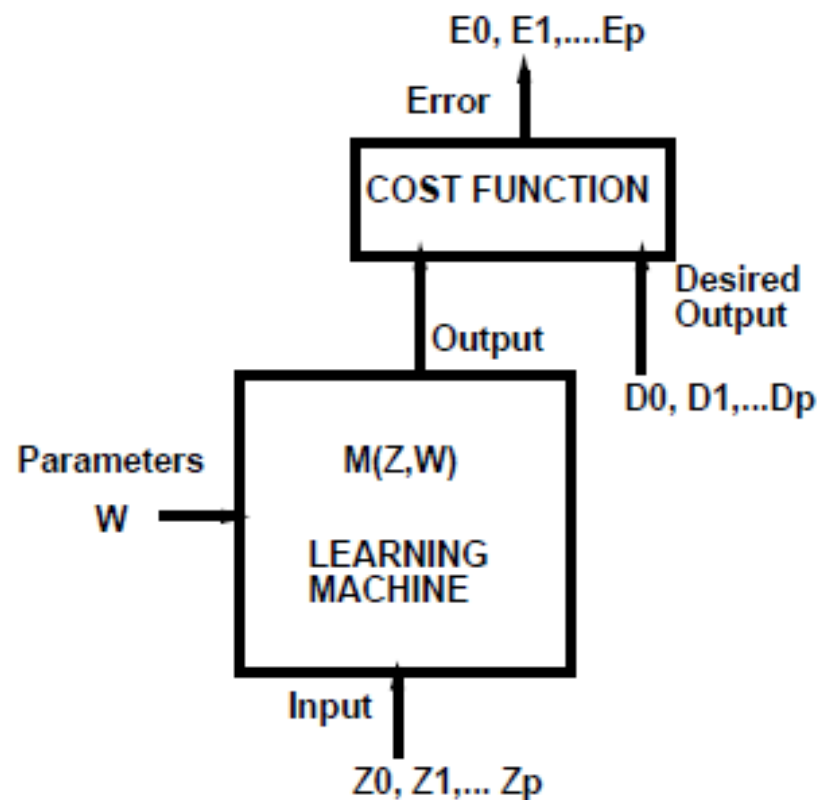


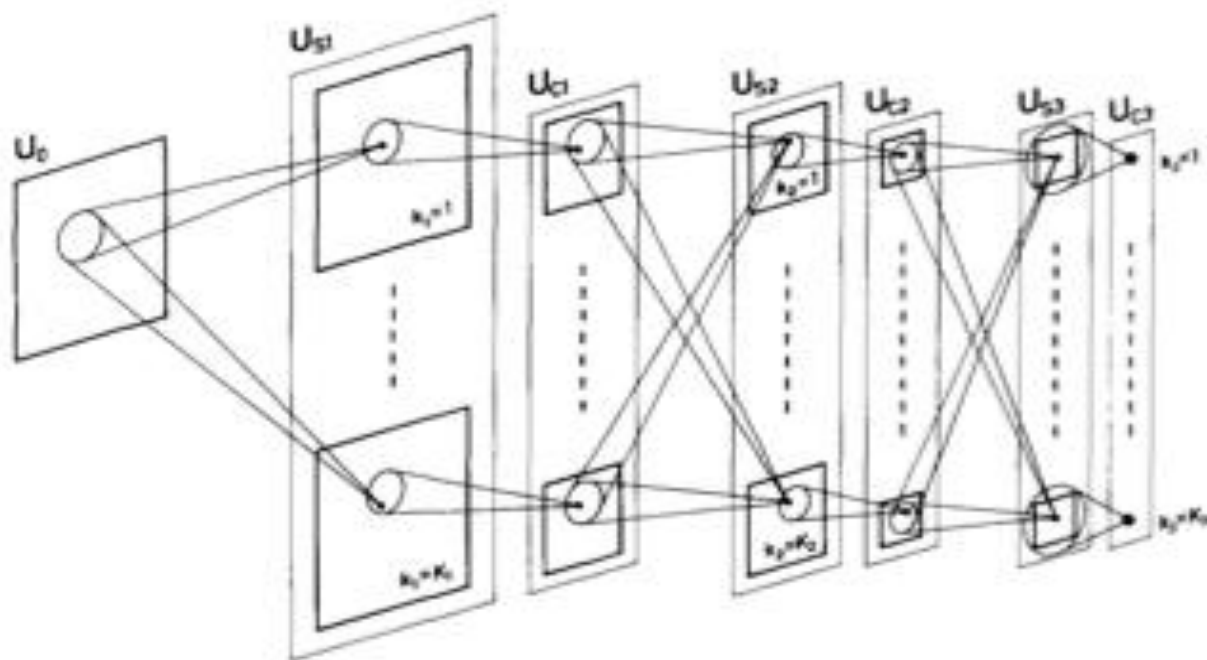
Figure 1. The encoding problem (Rumelhart, Hinton, & Williams, 1986)



4.1 深度学习概述：发展历史

➤ Neocognitron 模型 (Fukushima, 1980)

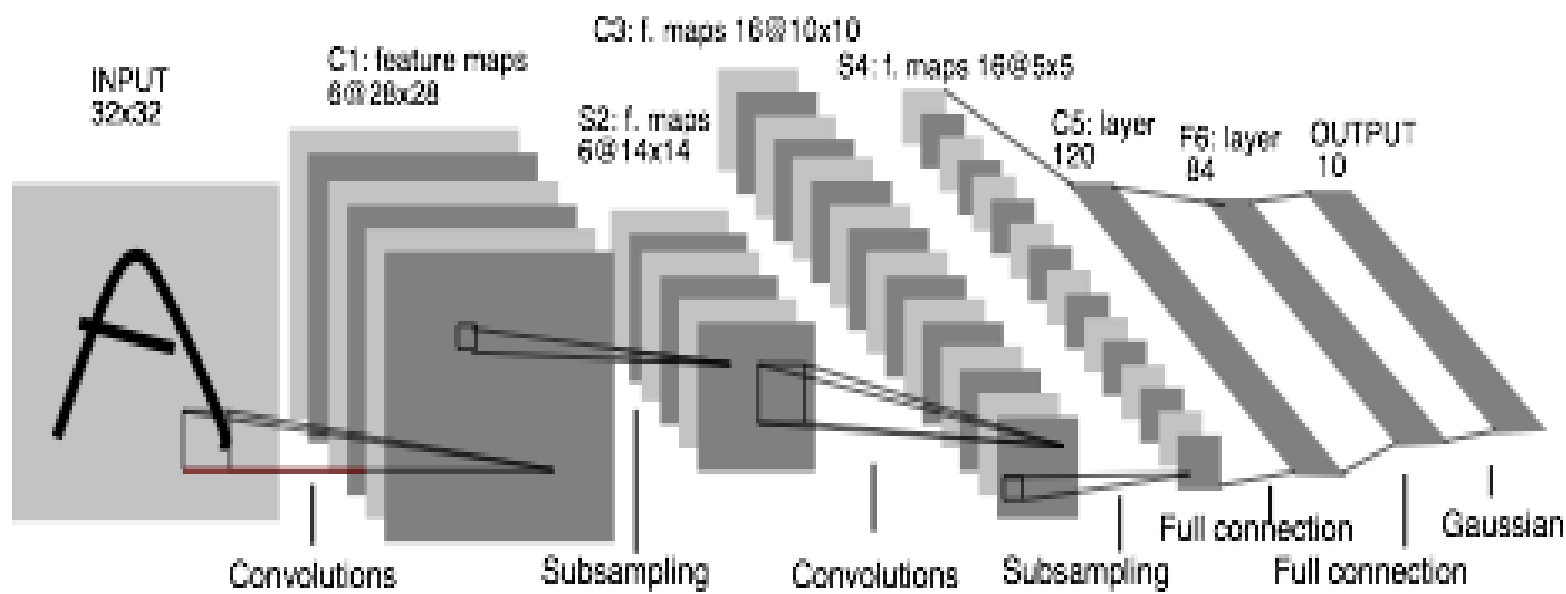
- ◆ 深度卷积神经网络；
- ◆ 受启发于Hubel / Wiesel 对猫大脑中的视觉系统研究；
- ◆ 将神经科学原理应用于工程的开创性研究。



4.1 深度学习概述：发展历史

➤ 卷积神经网络CNN (LeCun 1989, 1998)

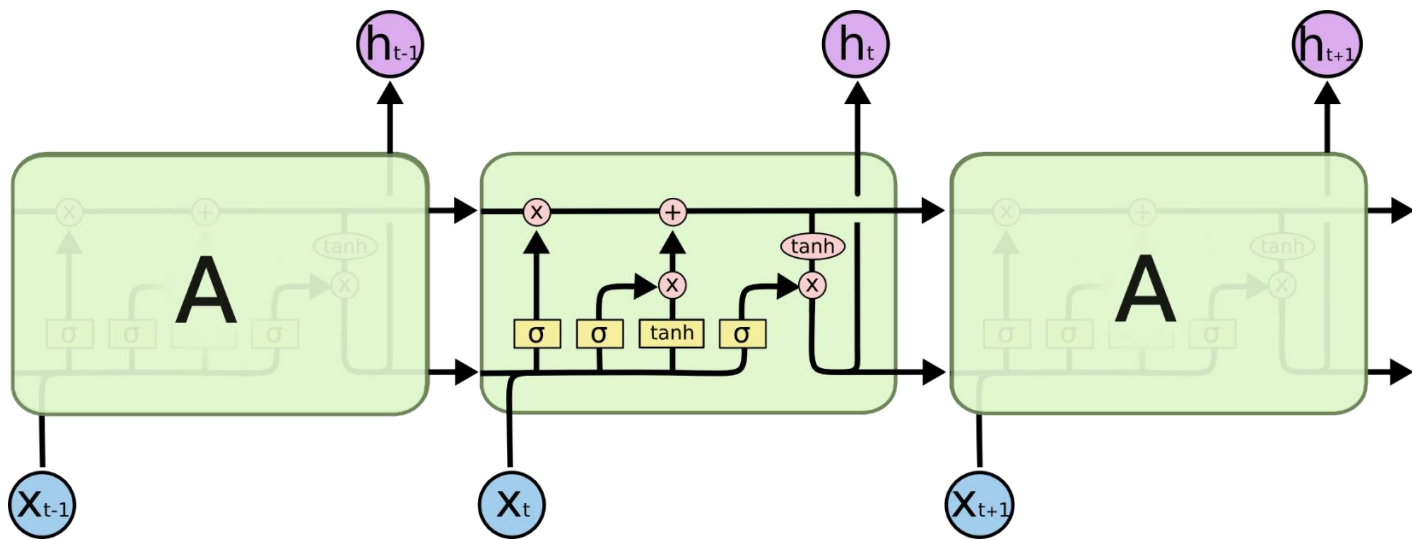
- ◆ 有监督反向传播算法 + 深度卷积神经网络
- ◆ LeNet: 应用于手写数字识别



4.1 深度学习概述：发展历史

➤ 长短时记忆网络 LSTM (Hochreiter, Schmidhuber, 1997)

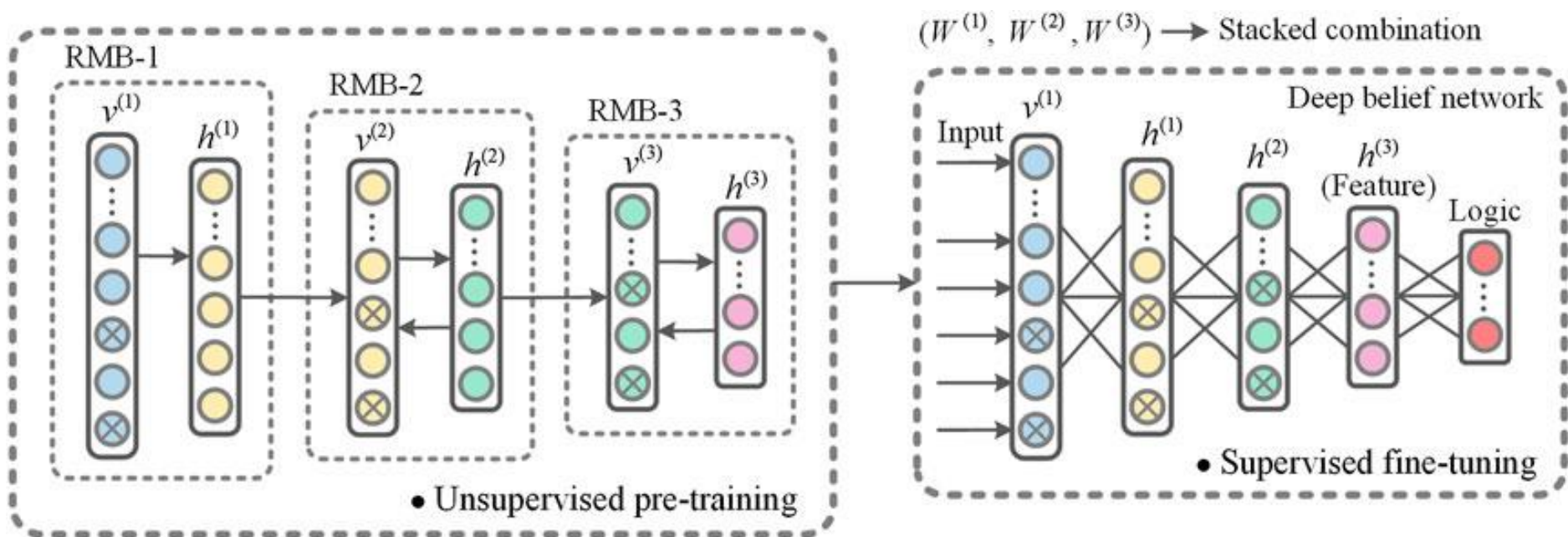
- ◆ 解决循环神经网络梯度消失问题
- ◆ 引入门控机制



4.1 深度学习概述：发展历史

➤ 深度置信网络DBN (Hinton, Osindero, 2006)

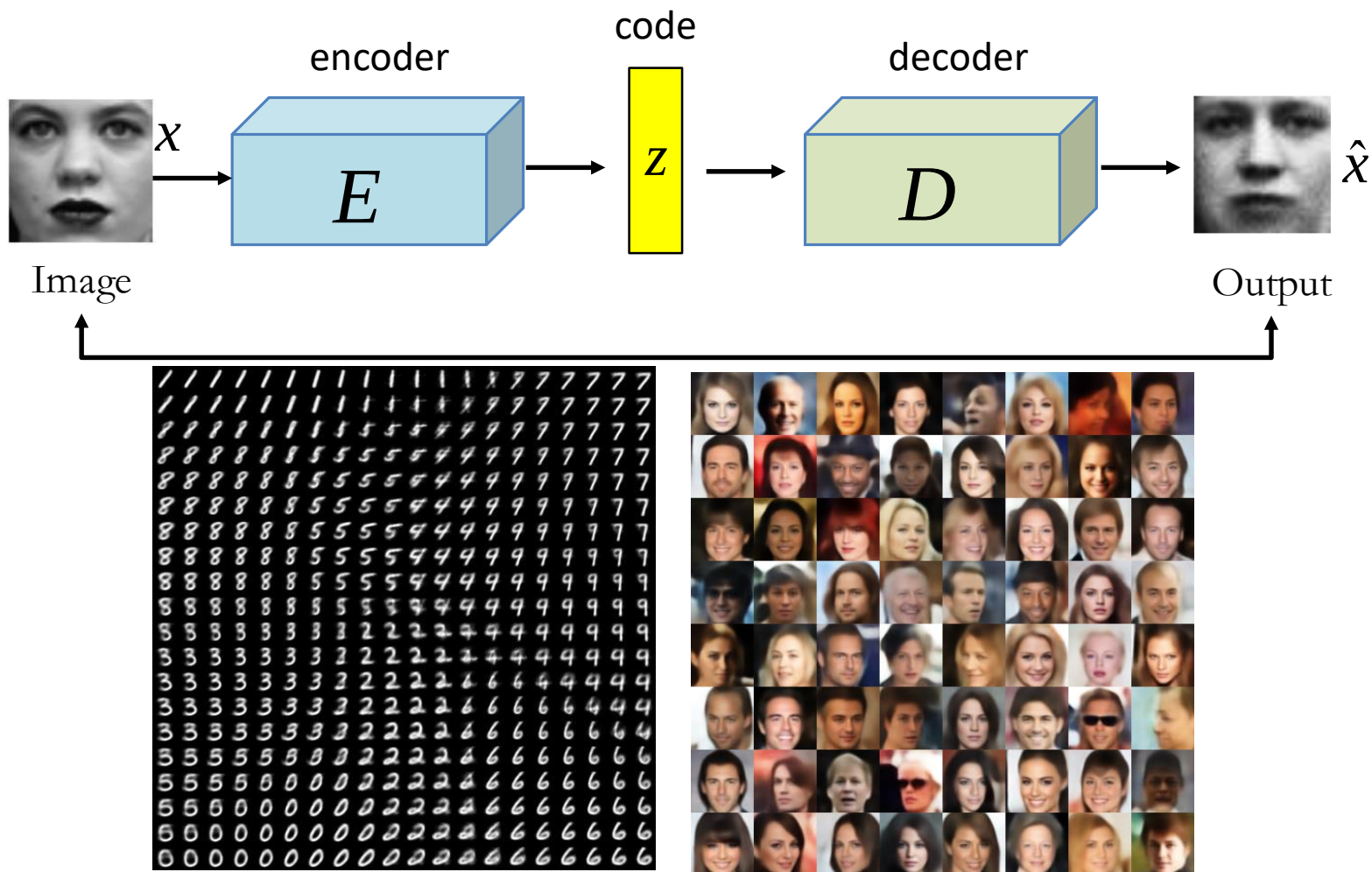
- ◆ 受限玻尔兹曼机
- ◆ unsupervised multilayer pre-training + supervised fine-tuning (BP)



4.1 深度学习概述：发展历史

➤ 自编码器 Auto encoder (Hinton, Salakhutdinov 2006)

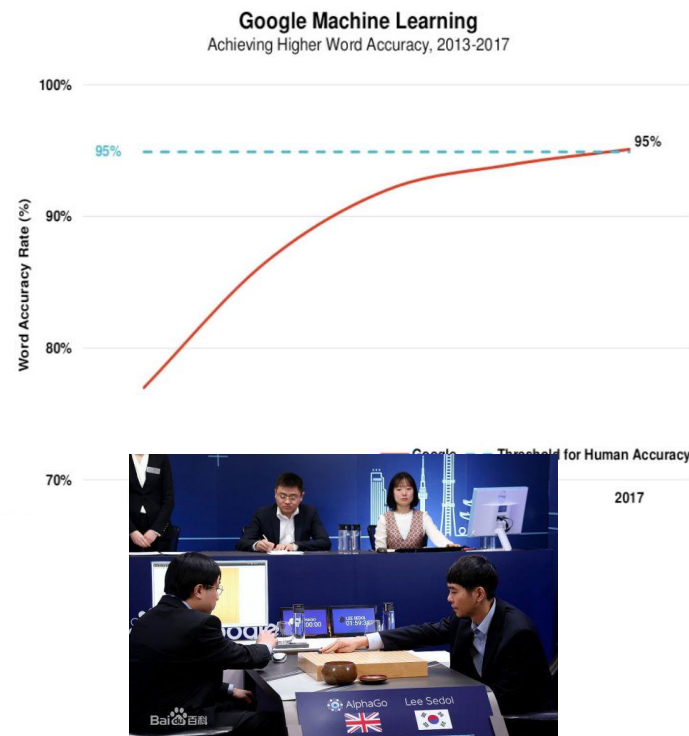
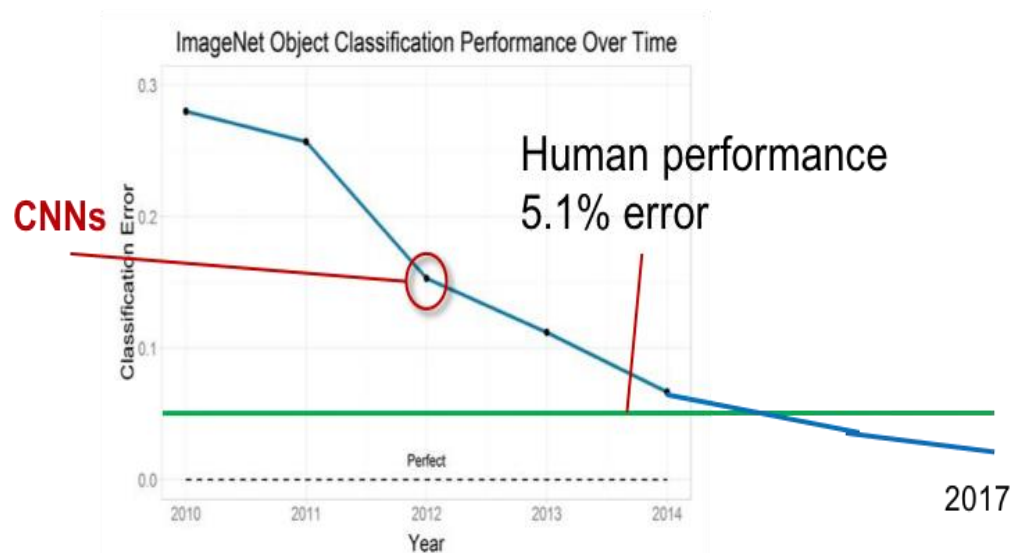
- ◆ 可应用于特征提取以及去噪



4.1 深度学习概述：发展历史

➤ 深度学习 Deep Learning (Hinton, 2007; LeCun, Bengio, Hinton 2015)

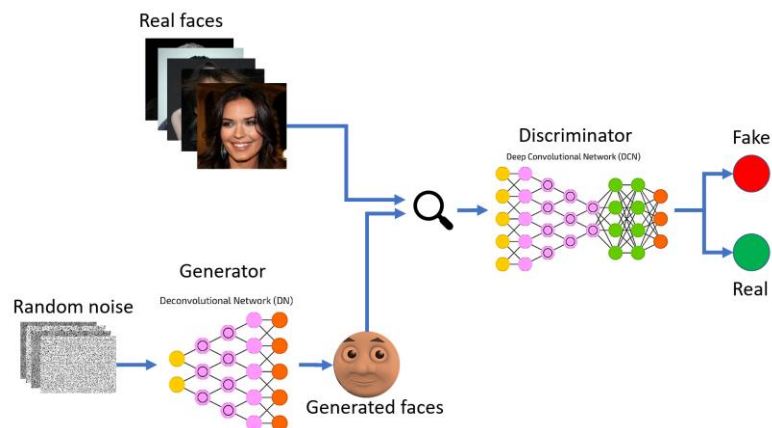
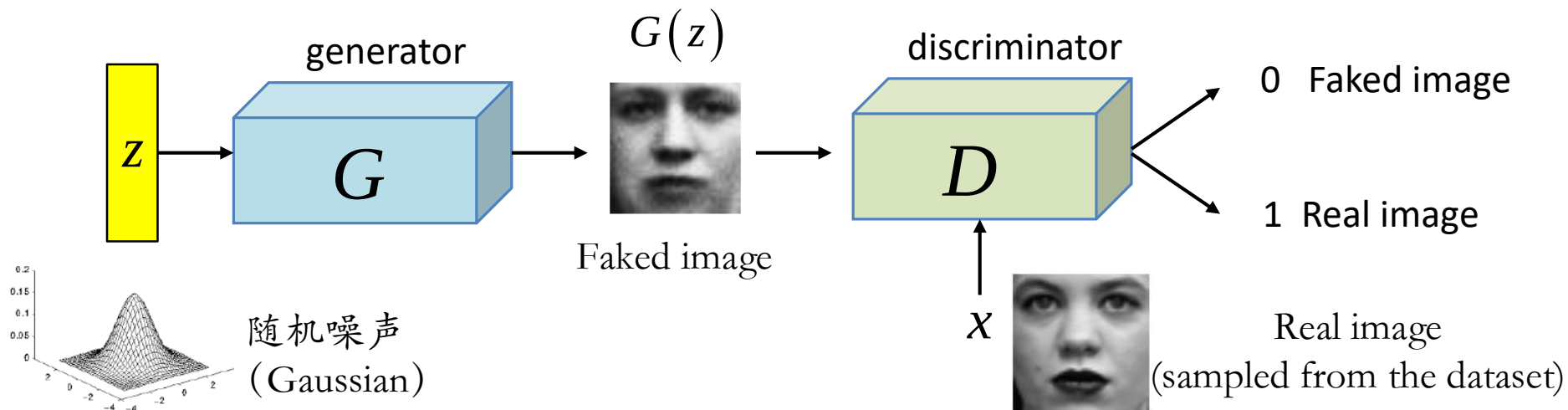
- ◆ Hinton. Learning multiple layers of representations. Trends in Cognitive Sciences, 2007.
- ◆ Yann LeCun, Yoshua Bengio & Geoffrey Hinton. Nature, 2015.



4.1 深度学习概述： 发展历史

➤ 生成式对抗网络 GAN (Goodfellow, 2014)

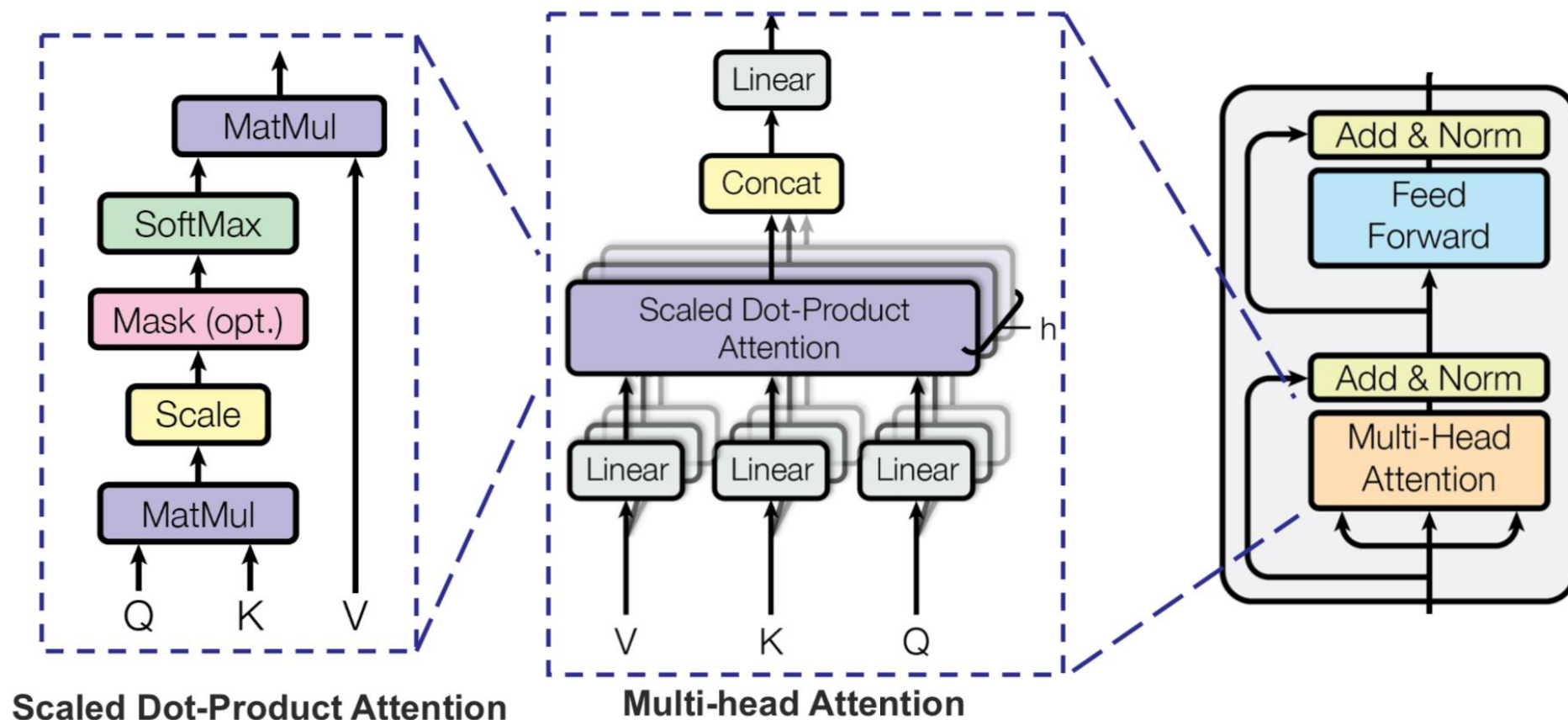
- ◆ 相互对抗的深度神经网络:生成器和判别器组成



4.1 深度学习概述：发展历史

➤ Self-attention & Transformer (Vaswani et al., 2017)

- ◆ 在机器翻译中，利用Attention机制计算每个单词与其他所有单词之间的关联



4.1 深度学习概述

2019年图灵奖(A. M. Turing Award)得主

Geoffrey Hinton



- Error Backpropagation
- 玻尔兹曼机
- 线性整流单元ReLU
- Dropout正则化
- AlexNet: ImageNet错误率50%相对下降
- Capsule Network

Yann LeCun



- 卷积神经网络CNN
- LeNet: 手写数字识别
- 反向传播算法backprop
- 神经网络转变成应用计算模型

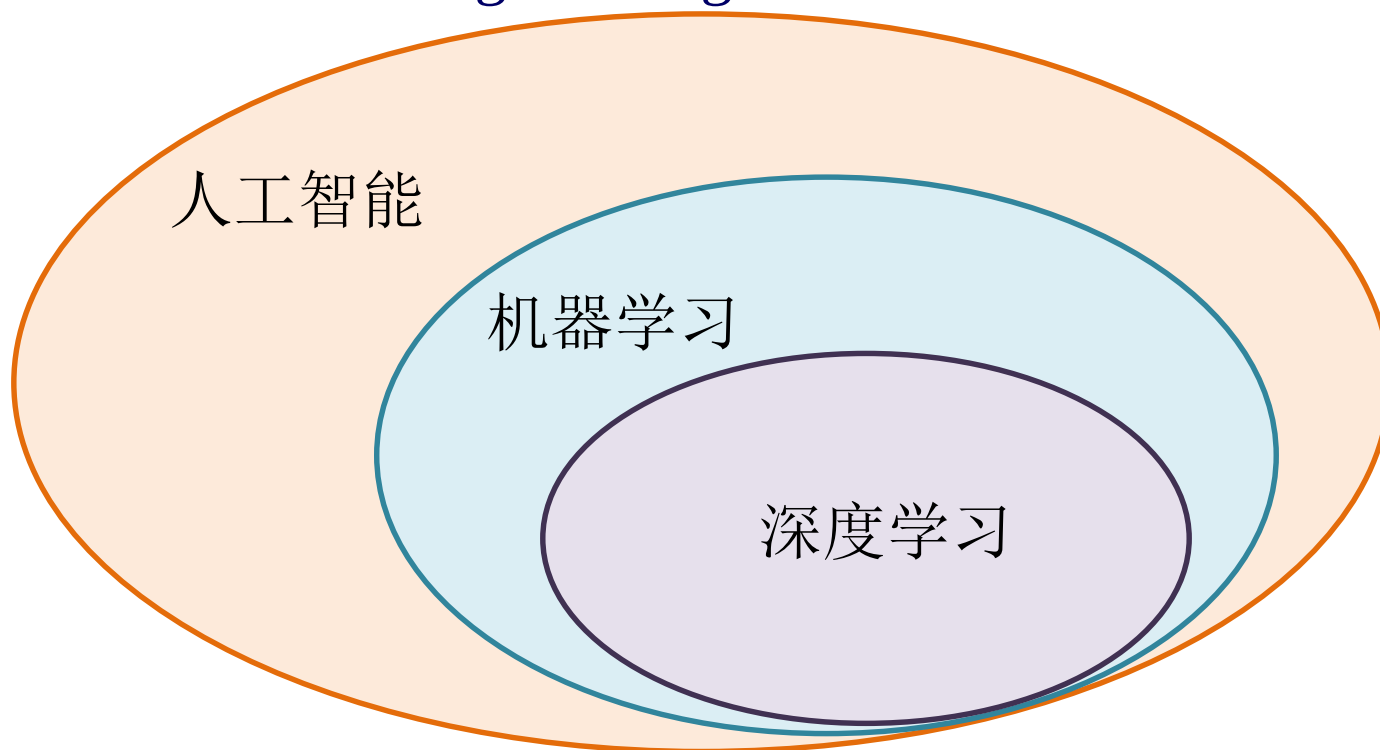
Yoshua Bengio



- 神经网络+概率模型
- 自然语言表征为高维词向量
- 注意力机制应用于机器翻译
- GAN

4.1 深度学习概述

- 胶囊网络 Capsule Network (Hinton, 2011)
- 自监督学习 Self-supervised learning
- 强化学习 Reinforcement learning
- 小样本学习/零样本学习/元学习 Few-shot / Zero-shot / Meta learning
- 终生学习 Lifelong learning

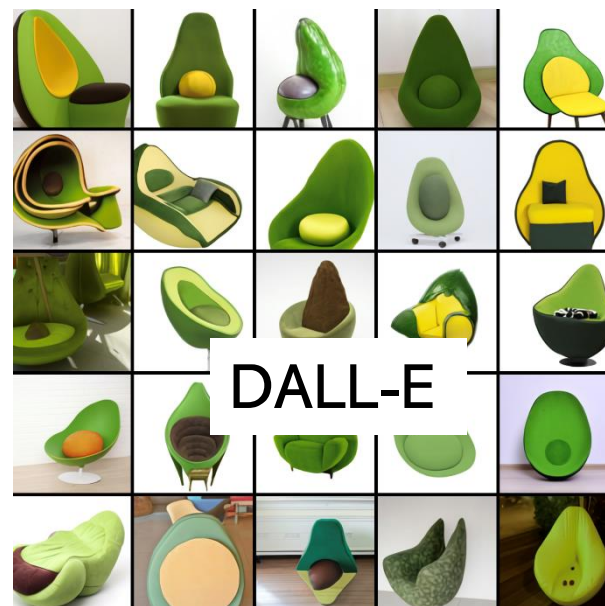
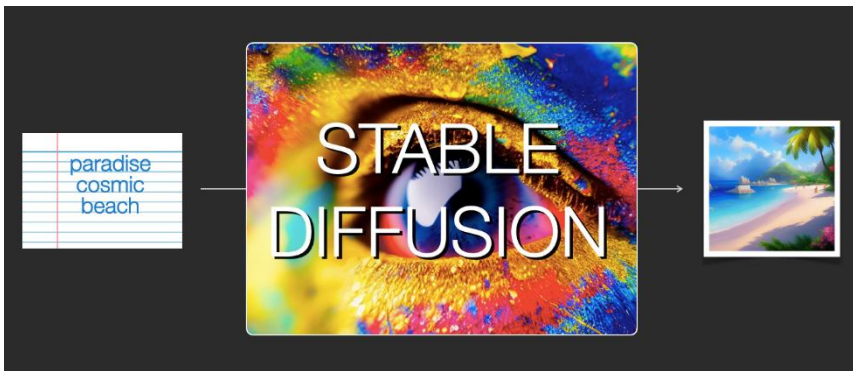
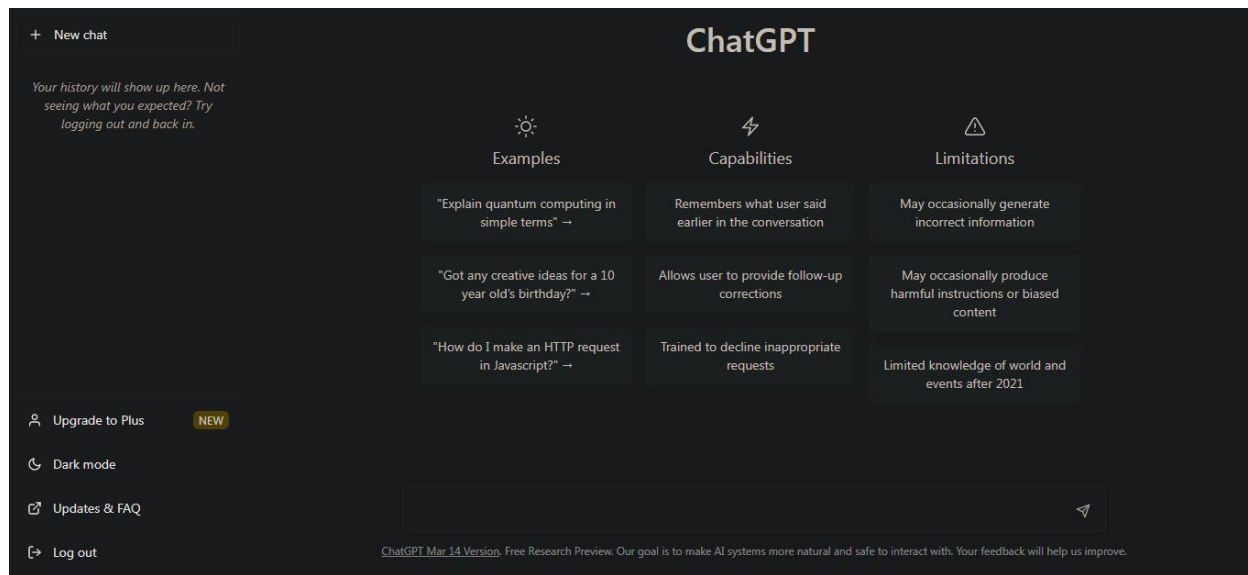


4.1 深度学习概述

大模型



ChatGPT

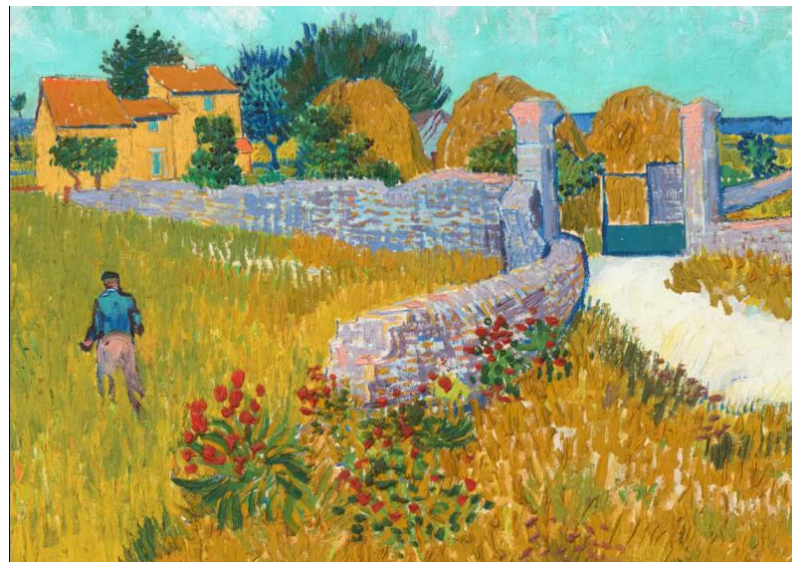


4.1 深度学习概述

SAM - Segment Anything Model

- Data Centric AI
- AI model from Meta AI that can "cut out" any object, in any image, with a single click

大
模
型

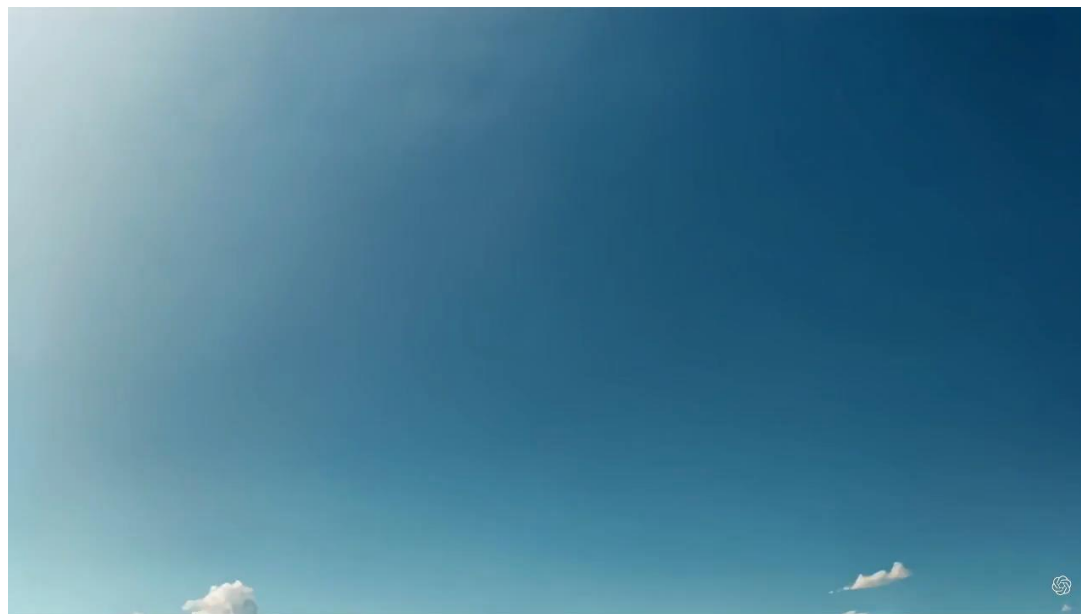


4.1 深度学习概述

Sora - generating a minute of high fidelity video

- scaling video generation models, developed from DALL-E
- towards building general purpose simulators of the physical world

大
模
型



第4讲 深度学习

4.1 深度学习概述

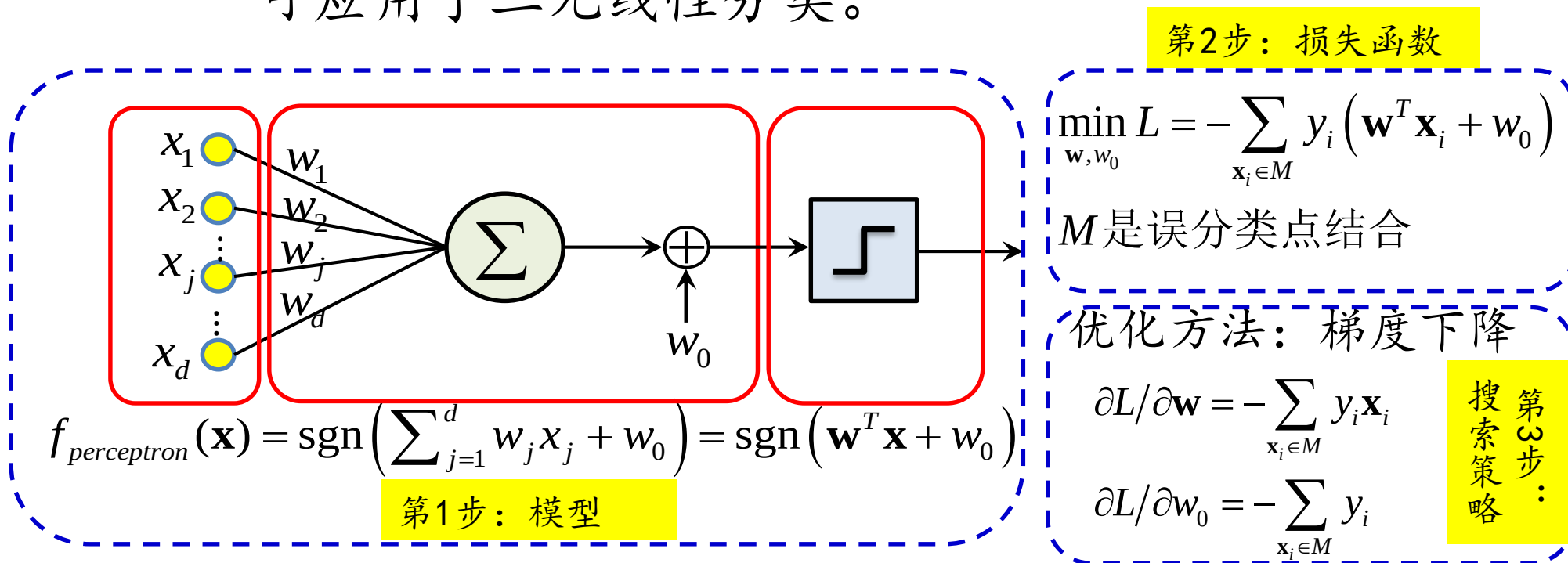
4.2 人工神经元模型

4.3 全连接前馈神经网络

4.2 人工神经元模型

➤ 回顾：线性分类器—感知器 (Perceptron)

- ◆ Frank Rosenblatt在1957年发明的一种人工神经网络。感知器是一种最简单形式的前馈神经网络，可应用于二元线性分类。

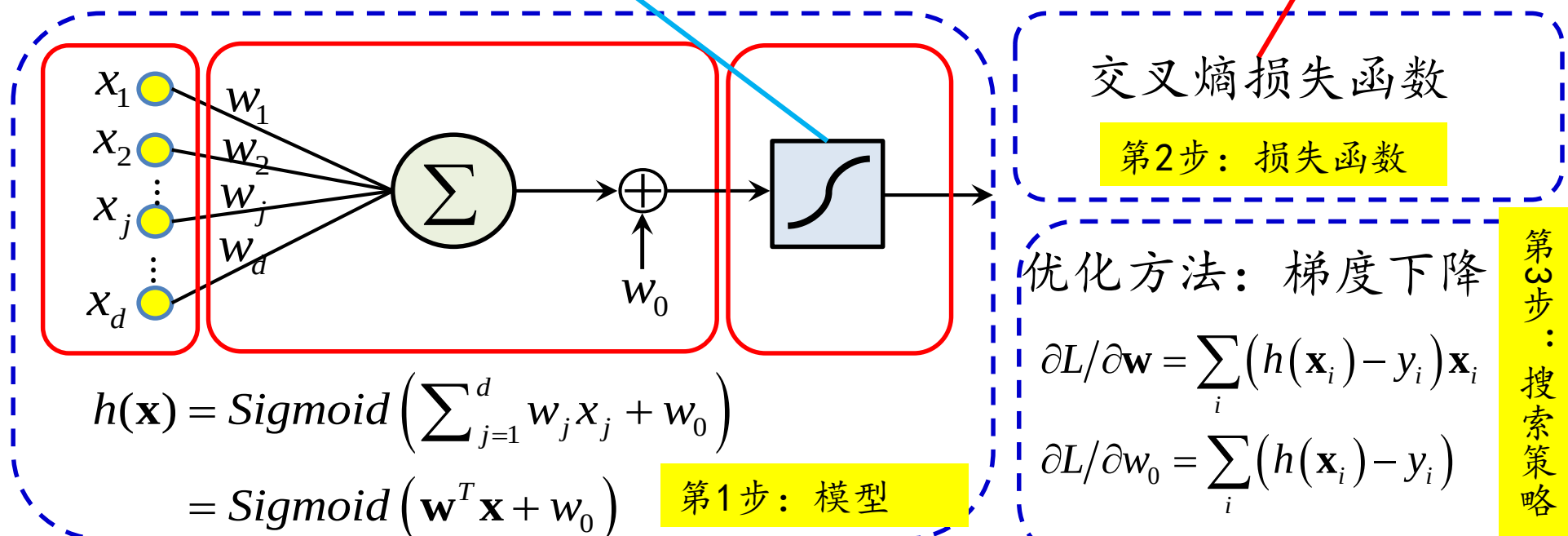


4.2 人工神经元模型

➤ 回顾：线性分类器--Logistic Regression

- ◆ 解决二分类问题的机器学习方法，用于估计某种事物的可能性。

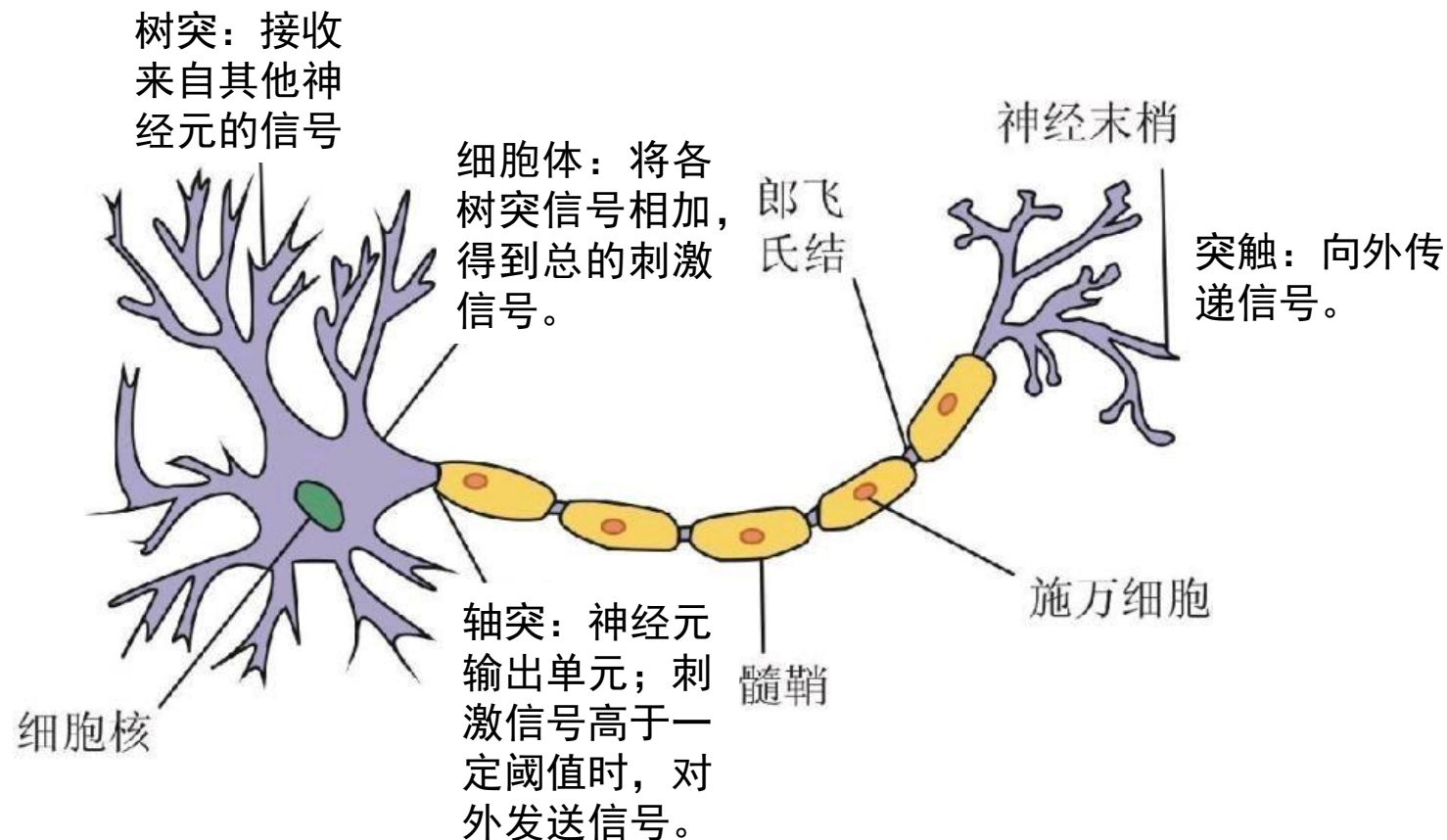
$$h(\mathbf{x}_i) = \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x}_i + w_0))} \quad \min_{\mathbf{w}, w_0} L = -\sum_i \left[y_i \log(h(\mathbf{x}_i)) + (1 - y_i) \log(1 - h(\mathbf{x}_i)) \right]$$



4.2 人工神经元模型

►生物学中的神经元：

- ◆ 人脑中大约有860亿个神经元；神经元之间相互连接，构成了复杂的神经系统；



4.2 人工神经元模型

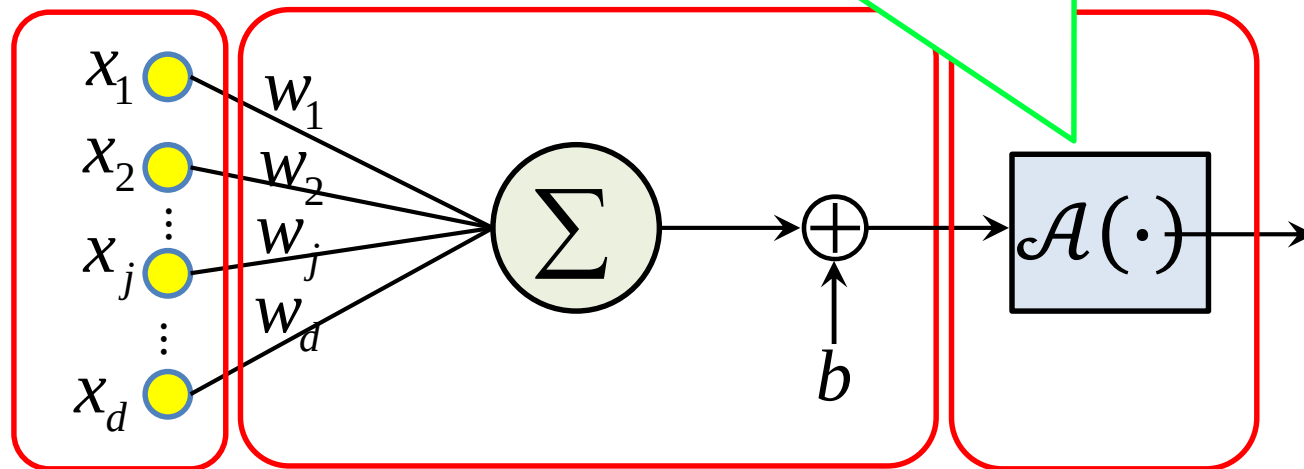
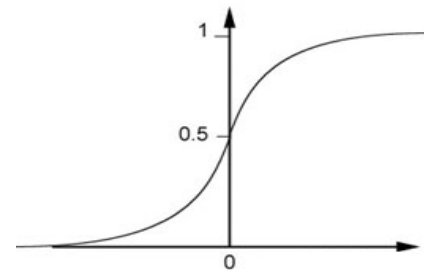
➤ 人工神经元模型(Neuron): 神经网络的基本单元

- ◆ 输入: 接受外界或者前层输入;
- ◆ 连接: 根据权重对输入加权;
- ◆ 激活函数: 连接层输出的非线性映射;
- ◆ 输出: 输出至下个隐含层。

$$f(\mathbf{x}) = \sum_{j=1}^d w_j x_j + b = \mathbf{w}^T \mathbf{x} + b$$

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



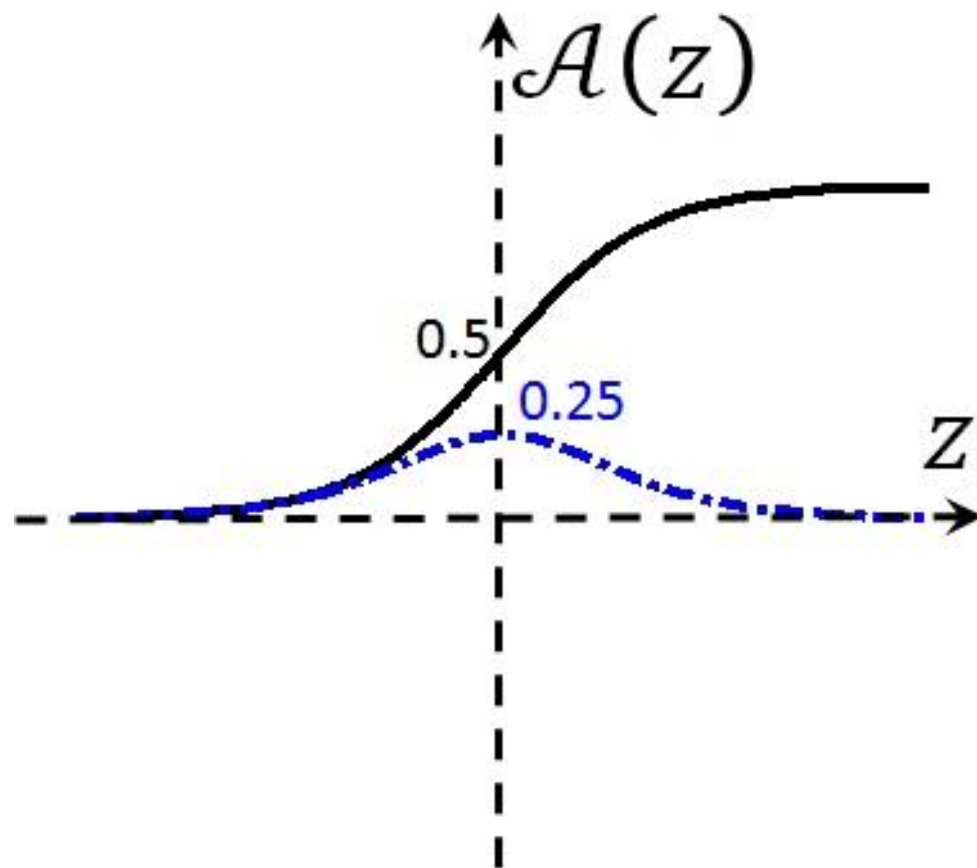
4.2.1 激活函数(Activation Function)

➤ Sigmoid激活函数

$$A(z) = \frac{1}{1 + e^{-z}}$$

$$\begin{aligned} A'(z) &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= A(z)(1 - A(z)) \end{aligned}$$

- ◆ 非零中心;
- ◆ 饱和区域梯度消失;
- ◆ 通常应用DNN最后一层。



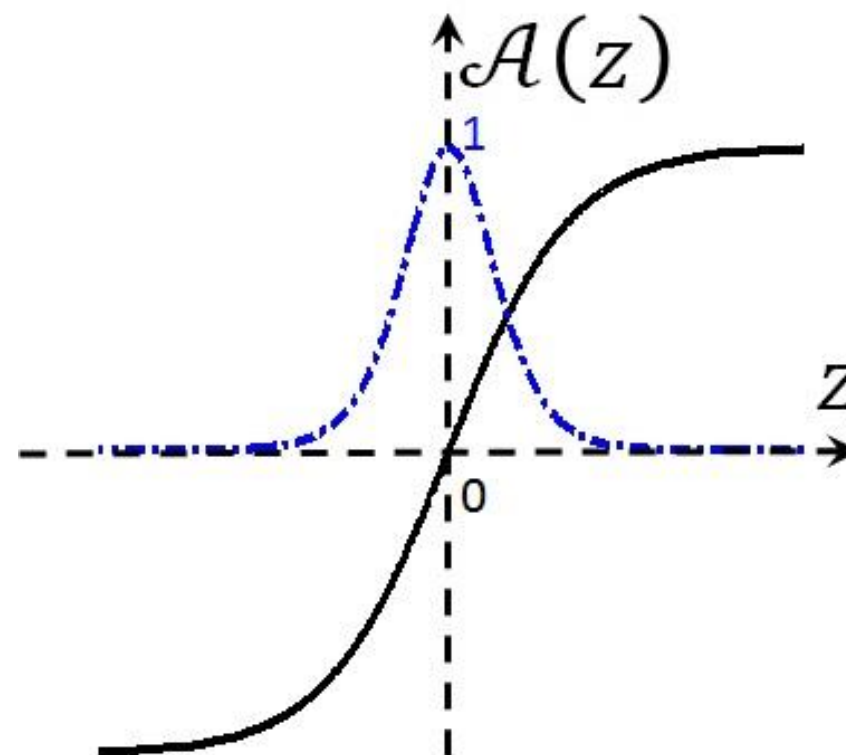
4.2.1 激活函数(Activation Function)

➤ Tanh双曲正切激活函数

$$A(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$A'(z) = \frac{4e^{-2z}}{(1 + e^{-2z})^2}$$

- ◆ 零中心;
- ◆ 饱和区域梯度消失;
- ◆ 通常应用RNN (循环神经网络)

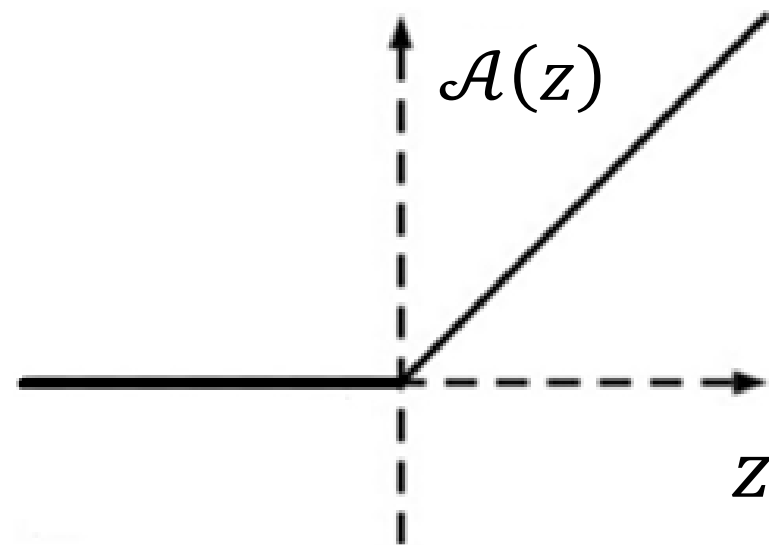


4.2.1 激活函数(Activation Function)

► ReLU激活函数

$$A(z) = \max(0, z)$$

$$A'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$



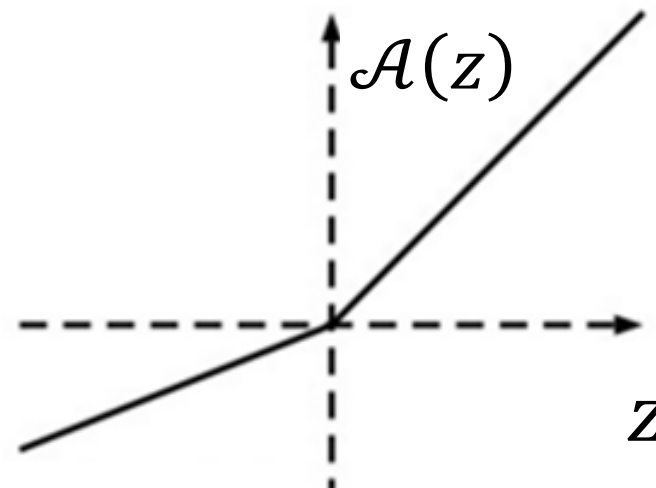
- ◆ 正值部分梯度有效传递，有效解决梯度消失；
- ◆ 非零中心；
- ◆ “dead neurons”

4.2.1 激活函数(Activation Function)

➤ Leaky ReLU/PReLU激活函数

$$A(z) = \begin{cases} z & z \geq 0 \\ \alpha z & z < 0 \end{cases}$$

$$A'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z < 0 \end{cases}$$



α 为小常数(Leaky ReLU);
可为随机变量 (RReLU);
也可作为参数在网络中进行学习 (PReLU)。

◆ 特点：部分解决负值信息截断；

4.2.1 激活函数(Activation Function)

➤ ELU/GELU 激活函数

$$A_{ELU}(z) = \begin{cases} z & z \geq 0 \\ \alpha(\exp(z) - 1) & z < 0 \end{cases}$$

*Cumulative Distribution Function
for Gaussian Distribution*

$$A_{GELU}(z) = z \cdot \phi(z)$$

$$A_{GELU}(z) = 0.5z \left(1 + \tanh \left(\sqrt{2/\pi} \left(z + 0.044715z^3 \right) \right) \right)$$

Approximation with Tanh

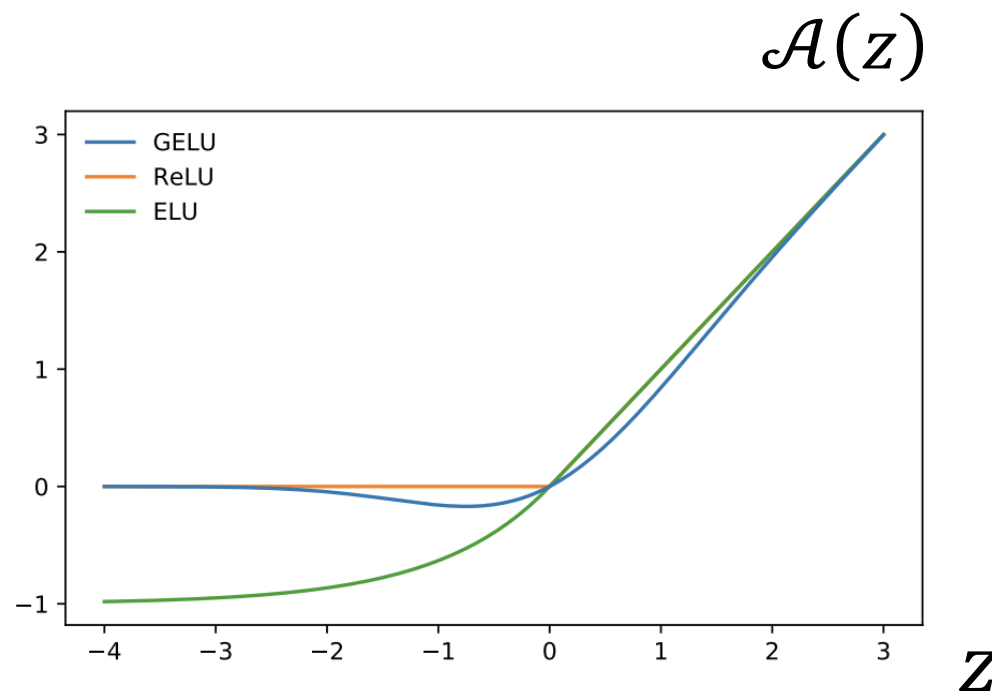
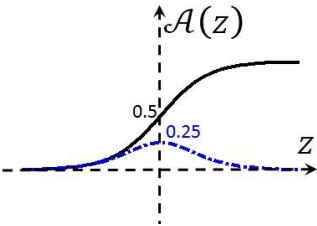
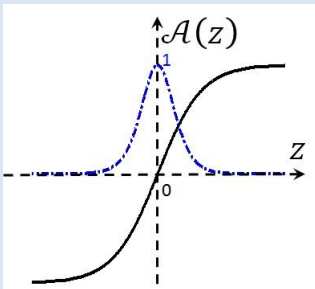
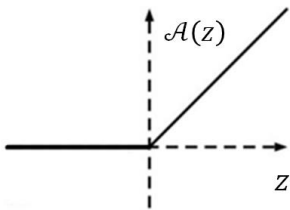
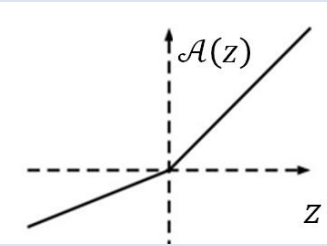


Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

4.2.1 激活函数(总结)

Sigmoid	$A(z) = \frac{1}{1 + e^{-z}}$	$A'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}$		1 非零中心; 2 饱和区域梯度消失; 3 通常应用DNN最后一层
Tanh双曲正切	$A(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$A'(z) = \frac{4e^{-2z}}{(1 + e^{-2z})^2}$		1 零中心; 2 饱和区域梯度消失; 3 应用于RNN
ReLU	$A(z) = \max(0, z)$	$A'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$		1 正值部分梯度有效传递, 有效解决梯度消失; 2 非零中心; 3 “dead neurons”
Leaky ReLU / PReLU	$A(z) = \begin{cases} z & z \geq 0 \\ \alpha z & z < 0 \end{cases}$ α为小常数; 可为随机变量; 可在网络中进行学习。	$A'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z < 0 \end{cases}$		部分解决负值信息截断;

4.2.2 输出单元

➤ 输出单元:

- ◆ 某种意义上, 可认为是输出层的激活函数
- ◆ 回归: 直接输出/线性单元
- ◆ 分类: 概率转换--Sigmoid/Softmax
- ◆ Softmax:

$$p_j = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

输出向量和为1, 对应于多类别的概率输出。

4.2.2 输出单元

➤ 输出单元:

◆ Softmax:
$$p_j = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

输出向量和为1，对应于多类别的概率输出。

- ◆ 例1：现利用一深度网络完成一个分类任务，类别集合为{1,2,3}，采用Softmax输出单元，其输入为 $z_1 = 3, z_2 = 1, z_3 = -3$ ，则Softmax单元输出是？

◆ 求解：

类似于皮质中神经元的侧抑制：赢者通吃

$$p_1 = e^{z_1} / \sum_{k=1}^3 e^{z_k} = e^3 / (e^3 + e^1 + e^{-3}) = 0.88$$

属于第1类的概率为0.88

$$p_2 = e^{z_2} / \sum_{k=1}^3 e^{z_k} = e^1 / (e^3 + e^1 + e^{-3}) = 0.12$$

属于第2类的概率为0.12

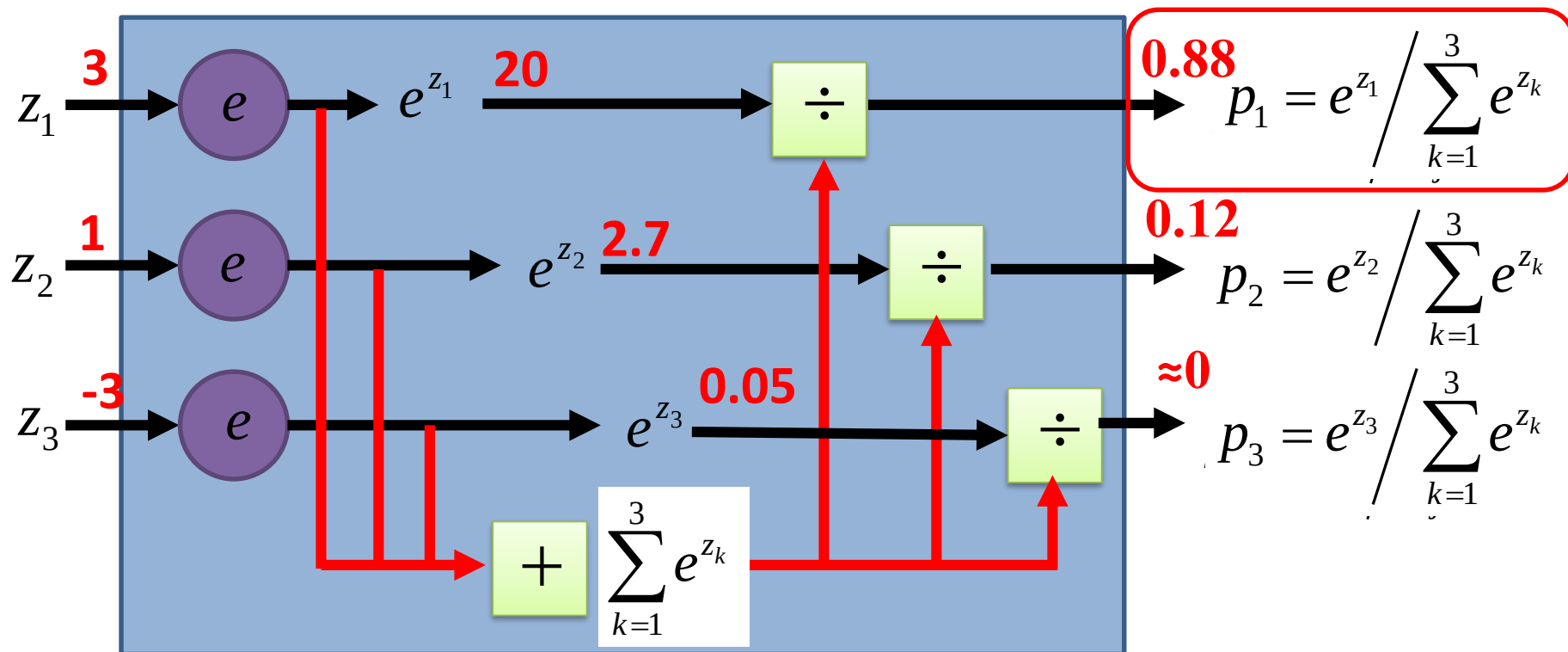
$$p_3 = e^{z_3} / \sum_{k=1}^3 e^{z_k} = e^{-3} / (e^3 + e^1 + e^{-3}) \approx 0$$

属于第3类的概率约为0.

4.2.2 输出单元

➤ 输出单元:

◆ Softmax: $p_j = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$



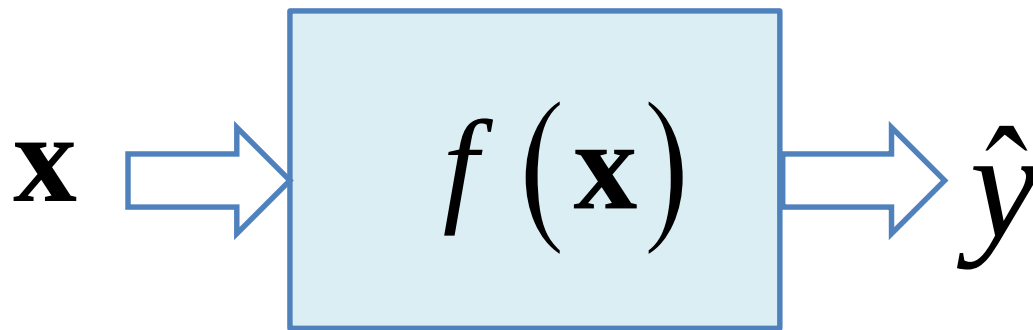
4.2.3 优化准则：损失函数

➤ 损失函数(Loss Function)

- ◆ 衡量模型预测值和真实值之间的差异, 以评价模型优劣
- ◆ 在监督学习的训练阶段指导模型优化
- ◆ 一般数学形式:

$$L(\textcircled{y}, \textcircled{\hat{y}=f(\mathbf{x})})$$

真实值(人工标定获取) 预测值(模型输出)



4.2.3 优化准则：损失函数

➤ 损失函数(Loss Function):

◆ 分类问题：Softmax损失函数(交叉熵损失)

$$L = - \sum_i \log \left(\frac{\exp(z_{i,j})}{\sum_k \exp(z_{i,k})} \right)$$

j : 第 i 个样本类别标签

$\{z_{i,k}\}$ 输出的预测向量

Softmax映射至多类别概率分布，适用于多类别单标签分类问题

假设有 K 个类别，某样本输出 K 维向量 $z_i = [z_{i,1}, z_{i,2}, \dots, z_{i,K}]^T$ ，则其属于第 j 类的概率为 $\hat{y}_{i,j} = \exp(z_{i,j}) / \sum_{k=1}^K \exp(z_{i,k})$ ；若样本的真实类别标签为第 j 类，则 $\hat{y}_{i,j}$ 越大越接近于1，则损失越小越接近于0；反之损失越大。

4.2.3 优化准则：损失函数

➤ 损失函数(Loss Function):

◆ 分类问题：Softmax损失函数(交叉熵损失)

$$L = - \sum_i \log \left(\frac{\exp(z_{i,j})}{\sum_k \exp(z_{i,k})} \right)$$

j : 第 i 个样本类别标签

$\{z_{i,k}\}$ 输出的预测向量

Softmax Activation + negative log-likelihood Loss

假设有 K 个类别，某样本输出 K 维向量 $z_i = [z_{i,1}, z_{i,2}, \dots, z_{i,K}]^T$ ，则其属于第 j 类的概率为 $\hat{y}_{i,j} = \exp(z_{i,j}) / \sum_{k=1}^K \exp(z_{i,k})$ ；若样本的真实类别标签为第 j 类，则该类别标签对应的one-hot编码为 $[0, \dots, 1, \dots, 0]^T$ (K 维向量, 仅第 j 位为1, 其余皆为0)，将其作为 $\{y_{i,k}\}$ 代入至负对数似然损失 $L_i = - \sum_k y_{ik} \log(\hat{y}_{ik})$ 。

4.2.3 优化准则：损失函数

► 损失函数(Loss Function):

- ◆ 分类问题：Softmax损失函数(交叉熵损失)

$$L = - \sum_i \log \left(\frac{\exp(z_{i,j})}{\sum_k \exp(z_{i,k})} \right)$$

j : 第 i 个样本类别标签

$\{z_{i,k}\}$ 输出的预测向量

✓ 对数似然 $\log$ 抵消 $\exp$ 运算: $L_i = -z_{i,j} + \log \left(\sum_k \exp(z_{i,k}) \right)$

$$\frac{\partial L}{\partial z_{i,k}} = \begin{cases} \hat{y}_{i,k} - 1 & k = c_i \\ \hat{y}_{i,k} & otherwise \end{cases}, \hat{y}_{i,j} = \frac{\exp(z_{i,j})}{\sum_k \exp(z_{i,k})}$$

惩罚最活跃的不正确预测

4.2.3 优化准则：损失函数

➤ 损失函数(Loss Function):

◆ 分类问题：Softmax损失函数(交叉熵损失)

$$L = - \sum_i \log \left(\frac{\exp(z_{i,j})}{\sum_k \exp(z_{i,k})} \right)$$

例2：现利用一深度网络完成一个分类任务，类别集合为{1, 2, 3}，网络最后一层为具有3个神经元节点的全连接层，采用softmax作为激活函数，试问训练时将一个真实类别为1的样本送入神经网络，全连接层softmax激活输入为 $z_1 = 3, z_2 = 1, z_3 = -3$ ，请问损失函数输出是多少？

求解：

$$L = -\log \left(e^{z_1} / \sum_{k=1}^3 e^{z_k} \right) = -\log(0.88) = 0.1278$$

以e为底

4.2.3 优化准则：损失函数

➤ 损失函数(Loss Function):

◆ 回归问题:

(1) 均方损失 (Square Loss/ Euclidean Loss)

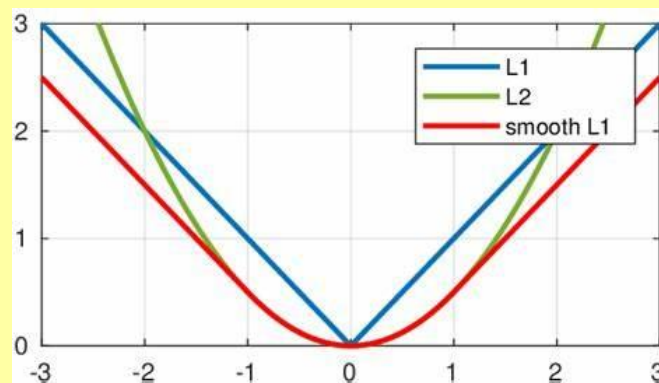
$$L = \frac{1}{2} \sum_i \|y_i - \hat{y}_i\|^2 \quad \hat{y}_i = h(\mathbf{x}_i)$$

$h(x_i)$ 是模型对样本的预测输出； y_i 是样本实际值

(2) L1损失 $L = \sum_i |y_i - \hat{y}_i|$

(3) Smooth-L1损失

$$L_i = \begin{cases} 0.5(y_i - \hat{y}_i)^2 & |y_i - \hat{y}_i| < 1 \\ |y_i - \hat{y}_i| - 0.5 & \text{otherwise} \end{cases}, L = \sum_i L_i$$



第4讲 深度学习

4.1 深度学习概述

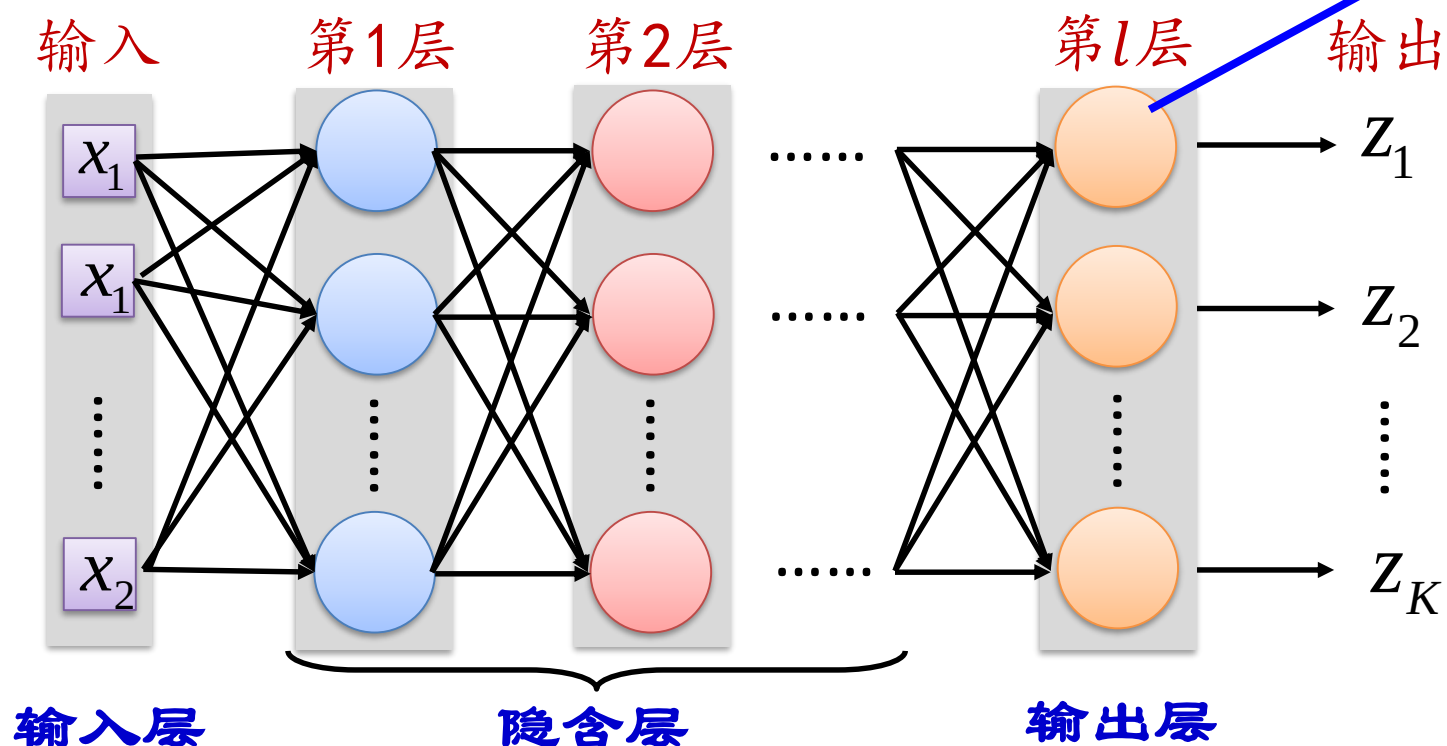
4.2 人工神经元模型

4.3 全连接前馈神经网络

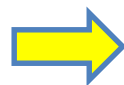
4.3 多层感知器模型

➤ 全连接前馈神经网络

(Fully-connected FeedForward Networks, FFN)



多个隐含层累积

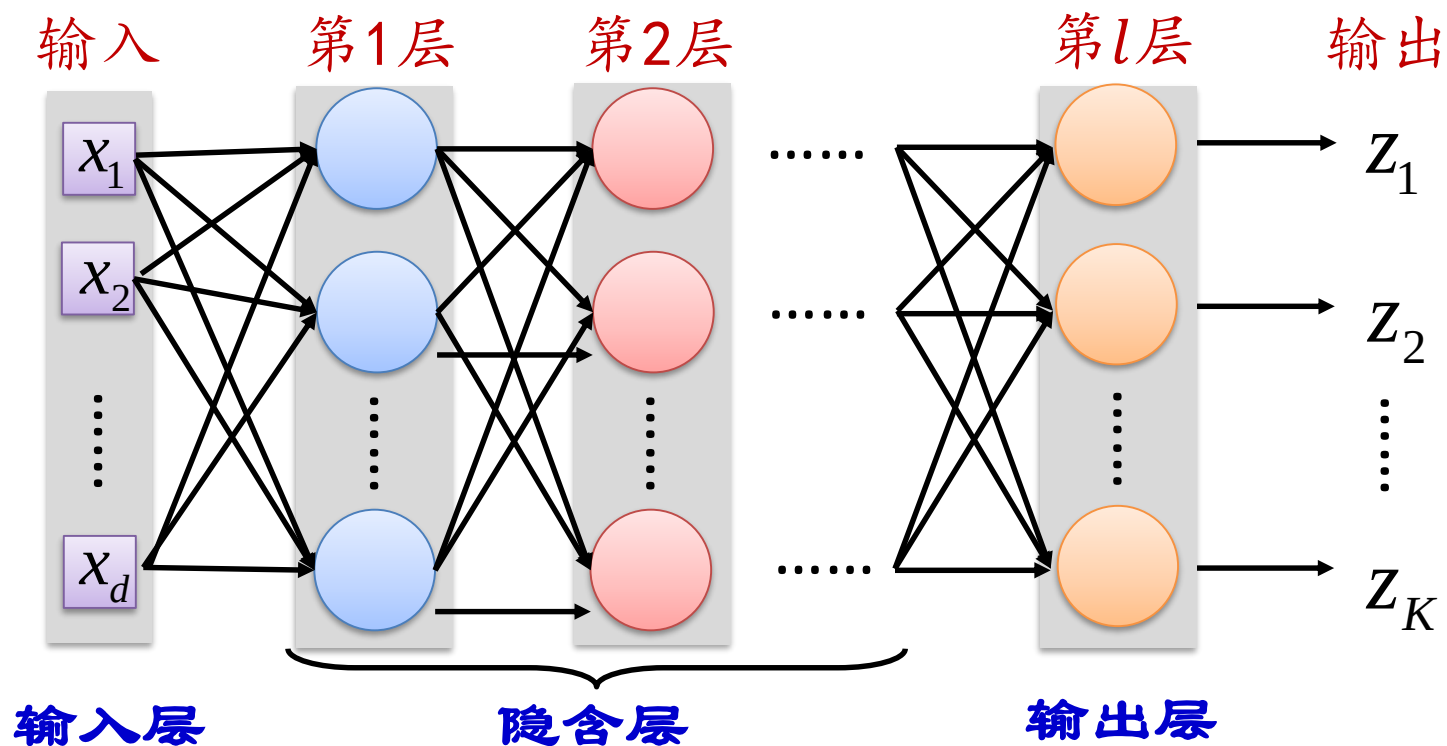


深度神经网络

4.3 多层感知器模型

➤ 全连接前馈神经网络

(Fully-connected FeedForward Networks, FFN)

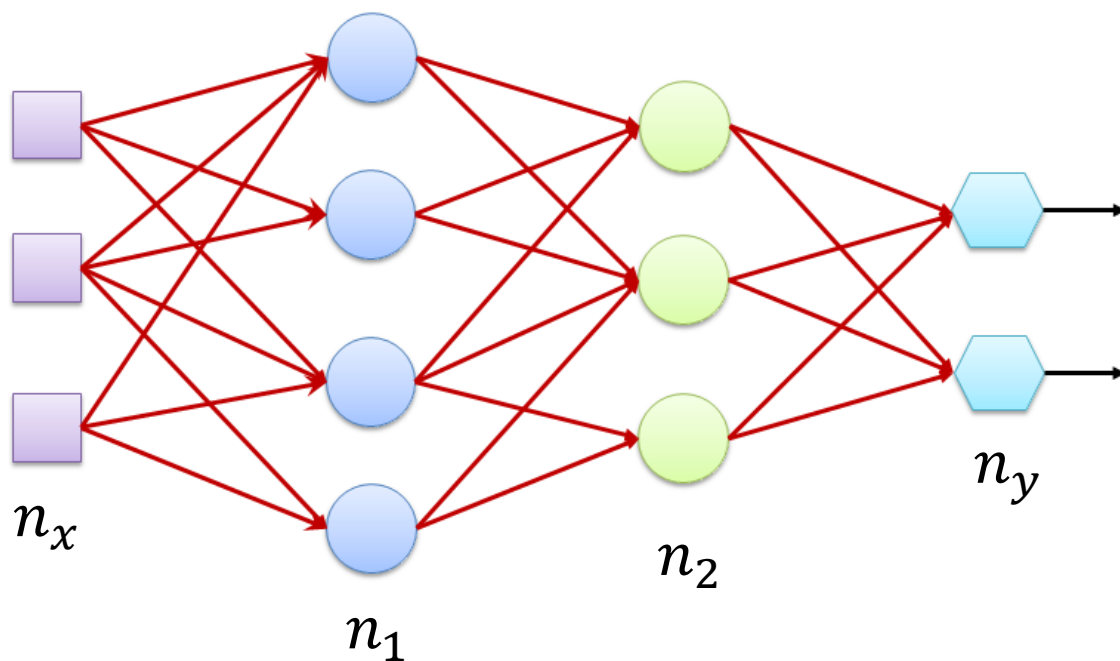


在网络结构中，每个神经元有对应权重和偏置参数；
所有神经元的权重和偏置定义为**网络参数**。

4.3 多层感知器模型

➤ 全连接前馈神经网络

例3：全连接前馈神经网络参数数量估计



第1层:

权重参数量: $n_x n_1$

偏置参数量: n_1

第2层:

权重参数量: $n_1 n_2$

偏置参数量: n_2

输出层:

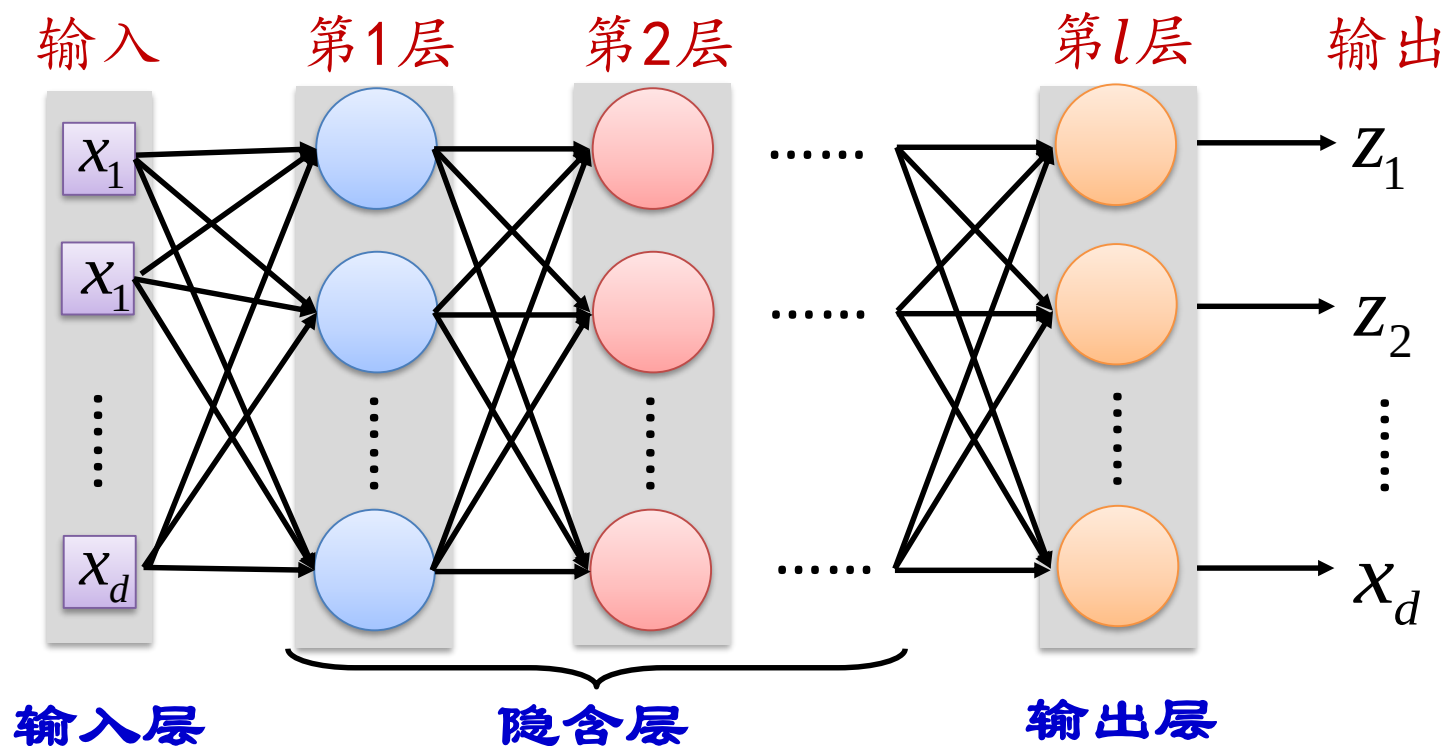
权重参数量: $n_2 n_y$

偏置参数量: n_y

4.3 多层感知器模型

➤ 全连接前馈神经网络

(Fully-connected FeedForward Networks, FFN)



给定网络模型，即定义了函数集合；
给定网络参数，即定义了函数映射；



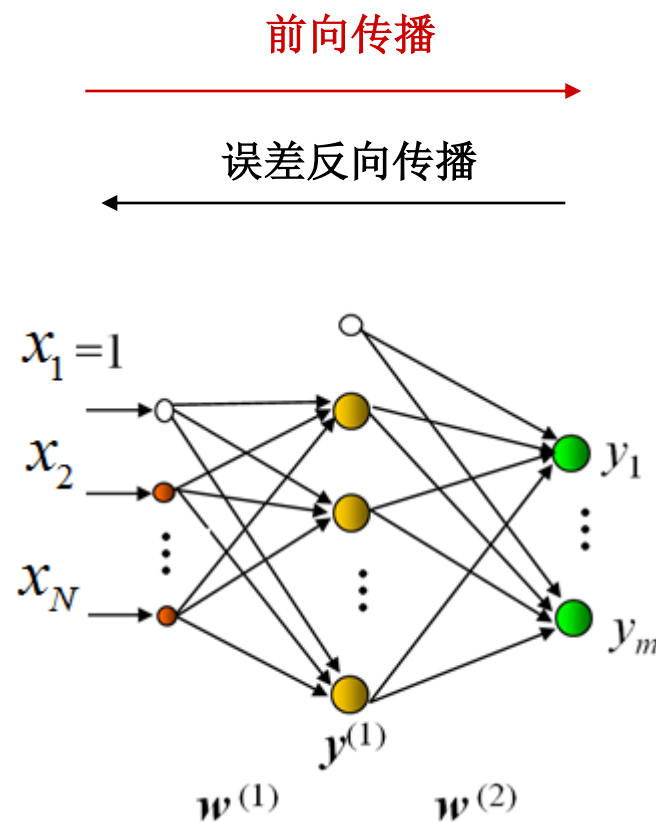
设计网络模型
学习网络参数

4.3.1 Back Propagation (BP)学习算法

发展历史：Werbos在1974年提出，Rumelhart等在1986年完善；

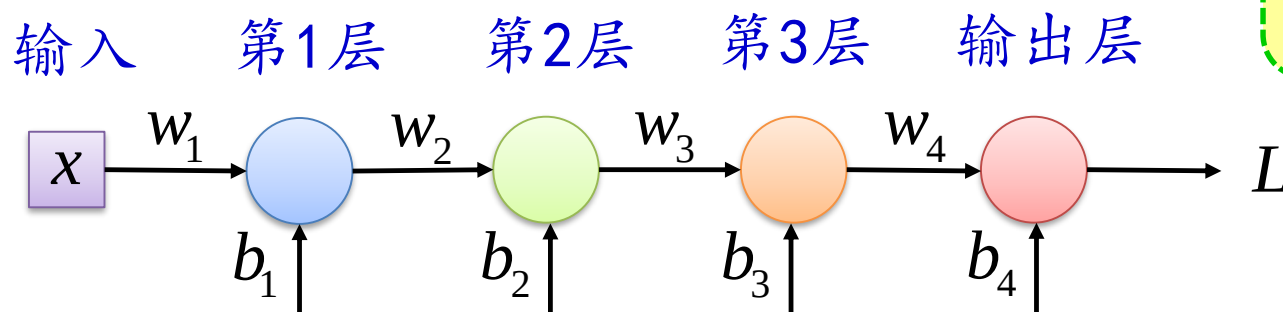
数学基础：链式求导

- 神经网络的监督学习算法
- BP学习算法由前向传播和误差反向传播组成：
 - ① 前向传播是输入信号从输入层经隐含层，传向输出层，若输出层得到了期望的输出，则学习算法结束；否则，转至反向传播。
 - ② 反向传播是将误差（样本输出与网络输出之差）按原联接通路反向计算，由梯度下降法调整各层节点的权值和阈值，使误差减小。



4.3.1 Back Propagation (BP)学习算法

BP学习算法：引子



$$z_l = w_l a_{l-1} + b_l = w_l A(z_{l-1}) + b_l$$

$$a_l = A(z_l)$$

$$a_1 = A_1(z_1) = A_1(w_1 x + b_1)$$

$$a_2 = A_2(z_2) = A_2(w_2 a_1 + b_2) = A_2(w_2 A_1(w_1 x + b_1) + b_2)$$

$$a_3 = A_3(z_3) = A_3(w_3 a_2 + b_3) = A_3(w_3 A_2(w_2 A_1(w_1 x + b_1) + b_2) + b_3)$$

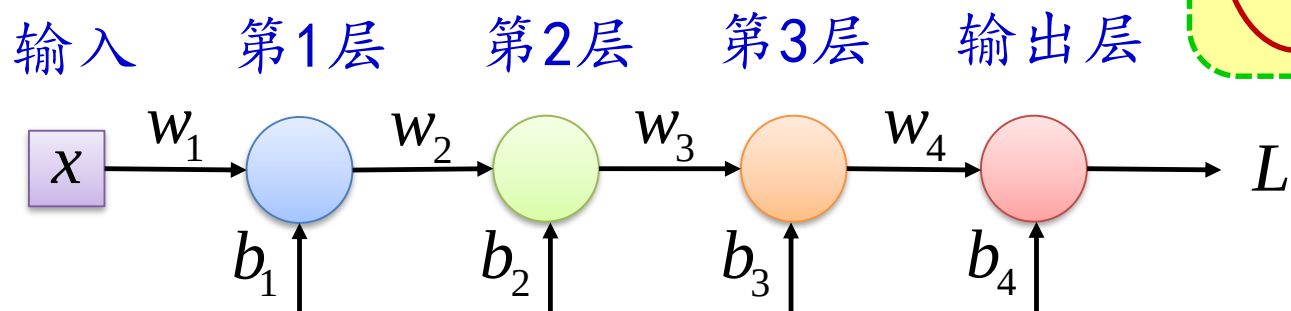
$$\hat{y} = A_4(z_4) = A_4(w_4 a_3 + b_4) = A_4(w_4 A_3(w_3 A_2(w_2 A_1(w_1 x + b_1) + b_2) + b_2) + b_4)$$

复合函数

$L(y, \hat{y})$ 衡量预测值和真实值之间的误差

4.3.1 Back Propagation (BP)学习算法

BP学习算法：引子



$$\frac{\partial L}{\partial z_{l-1}} = \frac{\partial L}{\partial z_l} \frac{\partial z_l}{\partial z_{l-1}} = \frac{\partial L}{\partial z_l} w_l A'(z_{l-1})$$

$$\delta_{l-1} = \delta_l w_l A'(z_{l-1})$$

$$\frac{\partial L}{\partial w_l} = \frac{\partial L}{\partial z_l} a_{l-1} = \delta_l a_{l-1}$$

$$\frac{\partial L}{\partial w_4} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_4} = a_3 A'_4(z_4) \frac{\partial L}{\partial \hat{y}}$$

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial w_3} = a_2 A'_3(z_3) w_4 A'_4(z_4) \frac{\partial L}{\partial \hat{y}}$$

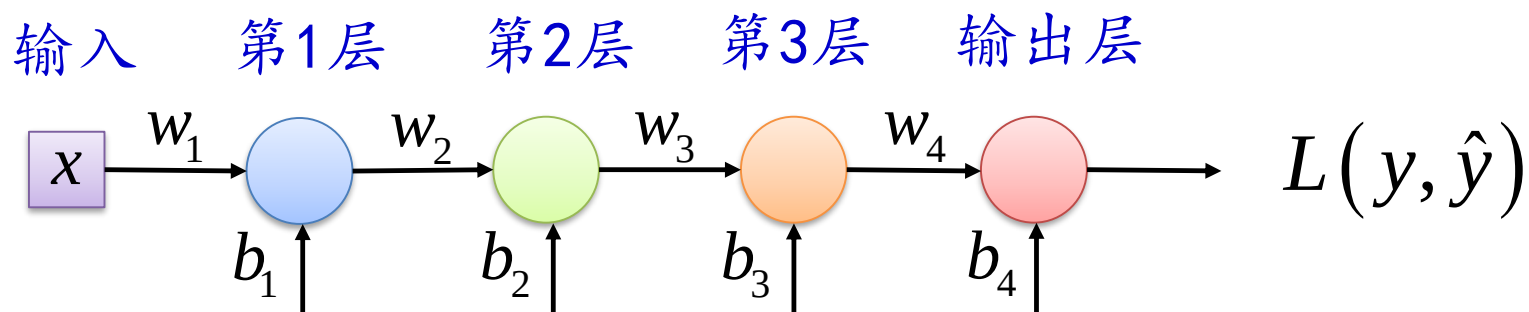
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial z_2} \frac{\partial z_2}{\partial w_2} = a_1 A'_2(z_2) w_3 A'_3(z_3) w_4 A'_4(z_4) \frac{\partial L}{\partial \hat{y}}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial z_2} \frac{\partial z_2}{\partial z_1} \frac{\partial z_1}{\partial w_1} = x A'_1(z_1) w_2 A'_2(z_2) w_3 A'_3(z_3) w_4 A'_4(z_4) \frac{\partial L}{\partial \hat{y}}$$

链式法则

4.3.1 Back Propagation (BP)学习算法

BP学习算法：引子



$$z_1 = w_1 x + b_1$$

...

$$z_{l-1} = w_{l-1} a_{l-2} + b_{l-1}$$

$$z_l = w_l a_{l-1} + b_l$$

$$a_1 = A(z_1)$$

...

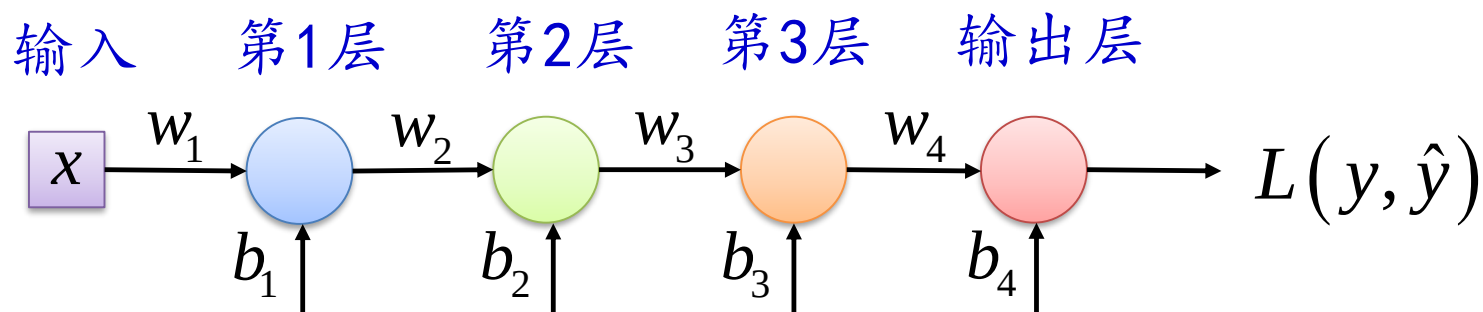
$$a_{l-1} = A(z_{l-1})$$

$$\hat{y} = A(z_l)$$

前向传播

4.3.1 Back Propagation (BP)学习算法

BP学习算法：引子

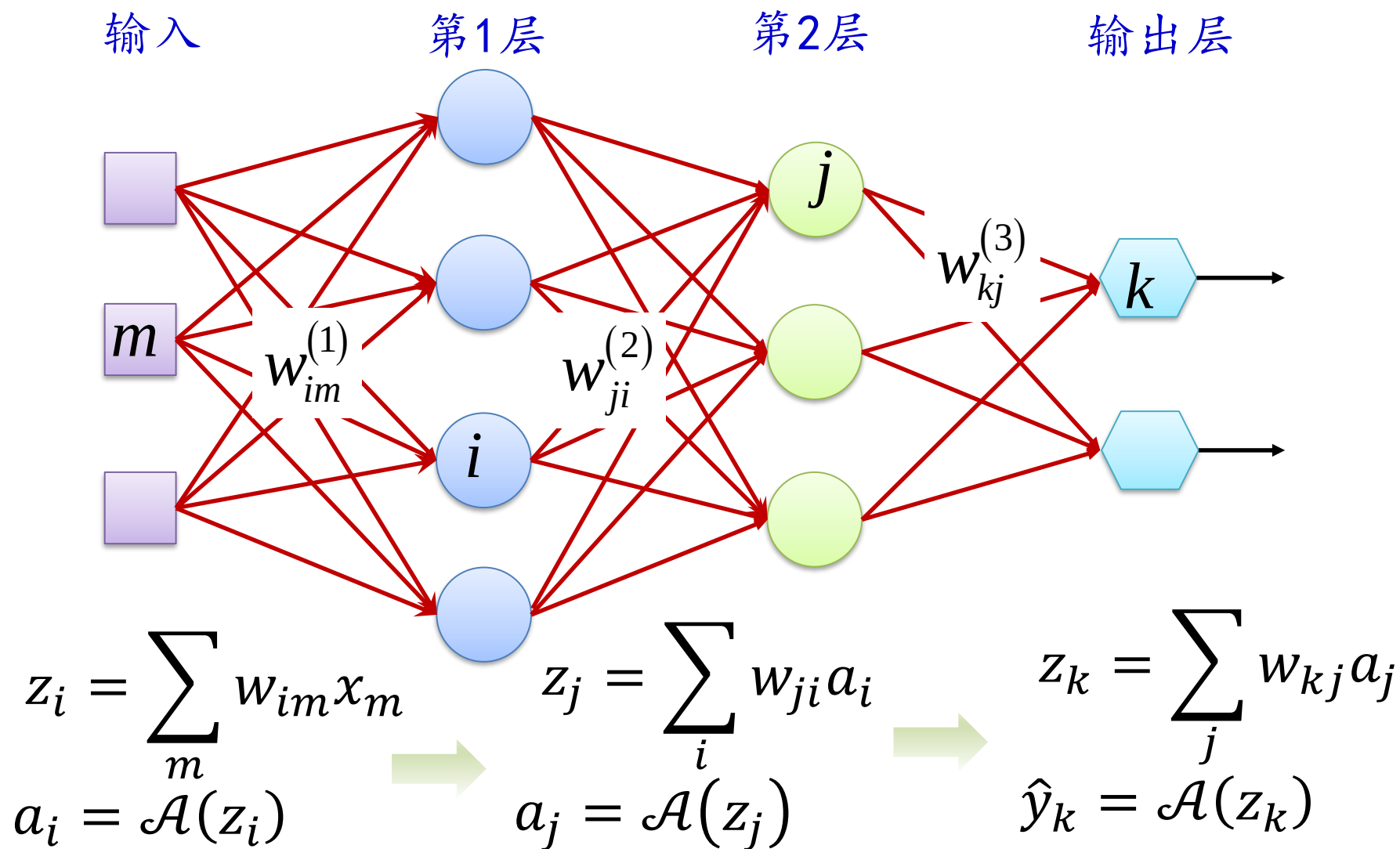


$$\begin{aligned}\delta_l &= \frac{\partial L}{\partial z_l} = \frac{\partial L}{\partial \hat{y}} A'(z_l) & \frac{\partial L}{\partial w_l} &= \delta_l a_{l-1} \\ \delta_{l-1} &= \delta_l w_l A'(z_{l-1}) & \frac{\partial L}{\partial w_{l-1}} &= \delta_{l-1} a_{l-2} \\ \dots & & & \\ \delta_1 &= \delta_2 w_2 A'(z_1) & \frac{\partial L}{\partial w_1} &= \delta_1 x\end{aligned}$$

误差反向传播

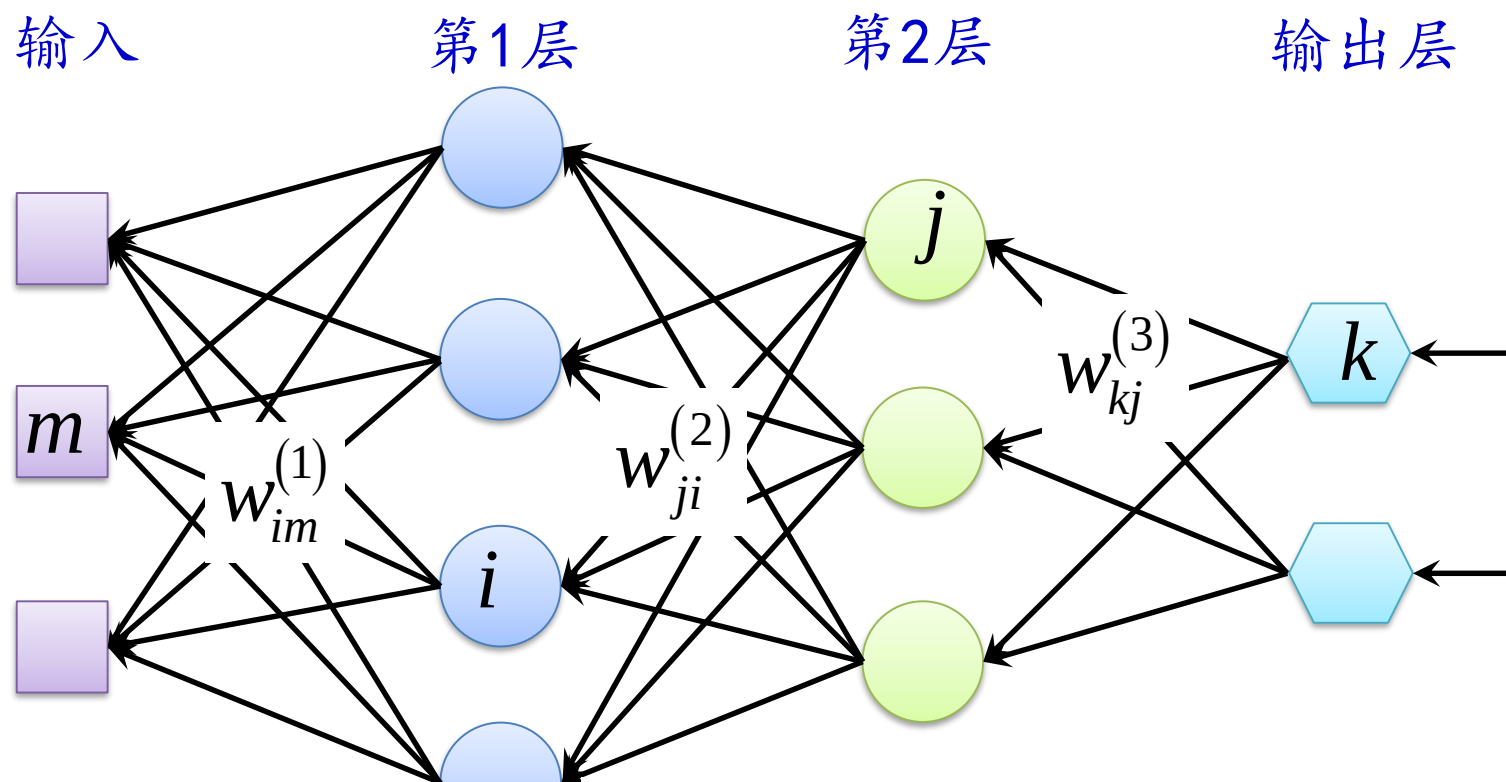
4.3.1 Back Propagation (BP)学习算法

BP学习算法：前向传播



4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播



链式法则

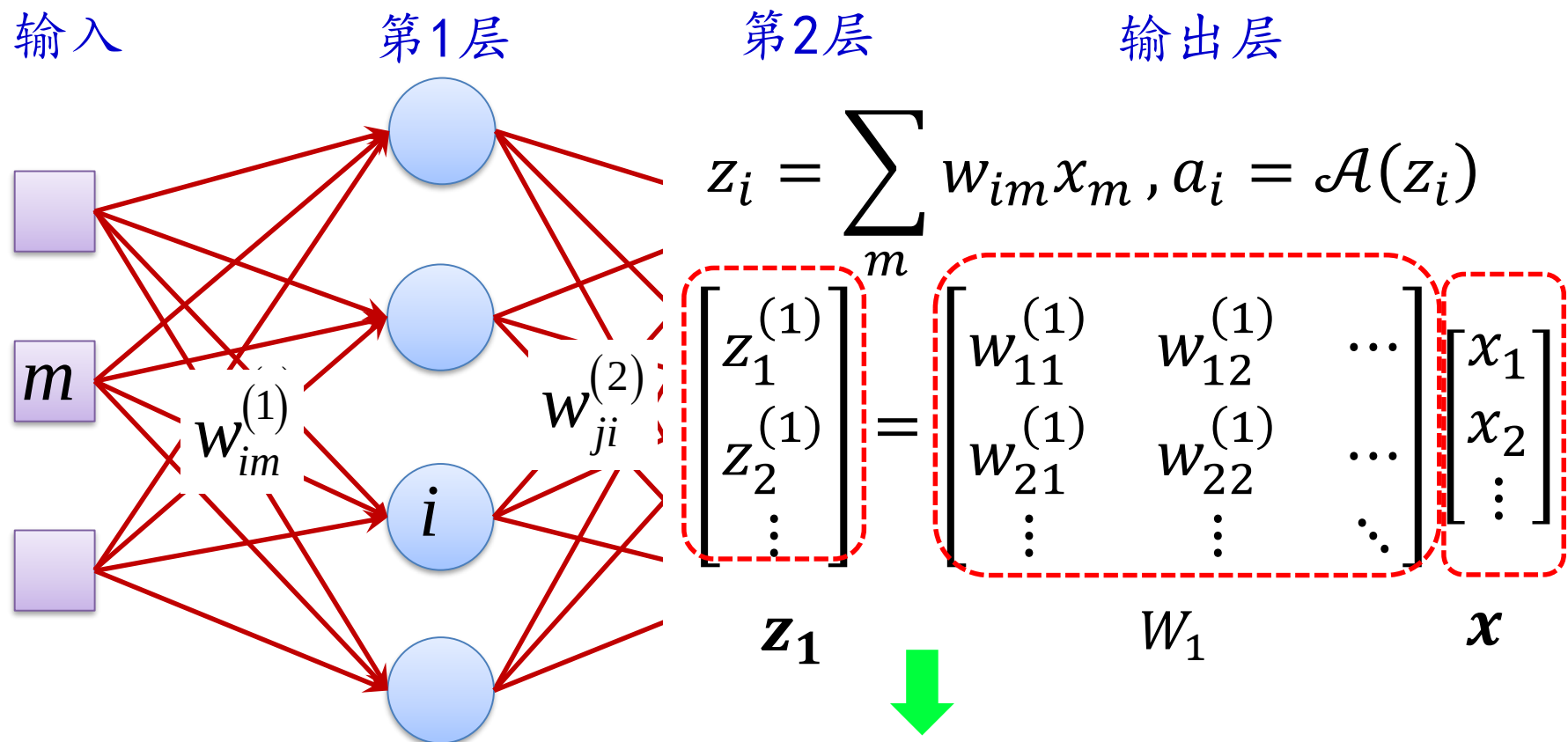
$$\frac{\partial L}{\partial a_i} = \sum_j w_{ji} \frac{\partial L}{\partial z_j}$$
$$\frac{\partial L}{\partial z_i} = \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_i}$$

$$\frac{\partial L}{\partial a_j} = \sum_k w_{kj} \frac{\partial L}{\partial z_k}$$
$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial a_j} \frac{\partial a_j}{\partial z_j}$$

$$\frac{\partial L}{\partial \hat{y}_k} = \hat{y}_k - y_k$$
$$\frac{\partial L}{\partial z_k} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_k}$$

4.3.1 Back Propagation (BP)学习算法

BP学习算法：前向传播（矩阵化表达）



第1层

$\mathbf{x} = \{x_m\}$ $\mathbf{z}_1 = \{z_i\}$

列向量

$W_1 = \{w_{im}\}$

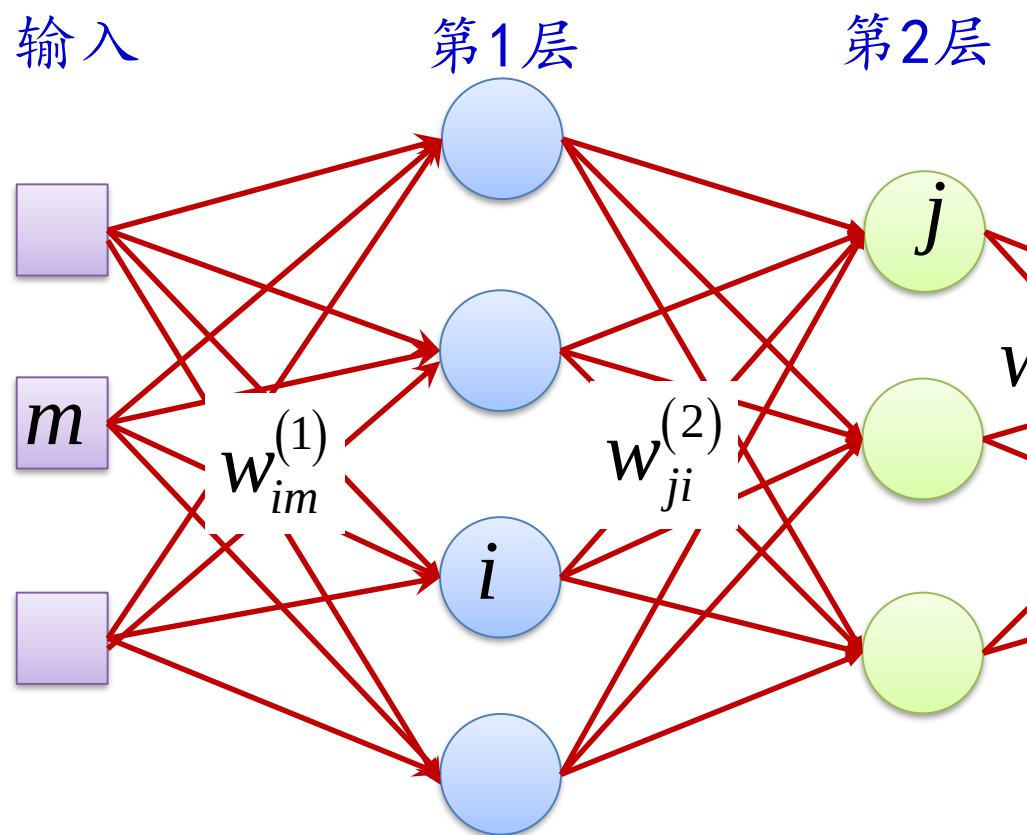
权重矩阵

$\mathbf{z}_1 = W_1 \mathbf{x}$

$\mathbf{a}_1 = \mathcal{A}(\mathbf{z}_1)$

4.3.1 Back Propagation (BP)学习算法

BP学习算法：前向传播（矩阵化表达）



$$z_j = \sum_i w_{ji} a_i, a_j = \mathcal{A}(z_j)$$



$$\mathbf{z}_2 = \mathbf{W}_2 \mathbf{a}_1, \mathbf{a}_2 = \mathcal{A}(\mathbf{z}_2)$$

第2层

$$\mathbf{a}_1 = \{a_i\} \quad \mathbf{z}_2 = \{z_j\}$$

列向量

$$\mathbf{W}_2 = \{w_{ji}\}$$

权重矩阵

4.3.1 Back Propagation (BP)学习算法

BP学习算法：前向传播（矩阵化表达）

输出层

$$\mathbf{a}_2 = \{a_j\} \quad \mathbf{z}_3 = \{z_k\}$$

列向量

$$\hat{\mathbf{y}} = \{\hat{y}_k\}$$

$$W_3 = \{w_{kj}\} \quad \text{权重矩阵}$$

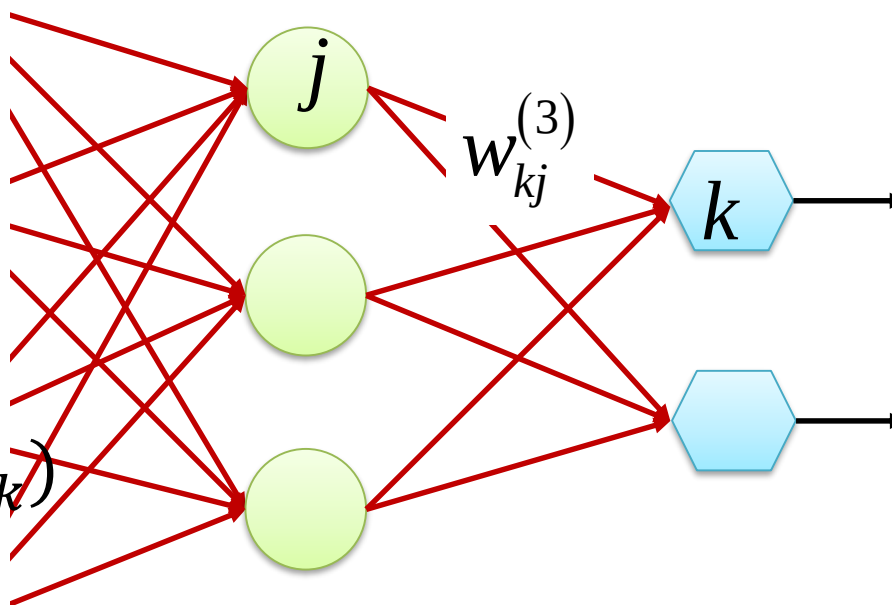
$$z_k = \sum_j w_{kj} a_j, \hat{y}_k = \mathcal{A}(z_k)$$



$$\mathbf{z}_3 = W_3 \mathbf{a}_2, \hat{\mathbf{y}} = \mathcal{A}(\mathbf{z}_3)$$

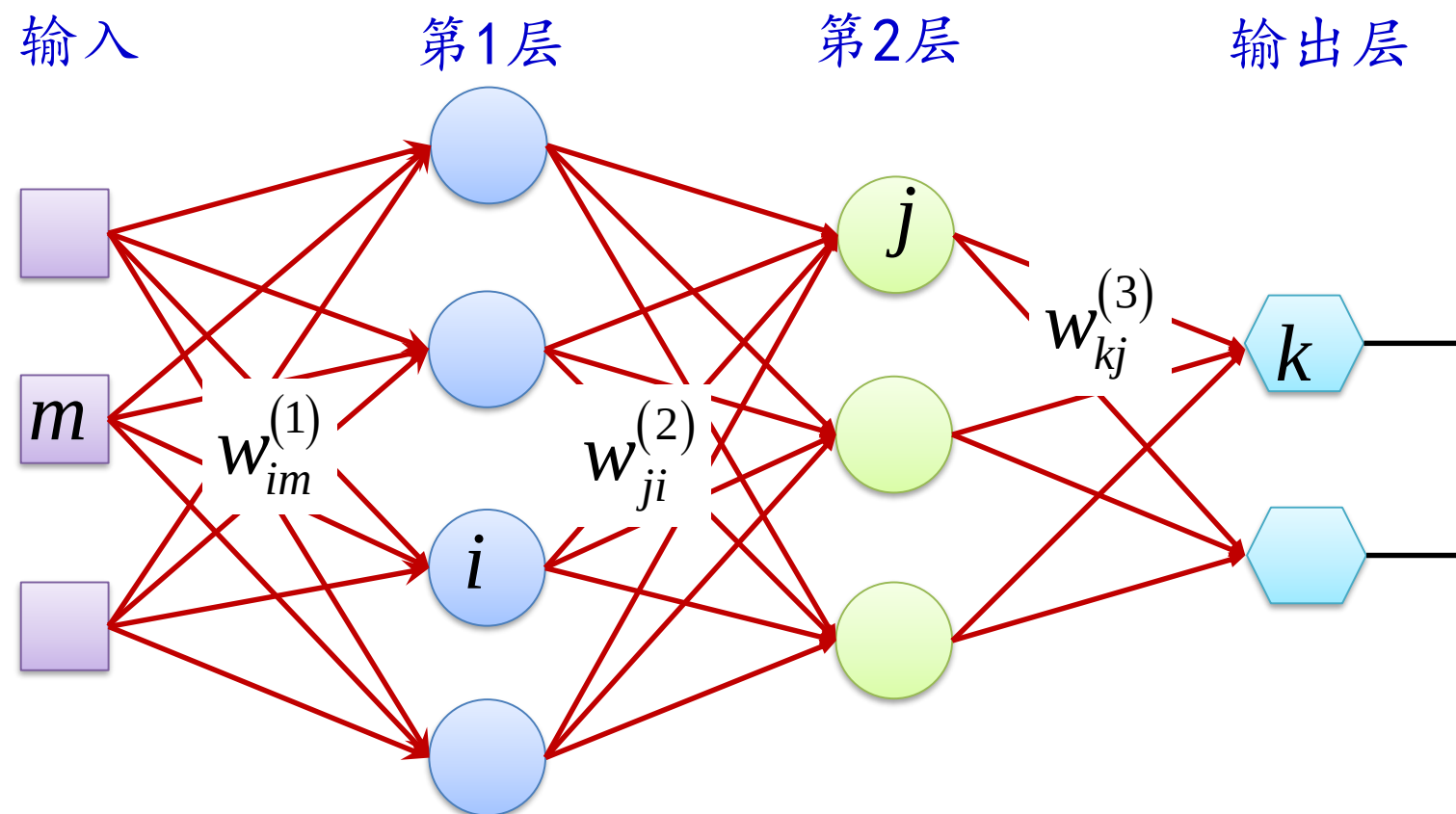
第2层

输出层



4.3.1 Back Propagation (BP)学习算法

BP学习算法：前向传播（矩阵化表达）



输出层：（损失函数计算） $L(\mathbf{y}, \hat{\mathbf{y}})$

4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)

输出层节点

$$\delta_3 = \{\delta_k\} = \left\{ \frac{\partial L}{\partial z_k} \right\}$$

$$\mathbf{z}_3 = \{z_k\} \quad \hat{\mathbf{y}} = \{\hat{y}_k\}$$

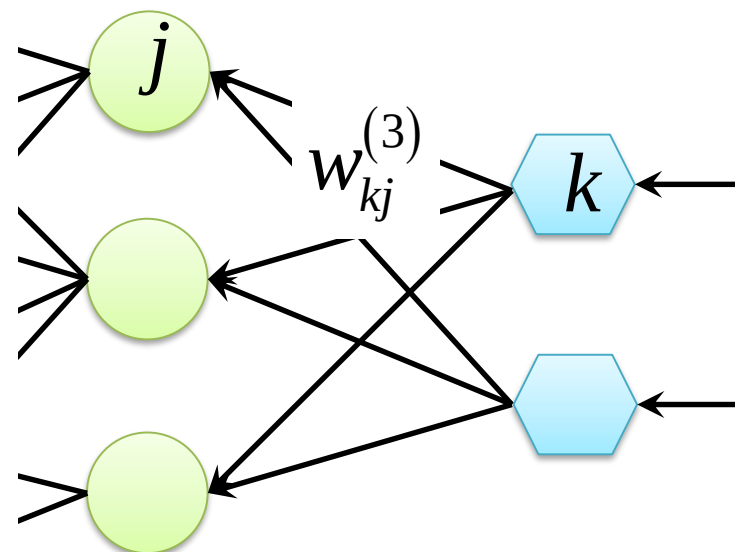
$$\frac{\partial \hat{y}_k}{\partial z_k} = \mathcal{A}'(z_k)$$

$$\delta_k = \frac{\partial L}{\partial z_k} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_k} = \frac{\partial L}{\partial \hat{y}_k} \mathcal{A}'(z_k)$$

$$\nabla_{\hat{\mathbf{y}}} L = \left\{ \frac{\partial L}{\partial \hat{y}_k} \right\} \quad \mathcal{A}'(\mathbf{z}_3) = \{\mathcal{A}'(z_k)\}$$

第2层

输出层



$$\delta_3 = \mathcal{A}'(\mathbf{z}_3) \circ (\nabla_{\hat{\mathbf{y}}} L)$$

4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)

第2层

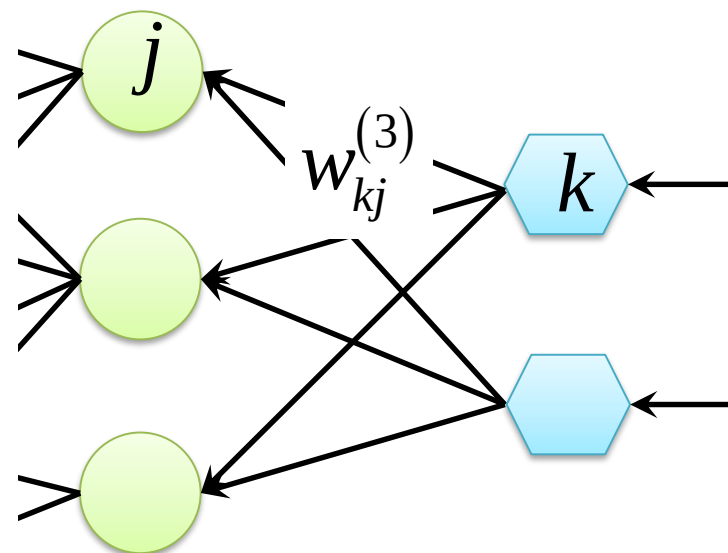
输出层

输出层至第2层连接权重

$$\delta_k = \frac{\partial L}{\partial z_k} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_k} = \frac{\partial L}{\partial \hat{y}_k} \mathcal{A}'(z_k)$$

$$\frac{\partial L}{\partial w_{kj}} = \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial w_{kj}} = \delta_k a_j$$

$$z_k = \sum_j w_{kj} a_j \quad \frac{\partial z_k}{\partial w_{kj}} = a_j$$



4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)

$$\delta_k = \frac{\partial L}{\partial z_k} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_k} = \frac{\partial L}{\partial \hat{y}_k} \mathcal{A}'(z_k)$$

$$\boldsymbol{\delta}_3 = \mathcal{A}'(\mathbf{z}_3) \circ (\nabla_{\hat{\mathbf{y}}} L)$$

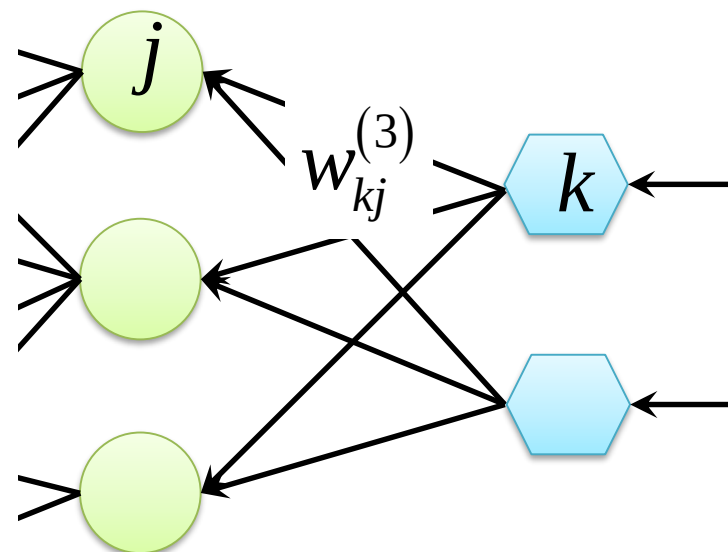
$$\frac{\partial L}{\partial w_{kj}} = \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial w_{kj}} = \delta_k a_j$$

$$\nabla_{W_3} L = \left\{ \frac{\partial L}{\partial w_{kj}} \right\} = \begin{bmatrix} \delta_1^{(3)} a_1^{(2)} & \delta_1^{(3)} a_2^{(2)} & \dots \\ \delta_2^{(3)} a_1^{(2)} & \delta_2^{(3)} a_2^{(2)} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} \delta_1^{(3)} \\ \delta_2^{(3)} \\ \vdots \end{bmatrix} [a_1^{(2)} \quad a_2^{(2)} \quad \dots] = \boldsymbol{\delta}_3 (\mathbf{a}_2)^T$$

$$\nabla_{W_3} L = \left(\mathcal{A}'(\mathbf{z}_3) \circ (\nabla_{\hat{\mathbf{y}}} L) \right) (\mathbf{a}_2)^T$$

第2层

输出层



4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)

第2层节点

$$\delta_2 = \{\delta_j\} = \left\{ \frac{\partial L}{\partial z_j} \right\}$$

$$\mathcal{A}'(\mathbf{z}_2) = \{\mathcal{A}'(z_j)\}$$

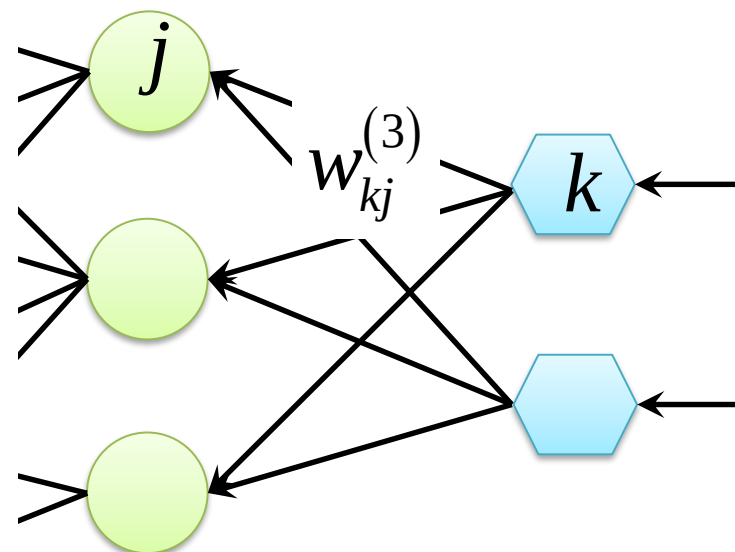
$$\mathbf{z}_2 = \{z_j\} \quad \mathbf{a}_2 = \{a_j\}$$

$$\frac{\partial L}{\partial a_j} = \sum_k \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial a_j} = \sum_k w_{kj} \frac{\partial L}{\partial z_k}$$

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial a_j} \frac{\partial a_j}{\partial z_j} = \frac{\partial L}{\partial a_j} \mathcal{A}'(z_j)$$

第2层

输出层



4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)

第2层节点

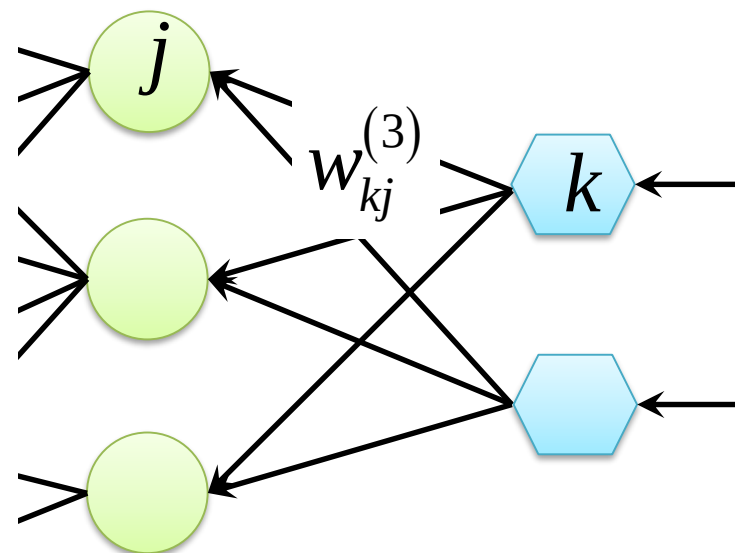
$$\frac{\partial L}{\partial a_j} = \sum_k \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial a_j} = \sum_k w_{kj} \frac{\partial L}{\partial z_k}$$

$$= \begin{bmatrix} w_{1j}^{(3)} & w_{2j}^{(3)} & \cdots & w_{kj}^{(3)} & \cdots \end{bmatrix} \begin{bmatrix} \delta_1^{(3)} \\ \delta_2^{(3)} \\ \vdots \\ \delta_k^{(3)} \\ \vdots \end{bmatrix}$$

$$\nabla_{a_2} L = \begin{bmatrix} \frac{\partial L}{\partial a_1^{(2)}} \\ \vdots \\ \frac{\partial L}{\partial a_j^{(2)}} \\ \vdots \end{bmatrix} = \begin{bmatrix} w_{11}^{(3)} & \cdots & w_{k1}^{(3)} & \cdots \\ \vdots & \vdots & \vdots & \ddots \\ w_{1j}^{(3)} & \ddots & w_{kj}^{(3)} & \vdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} \delta_1^{(3)} \\ \delta_2^{(3)} \\ \vdots \\ \delta_k^{(3)} \\ \vdots \end{bmatrix} = (W_3)^T \delta_3$$

第2层

输出层



$$\begin{aligned} \delta_2 &= \mathcal{A}'(\mathbf{z}_2) \circ (\nabla_{a_2} L) \\ &= \mathcal{A}'(\mathbf{z}_2) \circ ((W_3)^T \delta_3) \end{aligned}$$

4.3.1 Back Propagation (BP)学习算法

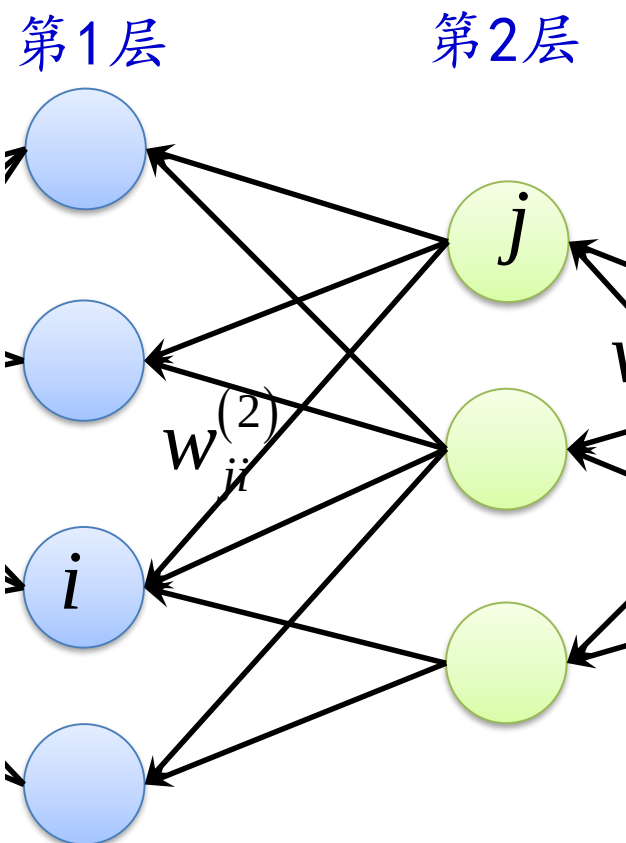
BP学习算法：误差反向传播 (矩阵化表达)

第2层至第1层连接权重

$$\mathbf{a}_1 = \{a_i\}$$

$$\frac{\partial L}{\partial w_{ji}} = \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial w_{ji}} = \delta_j a_i$$

$$\nabla_{W_2} L = \left\{ \frac{\partial L}{\partial w_{ji}} \right\} = \begin{bmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \\ \vdots \end{bmatrix} [a_1^{(1)} \quad a_2^{(1)} \quad \dots] = \boldsymbol{\delta}_2 (\mathbf{a}_1)^T = \left(\mathcal{A}'(\mathbf{z}_2) \circ ((W_3)^T \boldsymbol{\delta}_3) \right) (\mathbf{a}_1)^T$$



$$\boldsymbol{\delta}_2 = \{\delta_j\} = \left\{ \frac{\partial L}{\partial z_j} \right\}$$

$$\mathbf{z}_2 = \{z_j\}$$

$$\mathbf{a}_2 = \{a_j\}$$

$$\boldsymbol{\delta}_2 = \mathcal{A}'(\mathbf{z}_2) \circ ((W_3)^T \boldsymbol{\delta}_3)$$

4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)

第1层节点

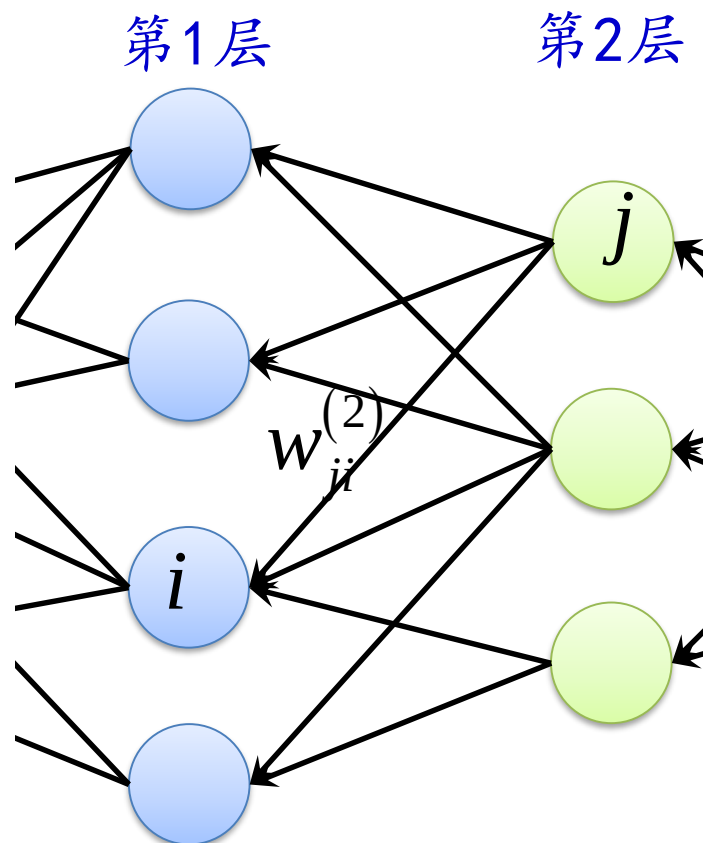
$$\delta_1 = \{\delta_i\} = \left\{ \frac{\partial L}{\partial z_i} \right\}$$

$$\mathcal{A}'(\mathbf{z}_1) = \{\mathcal{A}'(z_i)\}$$

$$\mathbf{z}_1 = \{z_i\} \quad \mathbf{a}_1 = \{a_i\}$$

$$\frac{\partial L}{\partial z_i} = \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_i} = \frac{\partial L}{\partial a_i} \mathcal{A}'(z_i)$$

$$\frac{\partial L}{\partial a_i} = \sum_j \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial a_i} = \sum_j w_{ji} \frac{\partial L}{\partial z_j}$$

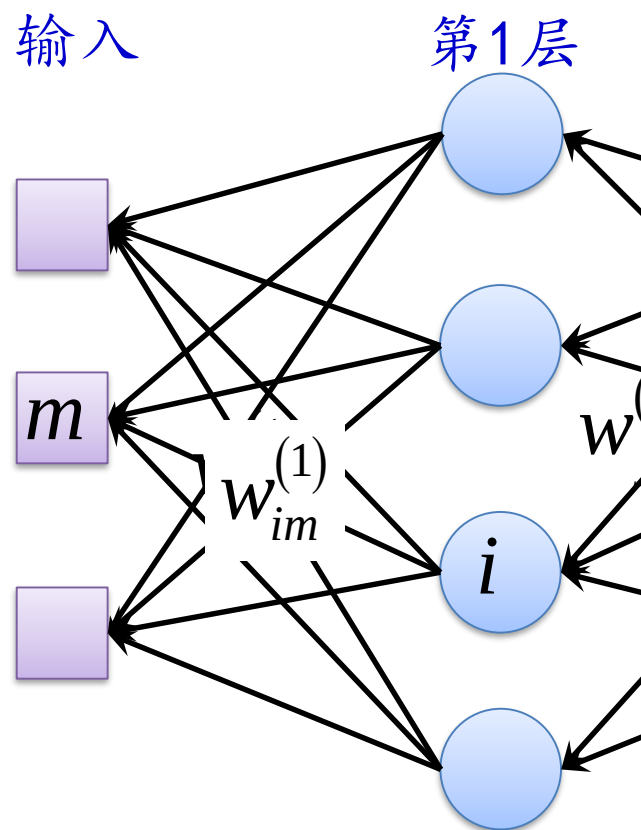


$$\nabla_{a_1} L = (W_2)^T \delta_2$$

$$\begin{aligned} \delta_1 &= \mathcal{A}'(\mathbf{z}_1) \circ (\nabla_{a_1} L) \\ &= \mathcal{A}'(\mathbf{z}_1) \circ ((W_2)^T \delta_2) \end{aligned}$$

4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)



$$\boldsymbol{\delta}_1 = \{\delta_i\} = \left\{ \frac{\partial L}{\partial z_i} \right\}$$

$$\mathbf{z}_1 = \{z_i\} \quad \mathbf{a}_1 = \{a_i\}$$

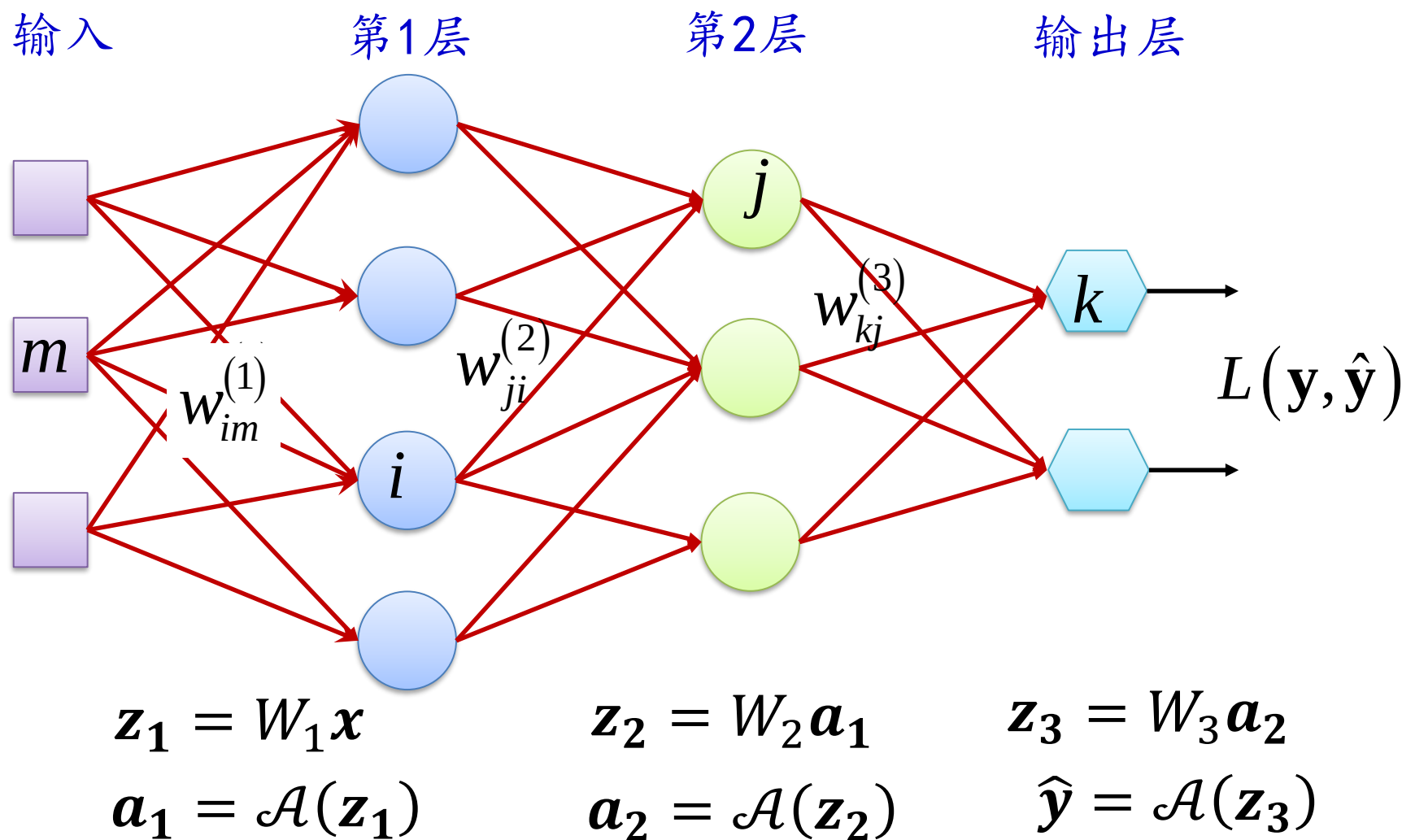
$$\frac{\partial L}{\partial w_{im}} = \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial w_{im}} = \delta_i x_m$$

$$\begin{aligned} \nabla_{W_1} L &= \left\{ \frac{\partial L}{\partial w_{im}} \right\} = \boldsymbol{\delta}_1 (\mathbf{x})^T \\ &= \left(\mathcal{A}'(\mathbf{z}_1) \circ ((W_2)^T \boldsymbol{\delta}_2) \right) (\mathbf{x})^T \end{aligned}$$

第1层至输入层连接权重

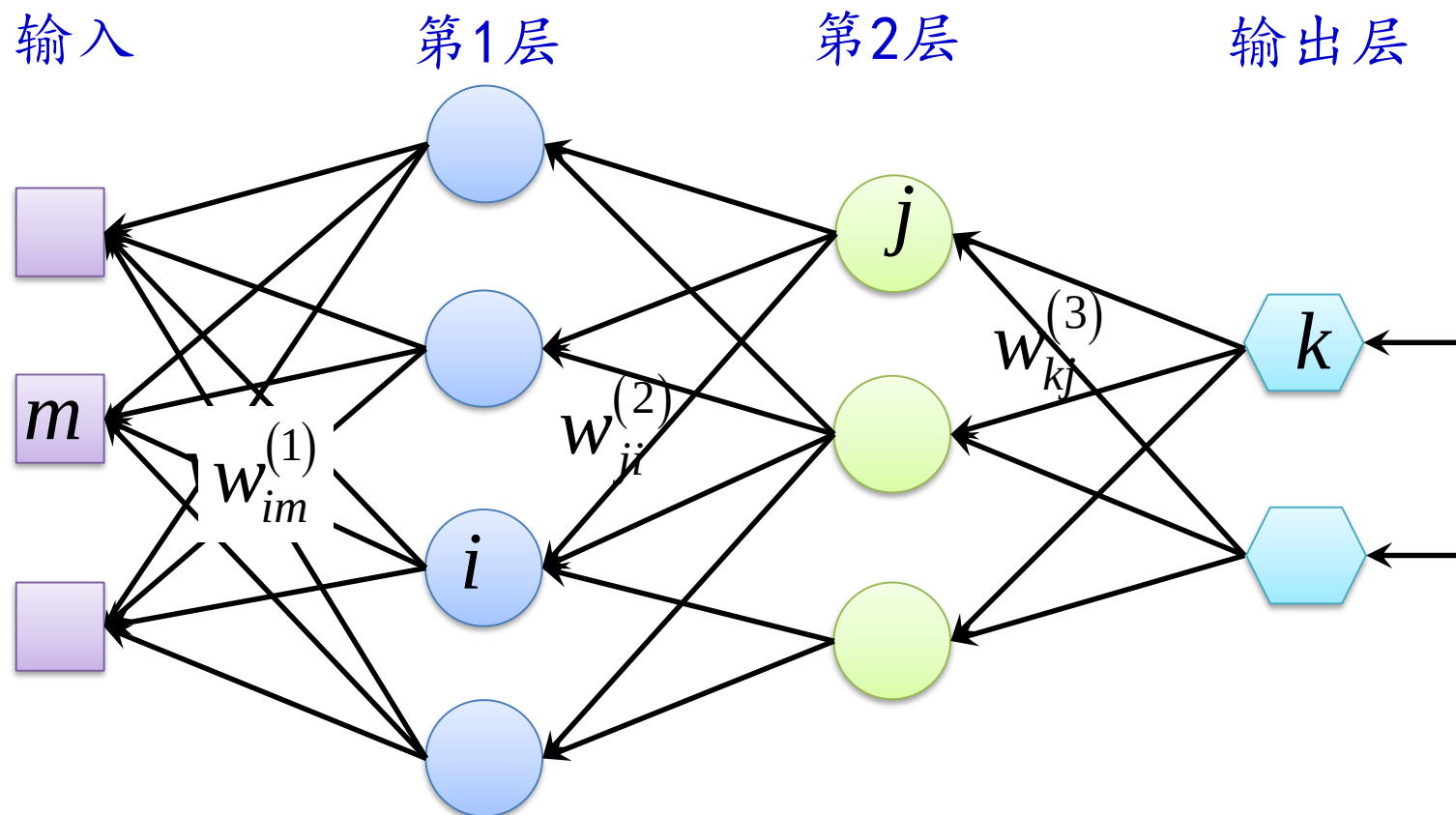
4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)



4.3.1 Back Propagation (BP)学习算法

BP学习算法：误差反向传播 (矩阵化表达)



$$\delta_1 = \mathcal{A}'(\mathbf{z}_1) \circ ((W_2)^T \delta_2) \quad \delta_2 = \mathcal{A}'(\mathbf{z}_2) \circ ((W_3)^T \delta_3) \quad \delta_3 = \mathcal{A}'(\mathbf{z}_3) \circ (\nabla_{\hat{\mathbf{y}}} L)$$

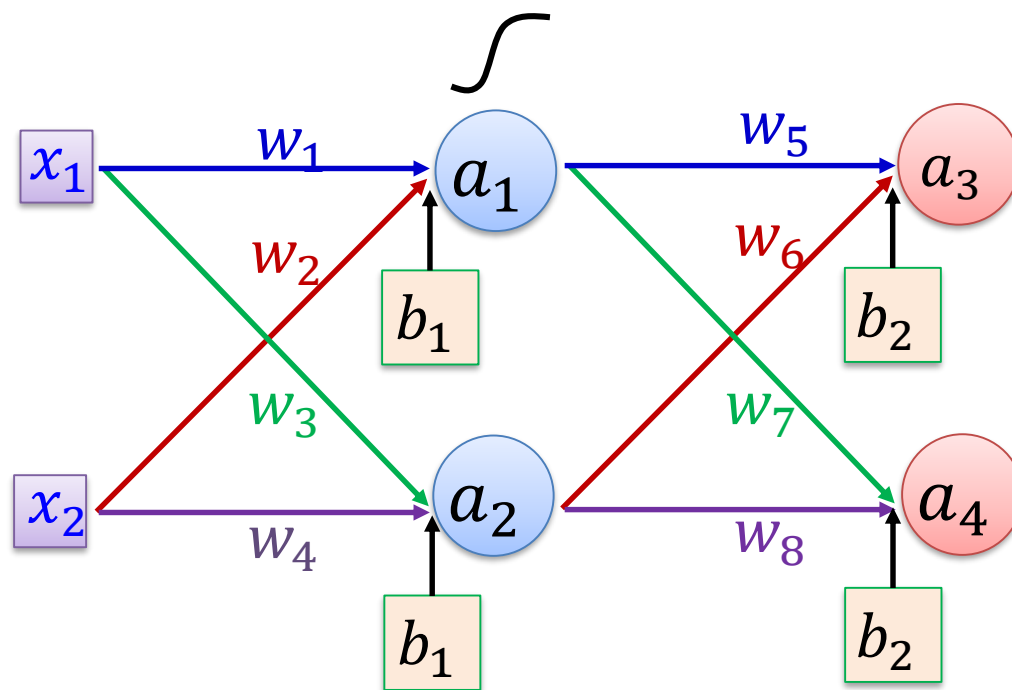
$$\nabla_{W_1} L = \delta_1(\mathbf{x})^T$$

$$\nabla_{W_2} L = \delta_2(\mathbf{a}_1)^T$$

$$\nabla_{W_3} L = \delta_3(\mathbf{a}_2)^T$$

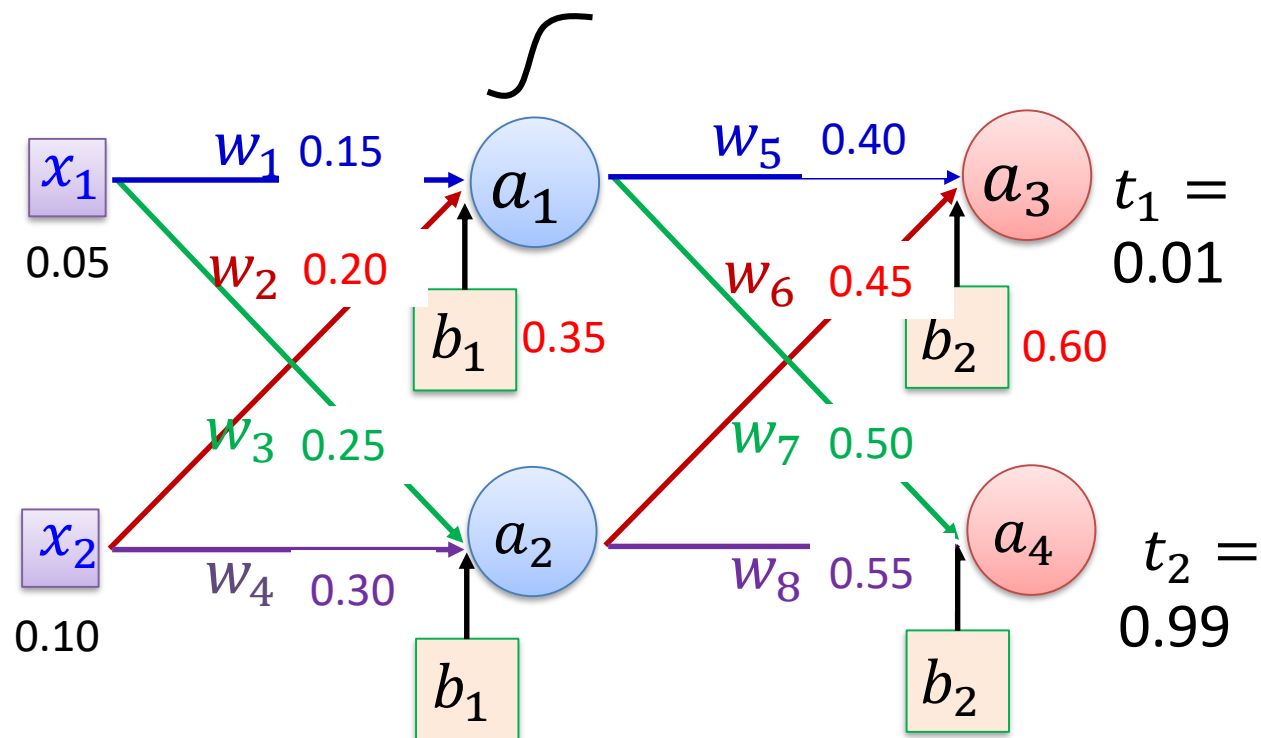
4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

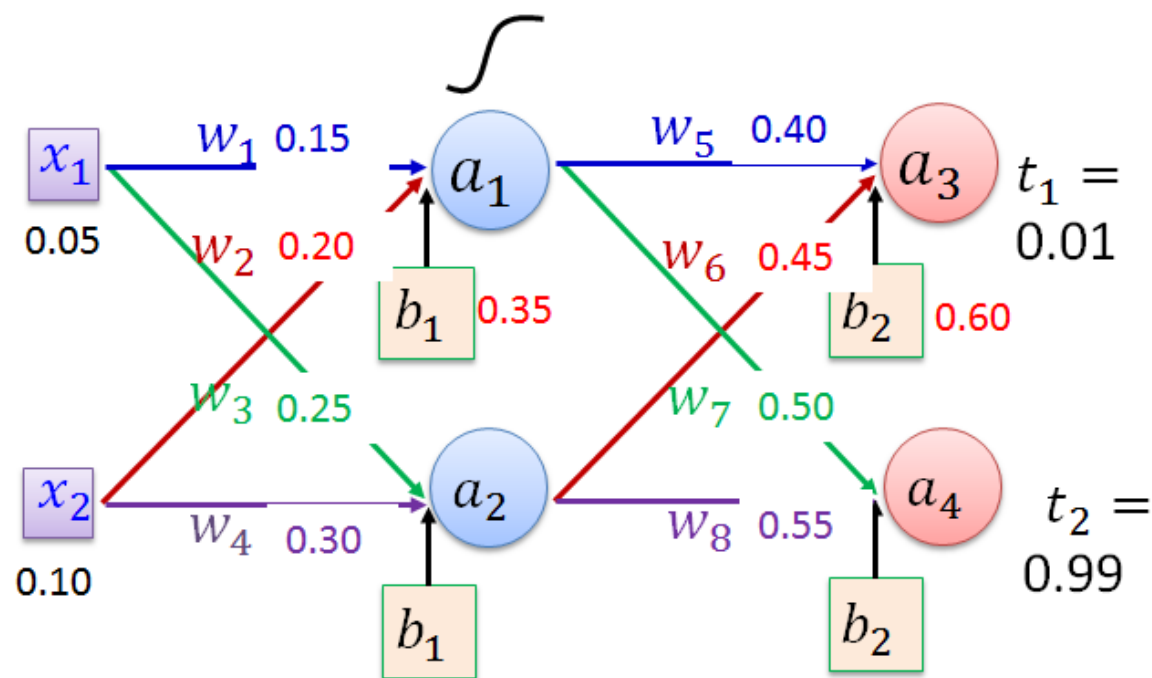
前向计算

$$\begin{aligned} z_1 &= net_{h1} = w_1 x_1 + w_2 x_2 + b_1 \\ &= 0.15 * 0.05 + 0.2 * 0.1 + 0.35 \\ &= 0.3775 \end{aligned}$$

$$a_1 = out_{h1} = \frac{1}{1 + e^{-z_1}} = 0.593269992$$

$$\begin{aligned} z_2 &= net_{h2} = w_3 x_1 + w_4 x_2 + b_1 \\ &= 0.25 * 0.05 + 0.3 * 0.1 + 0.35 \\ &= 0.3925 \end{aligned}$$

$$a_2 = out_{h2} = \frac{1}{1 + e^{-z_2}} = 0.596884378$$



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

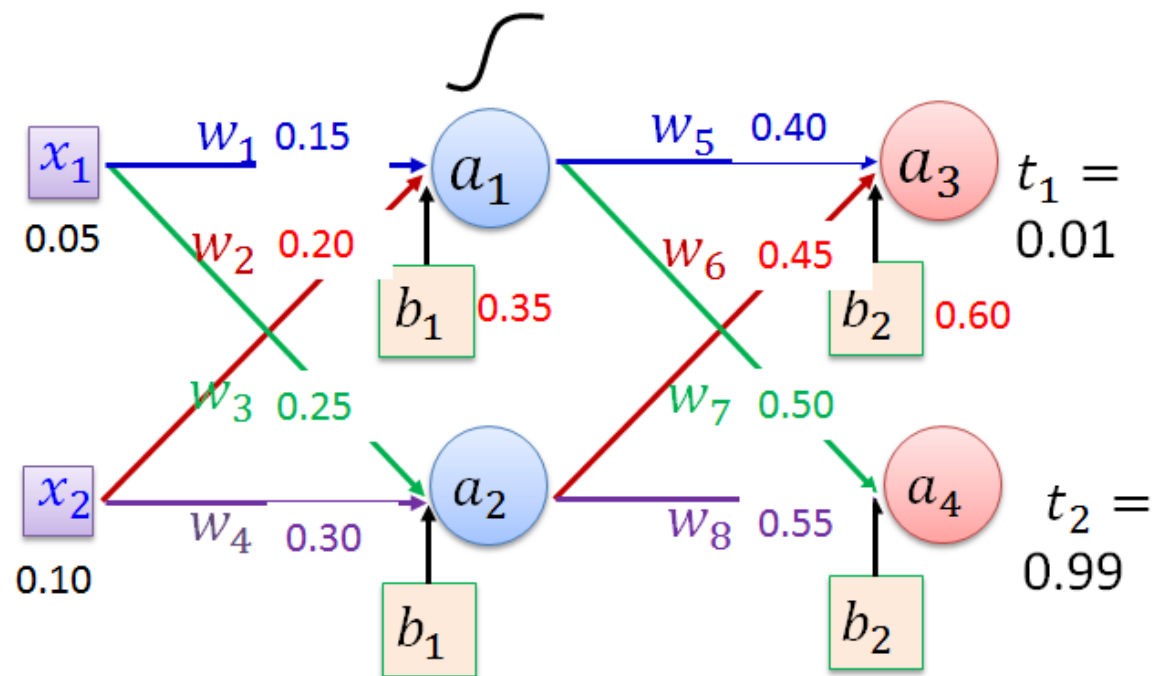
前向计算(续)

$$z_3 = net_{o1} = w_5 a_1 + w_6 a_2 + b_2$$
$$= 1.105905967$$

$$a_3 = out_{o1} = \frac{1}{1 + e^{-z_3}} = 0.75136507$$

$$z_4 = net_{o2} = w_7 a_1 + w_8 a_2 + b_2$$

$$a_4 = out_{o2} = \frac{1}{1 + e^{-z_4}} = 0.772928465$$



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

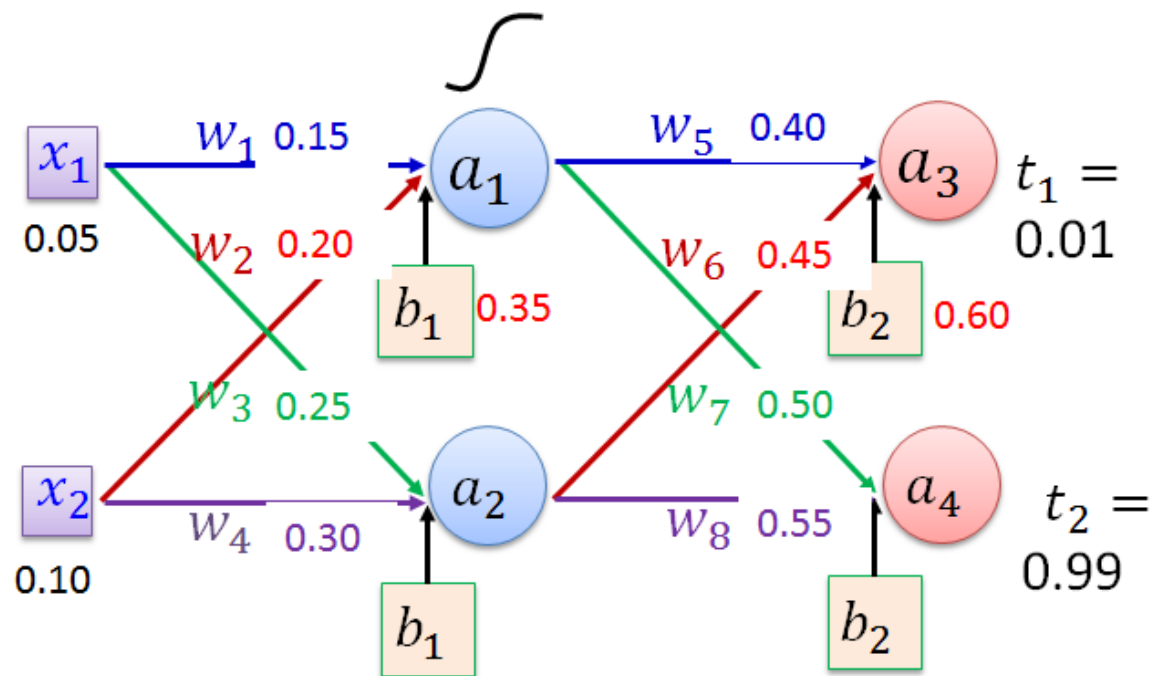
计算误差

$$E = E_{o1} + E_{o2}$$

$$E_{o1} = 0.5 * (0.01 - 0.75136507)^2$$
$$= 0.274811083$$

$$E_{o2} = 0.023560026$$

$$E = 0.298371109$$



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

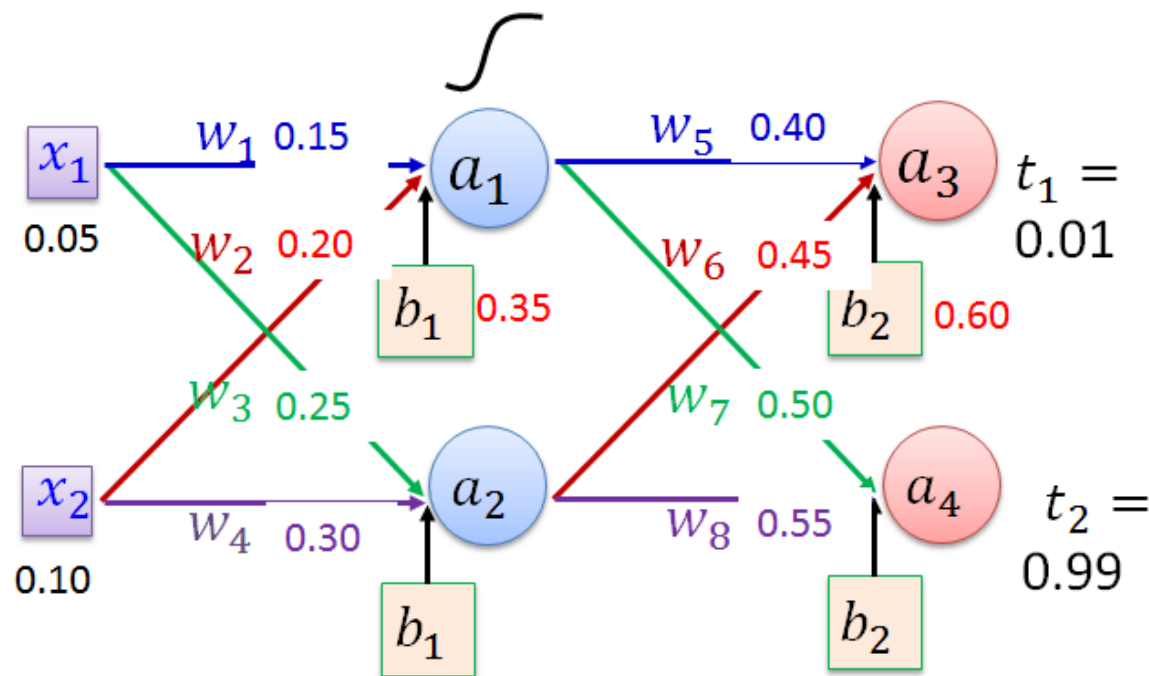
误差反向传播

以参数 w_5 为例

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_5}$$

$$\begin{aligned} \frac{\partial E}{\partial a_3} &= -(t_1 - a_3) \\ &= -(0.01 - 0.75136507) \\ &= 0.74136507 \end{aligned}$$

$$\begin{aligned} \frac{\partial a_3}{\partial z_3} &= a_3(1 - a_3) \\ &= 0.75136507 * (1 - 0.75136507) \\ &= 0.186815602 \end{aligned}$$



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

误差反向传播

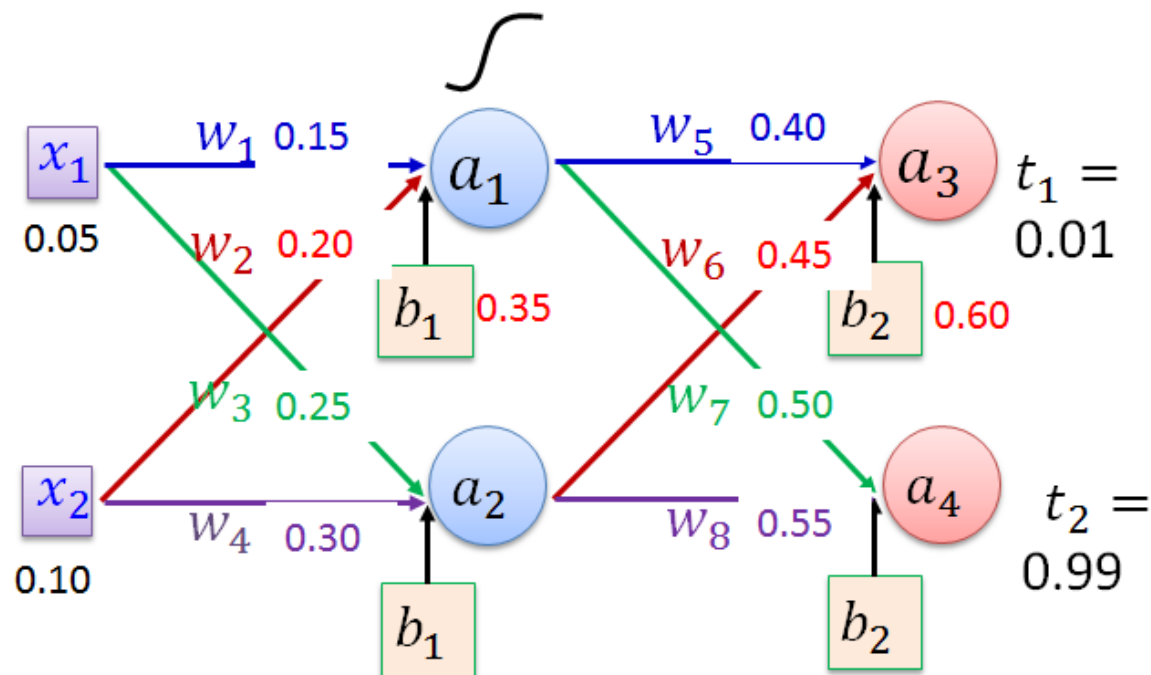
以参数 w_5 为例

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_5}$$

$$\frac{\partial z_3}{\partial w_5} = a_1 = 0.593269992$$

$$\frac{\partial E}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992$$

$$= 0.082167041$$



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

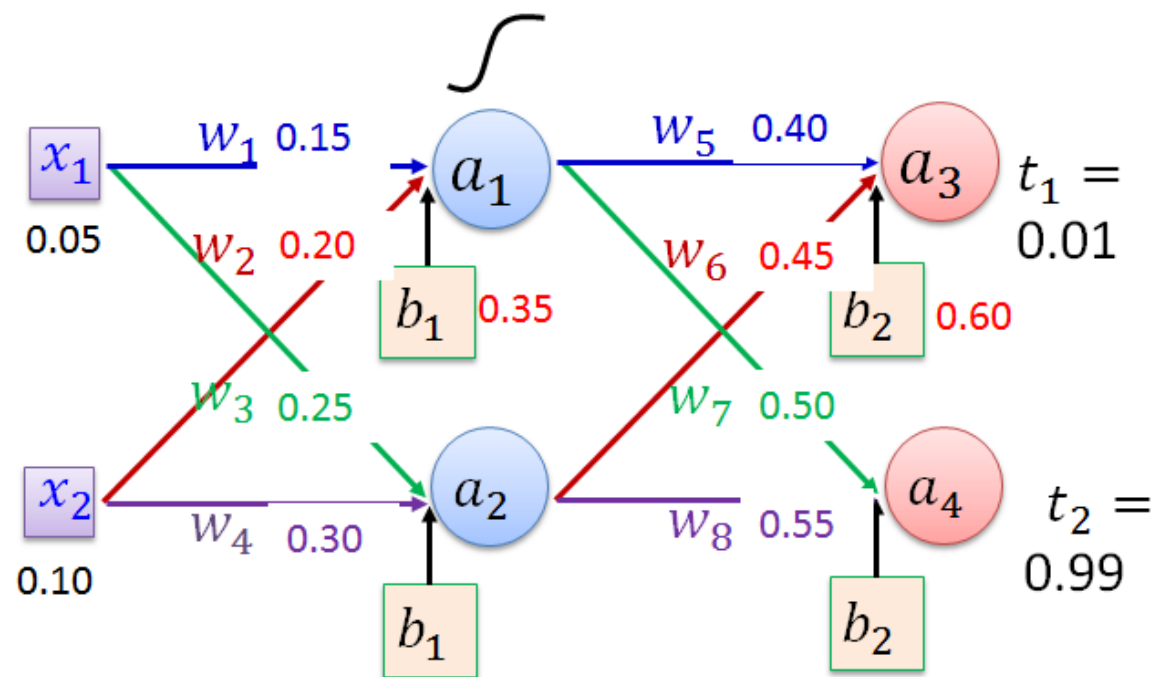
误差反向传播

以参数 w_5 为例

$$\begin{aligned}\frac{\partial E}{\partial w_5} &= \frac{\partial E}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_5} \\ &= -(t_1 - a_3) a_3 (1 - a_3) a_1\end{aligned}$$

更新参数 w_5

$$\begin{aligned}w_5^+ &= w_5 - \eta \frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_5} \\ &= 0.4 - 0.5 * 0.082167041 = 0.35891648\end{aligned}$$



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

误差反向传播

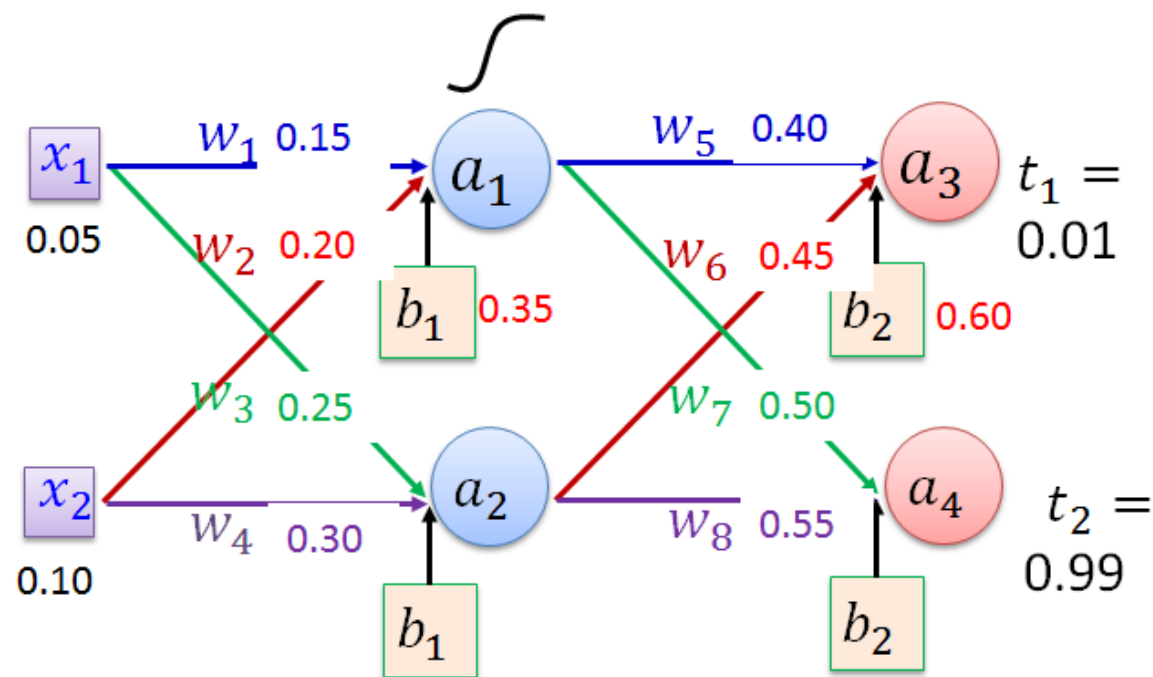
更新参数

$$w_5^+ = 0.35891648$$

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$



4.3.1 Back Propagation (BP)学习算法

例4：利用BP算法对神经网络参数进行学习

误差反向传播

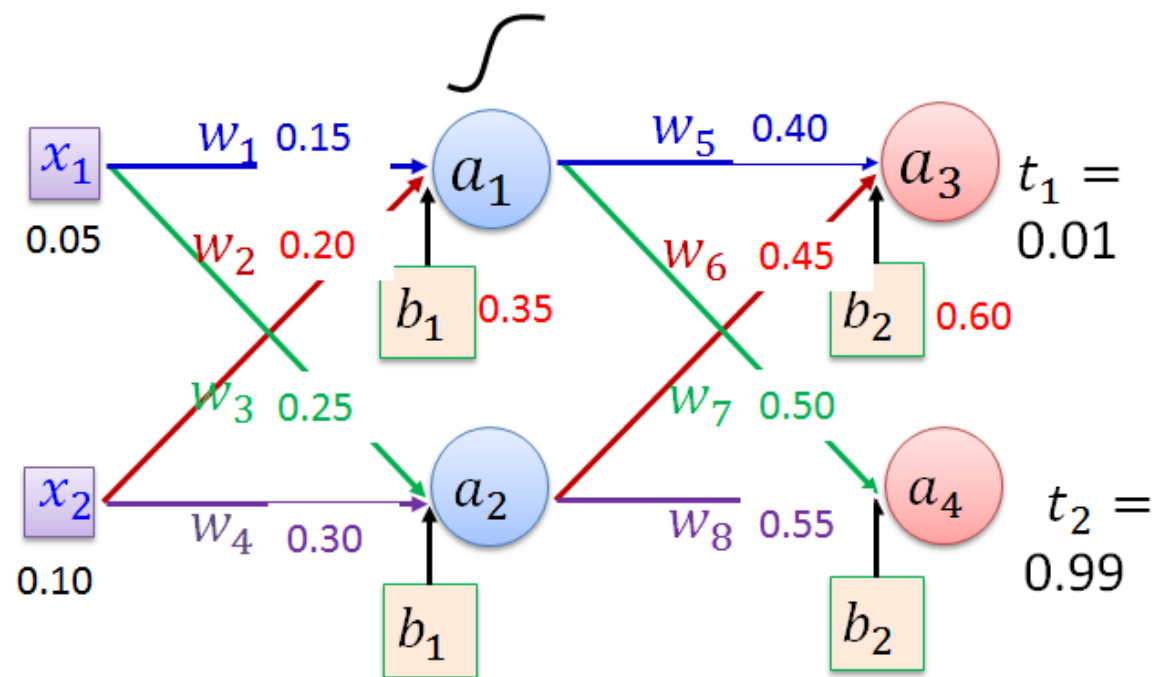
更新参数

$$w_1^+ = 0.149780716$$

$$w_2^+ = 0.19956143$$

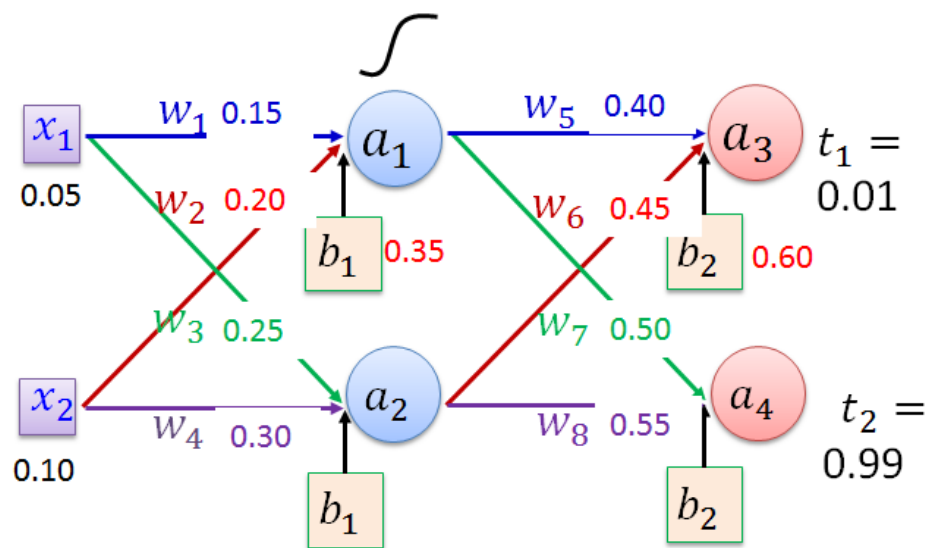
$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$



4.3.1 Back Propagation (BP)学习算法

例4(续): 利用BP算法对神经网络参数进行学习



前向计算(矩阵化表达)

$$W_1 = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \mathbf{z}_1 = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

$$\mathbf{z}_1 = W_1 \mathbf{x} + b_1$$

$$\mathbf{a}_1 = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}, \mathbf{a}_1 = \sigma(\mathbf{z}_1)$$

第1层

误差计算

$$E = \frac{1}{2} \|\mathbf{t} - \mathbf{a}_2\|^2, \mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} w_5 & w_6 \\ w_7 & w_8 \end{bmatrix}, \mathbf{z}_2 = \begin{bmatrix} z_3 \\ z_4 \end{bmatrix}, \mathbf{a}_2 = \begin{bmatrix} a_3 \\ a_4 \end{bmatrix}$$

$$\mathbf{z}_2 = W_2 \mathbf{a}_1 + b_2, \mathbf{a}_2 = \sigma(\mathbf{z}_2)$$

第2层

4.3.1 Back Propagation (BP)学习算法

例4(续)：利用BP算法对神经网络参数进行学习

前向计算

$$\mathbf{z}_1 = W_1 \mathbf{x} + b_1 = [0.3775, 0.3925]^T$$

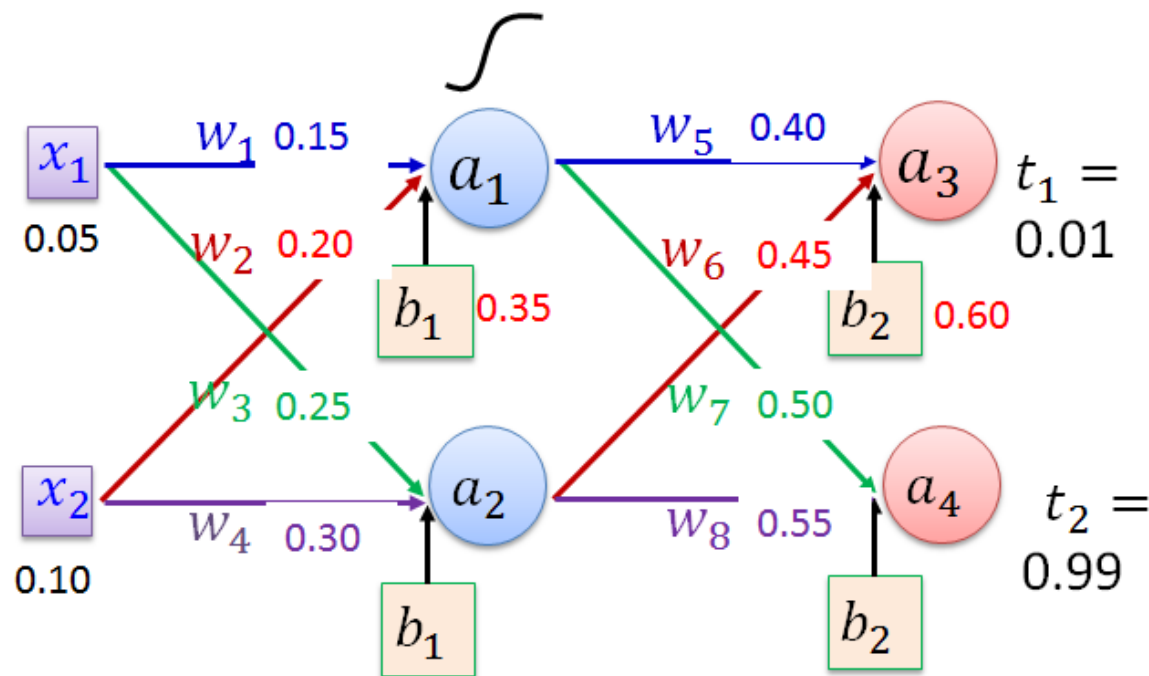
$$\mathbf{a}_1 = \sigma(\mathbf{z}_1) = [0.5933, 0.5969]^T$$

$$\mathbf{z}_2 = W_2 \mathbf{a}_1 + b_2 = [1.1059, 1.2249]^T$$

$$\mathbf{a}_2 = \sigma(\mathbf{z}_2) = [0.7514, 0.7729]^T$$

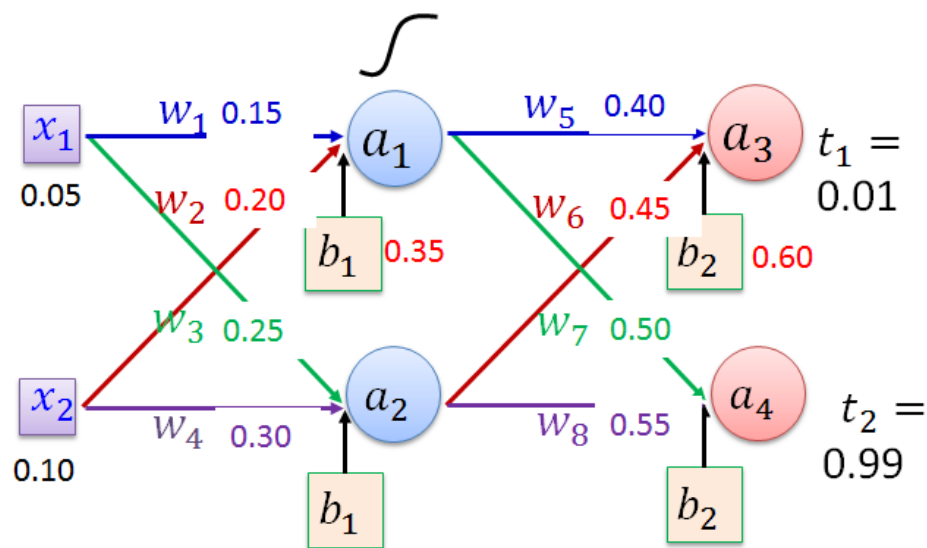
计算误差

$$E = \frac{1}{2} \|\mathbf{t} - \mathbf{a}_2\|^2, \mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$



4.3.1 Back Propagation (BP)学习算法

例4(续): 利用BP算法对神经网络参数进行学习



误差反向传播(矩阵化表达)

$$\nabla_{\mathbf{a}_2} E = \begin{bmatrix} a_3 - t_1 \\ a_4 - t_2 \end{bmatrix}, \quad \nabla_{\mathbf{z}_2} E = \sigma'(\mathbf{z}_2) \circ \nabla_{\mathbf{a}_2} E$$

$$\nabla_{W_2} E = (\sigma'(\mathbf{z}_2) \circ \nabla_{\mathbf{a}_2} E) (\mathbf{a}_1)^T$$

第2层

$$\nabla_{\mathbf{a}_1} E = (W_2)^T \boldsymbol{\delta}_2, \quad \nabla_{\mathbf{z}_1} E = \sigma'(\mathbf{z}_1) \circ \nabla_{\mathbf{a}_1} E$$

$$\nabla_{W_1} E = (\sigma'(\mathbf{z}_1) \circ \nabla_{\mathbf{a}_1} E) (\mathbf{x})^T$$

第1层

4.3.1 Back Propagation (BP)学习算法

例4(续)：利用BP算法对神经网络参数进行学习

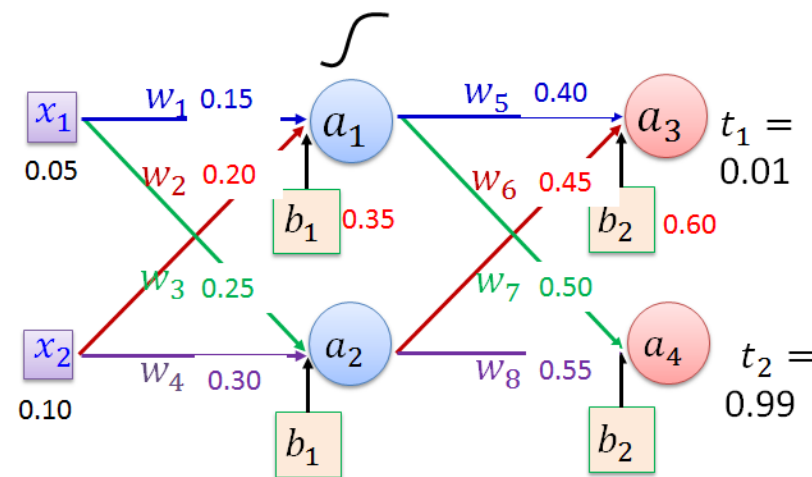
误差反向传播

$$\nabla_{\mathbf{a}_2} E = \begin{bmatrix} a_3 - t_1 \\ a_4 - t_2 \end{bmatrix} = [0.7414, -0.2371]^T$$

$$\nabla_{\mathbf{z}_2} E = \sigma'(\mathbf{z}_2) \circ \nabla_{\mathbf{a}_2} E = \begin{bmatrix} 0.1868 \\ 0.1755 \end{bmatrix} \circ \begin{bmatrix} 0.7414 \\ -0.2371 \end{bmatrix} = [0.1385, -0.0416]^T$$

$$\nabla_{W_2} E = (\sigma'(\mathbf{z}_2) \circ \nabla_{\mathbf{a}_2} E)(\mathbf{a}_1)^T = \begin{bmatrix} 0.1385 \\ -0.0416 \end{bmatrix} [0.5933, 0.5969] = \begin{bmatrix} 0.0822 & 0.0827 \\ -0.0247 & -0.0248 \end{bmatrix}$$

$$W_2 \leftarrow W_2 - \eta \nabla_{W_2} E = \begin{bmatrix} 0.3589 & 0.4087 \\ 0.5113 & 0.5614 \end{bmatrix}$$



4.3.1 Back Propagation (BP)学习算法

例4(续)：利用BP算法对神经网络参数进行学习

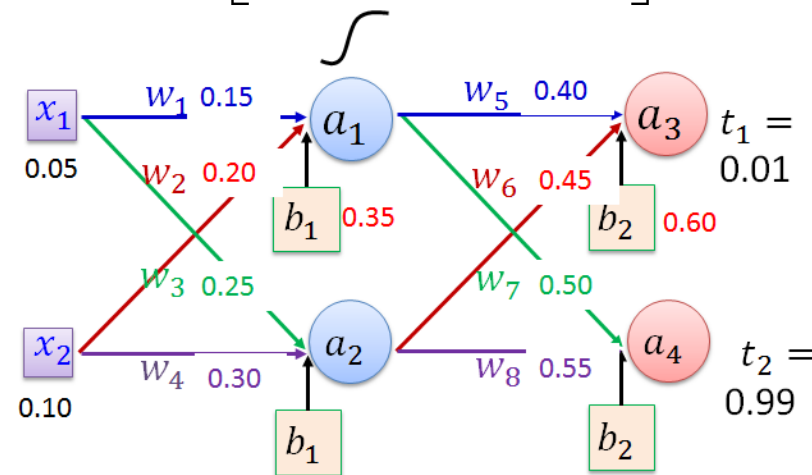
误差反向传播

$$\nabla_{\mathbf{a}_1} E = (\mathbf{W}_2)^T \boldsymbol{\delta}_2 = \begin{bmatrix} 0.40 & 0.50 \\ 0.45 & 0.55 \end{bmatrix} \begin{bmatrix} 0.1385 \\ -0.0416 \end{bmatrix} = [0.0346, 0.0394]^T$$

$$\nabla_{\mathbf{z}_1} E = \sigma'(\mathbf{z}_1) \circ \nabla_{\mathbf{a}_1} E = \begin{bmatrix} 0.2413 \\ 0.2406 \end{bmatrix} \circ \begin{bmatrix} 0.0346 \\ 0.0394 \end{bmatrix} = [0.0083, 0.0095]^T$$

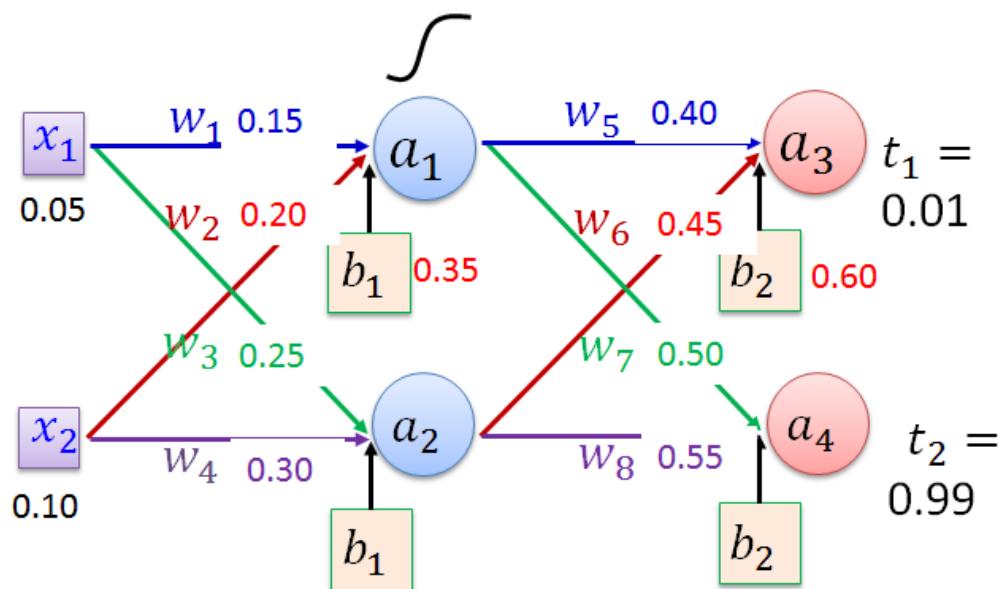
$$\nabla_{\mathbf{W}_1} E = (\sigma'(\mathbf{z}_1) \circ \nabla_{\mathbf{a}_1} E) (\mathbf{x})^T = \begin{bmatrix} 0.0083 \\ 0.0095 \end{bmatrix} [0.05, 0.10] = \begin{bmatrix} 0.00042 & 0.00083 \\ 0.00048 & 0.00095 \end{bmatrix}$$

$$\mathbf{W}_1 \leftarrow \mathbf{W}_1 - \eta \nabla_{\mathbf{W}_1} E = \begin{bmatrix} 0.1498 & 0.1996 \\ 0.2498 & 0.2995 \end{bmatrix}$$



4.3.1 Back Propagation (BP)学习算法

例4(续): 利用BP算法对神经网络参数进行学习



思考: 例3中未考虑偏置 b 的更新, 偏置 b 的更新公式是怎样的?

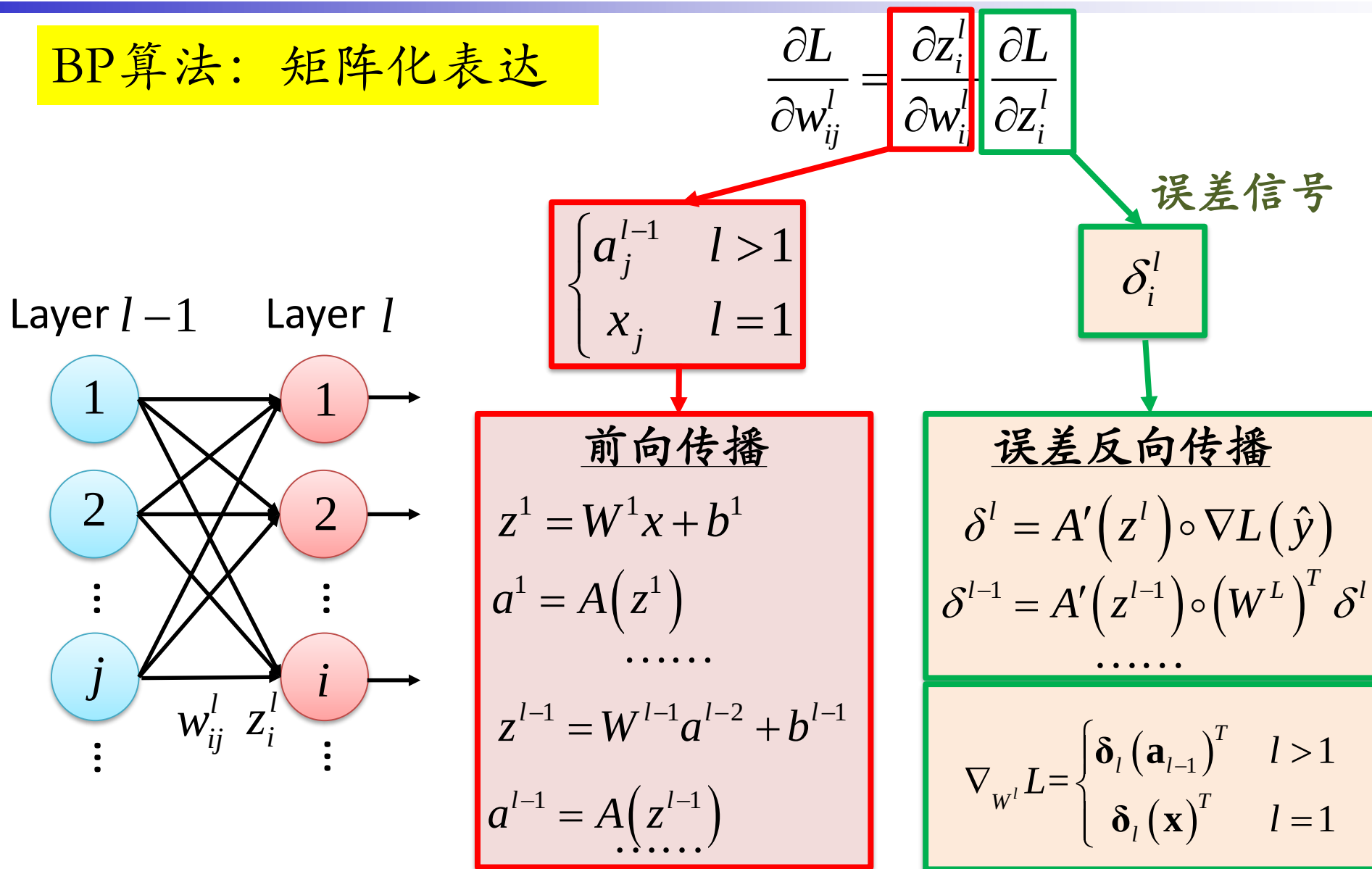
思考: 如果同时输入多个样本, BP算法推导的形式有什么变化?

思考: 如果变化误差 E 为一般损失函数, BP算法推导有什么变化?

思考: 如果变化激活函数为ReLU, BP算法推导有什么变化?

4.3.1 Back Propagation (BP)学习算法

BP算法：矩阵化表达



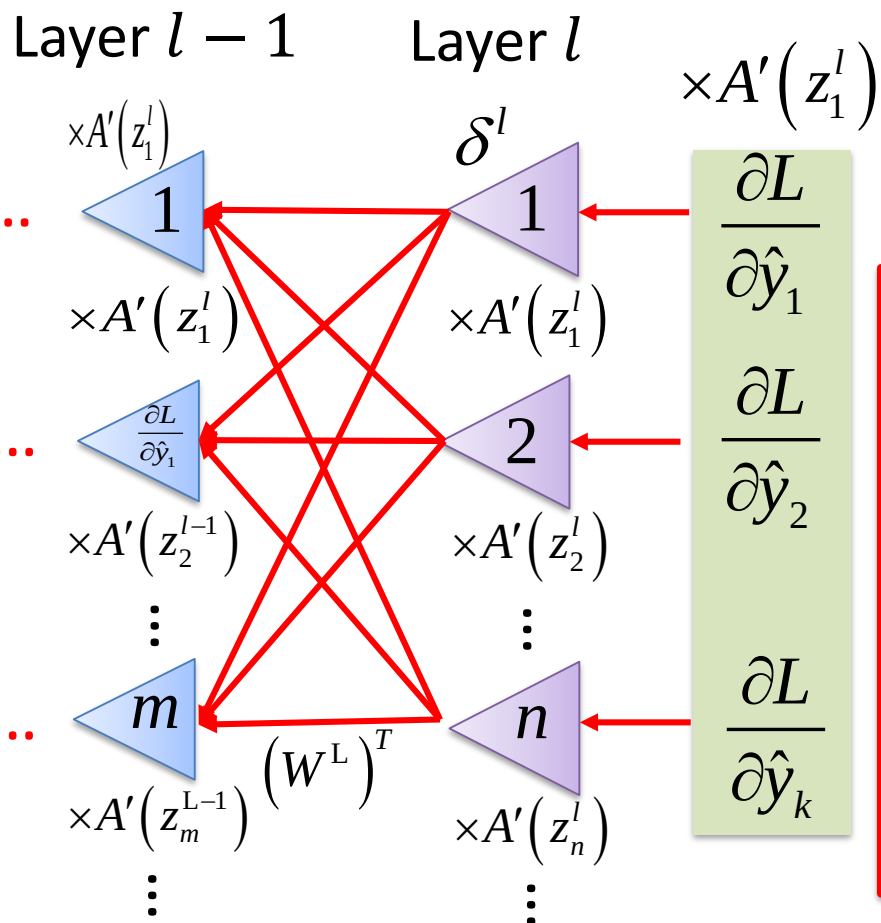
4.3.1 Back Propagation (BP)学习算法

BP算法：矩阵化表达

$$\frac{\partial L}{\partial w_{ij}^l} = \frac{\partial z_i^l}{\partial w_{ij}^l} \frac{\partial L}{\partial z_i^l}$$

误差信号

$$\delta_i^l$$



$$\begin{cases} a_j^{l-1} & l > 1 \\ x_j & l = 1 \end{cases}$$

前向传播

$$\begin{aligned} z^1 &= W^1 x + b^1 \\ a^1 &= A(z^1) \\ &\dots\dots\dots \\ z^{l-1} &= W^{l-1} a^{l-2} + b^{l-1} \\ a^{l-1} &= A(z^{l-1}) \\ &\dots\dots\dots \end{aligned}$$

误差反向传播

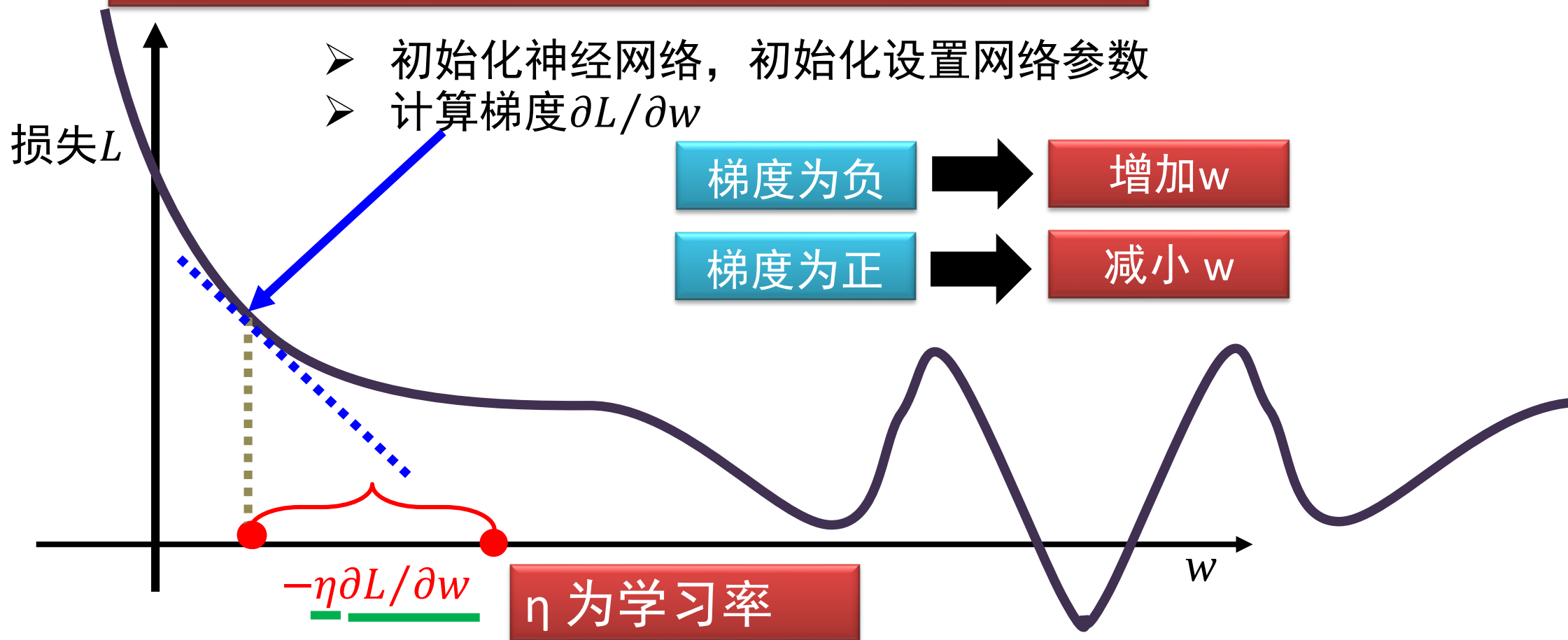
$$\begin{aligned} \delta^l &= A'(z^l) \circ \nabla L(\hat{y}) \\ \delta^{l-1} &= A'(z^{l-1}) \circ (W^L)^T \delta^l \\ &\dots\dots\dots \end{aligned}$$

$$\nabla_{w^l} L = \begin{cases} \delta_l (a_{l-1})^T & l > 1 \\ \delta_l (x)^T & l = 1 \end{cases}$$

4.3.2 训练方法：梯度下降

神经网络参数 $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

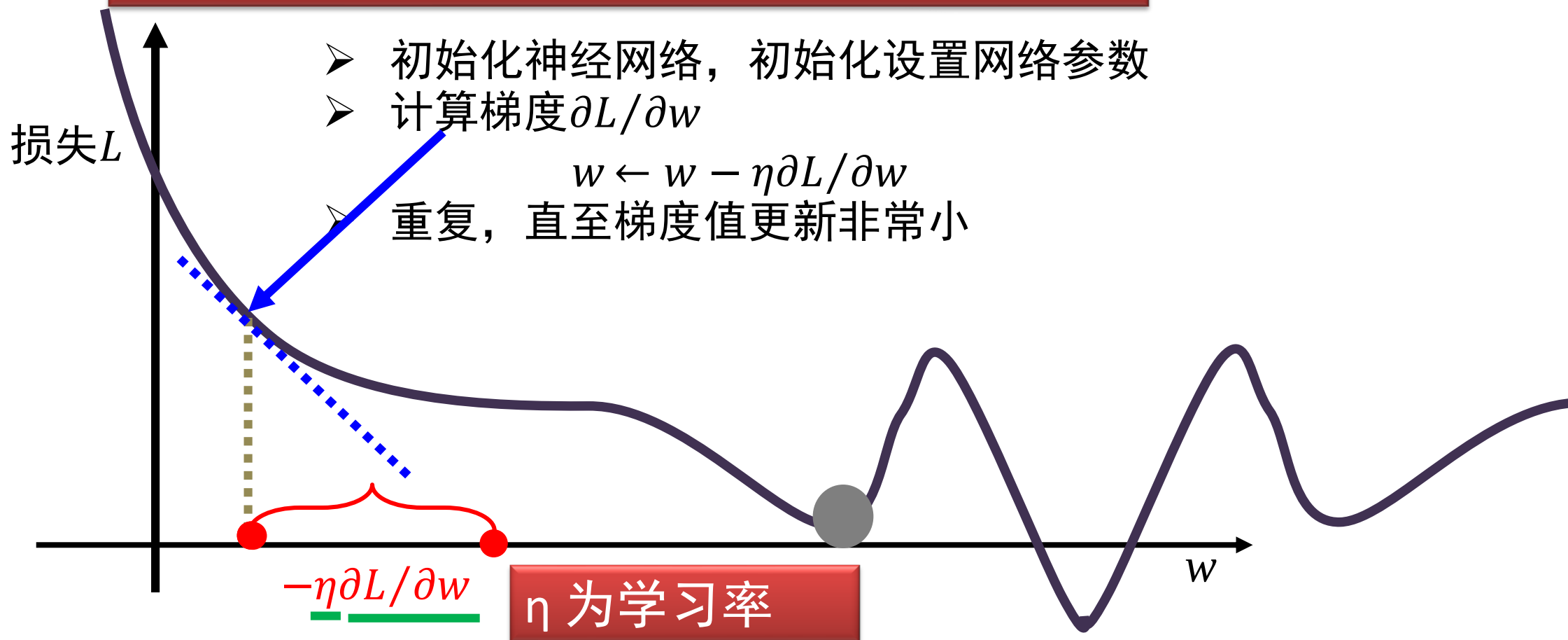
学习获取使损失函数最小化的网络参数 θ^*



4.3.2 训练方法：梯度下降

神经网络参数 $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

学习获取使损失函数最小化的网络参数 θ^*



4.3.2 训练方法：梯度下降

❑ Batch Gradient Descent (BGD)

- ✓ 利用所有样本计算梯度

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

速度慢，数据量大时内存不足

❑ Stochastic Gradient Descent (SGD)

- ✓ 随机选取单个样本计算梯度

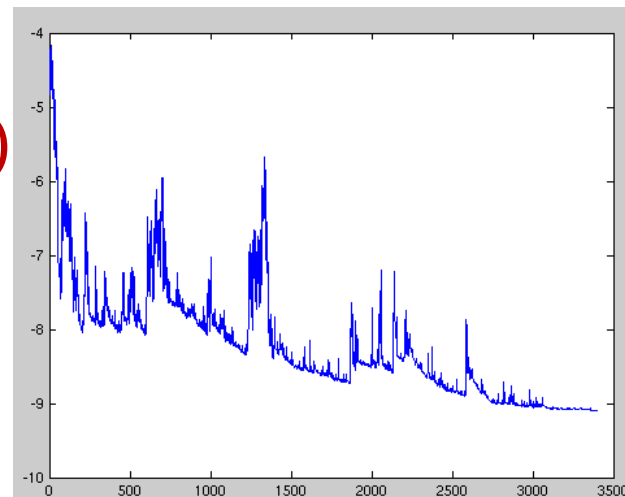
$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta, x_i, y_i)$$

方差大，损失函数震荡严重

❑ Mini-batch Stochastic Gradient Descent

- ✓ 随机选取mini-batch样本计算梯度

$$\theta \leftarrow \theta - \eta \left[\frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L(\theta, x_i, y_i) \right]$$



4.3.2 训练方法：梯度下降

□ Mini-batch Stochastic Gradient Descent (MSGD)

✓ 随机选取mini-batch样本计算梯度

$$\theta_{t+1} \leftarrow \theta_t - \eta_t \left[\frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L(\theta, x_i^t, y_i^t) \right]$$

	BGD	SGD	M-SGD
单次迭代样本数	整个数据集	单个样本	整个数据集的一个子集
算法复杂度	高	低	一般
时效性	低	一般	一般
收敛性	稳定	不稳定	较稳定

4.3.2 训练方法：梯度下降

□ Mini-batch Stochastic Gradient Descent(MSGD)

✓ 随机选取mini-batch样本计算梯度

$$\theta_{t+1} \leftarrow \theta_t - \eta_t \left[\frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L(\theta, x_i^t, y_i^t) \right]$$

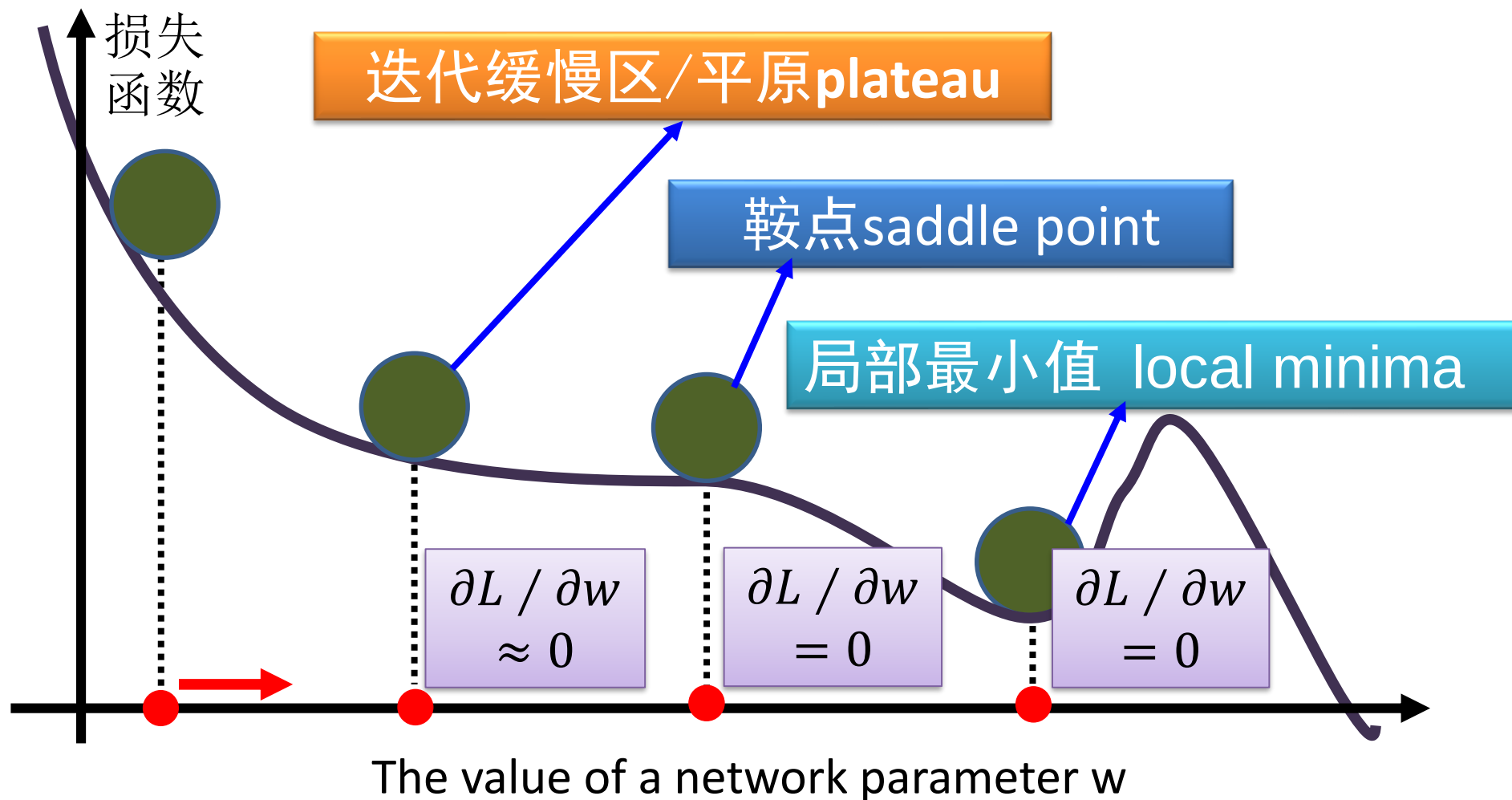
Mini-batch SGD

for $t = 1:T$

1. 随机采样Mini-batch的样本
 2. 前向计算：计算损失
 3. 后向操作：利用Back-Propagation计算梯度
 4. 网络参数更新：根据步骤3. 的梯度更新网络参数
- end*

4.3.2 训练方法：梯度下降

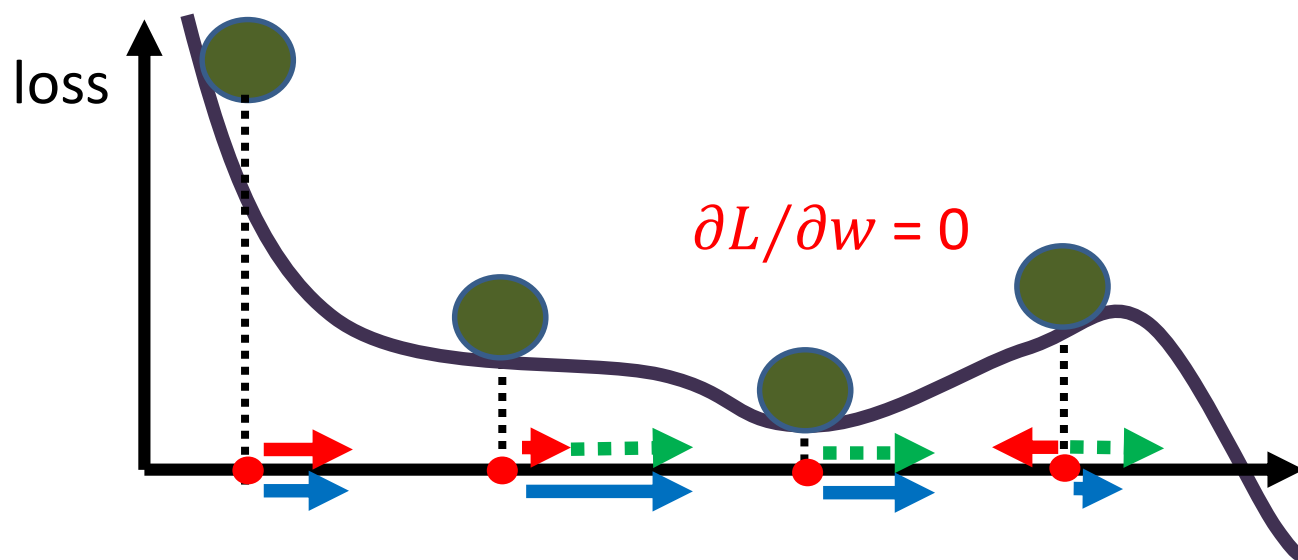
▣ 梯度下降可能存在的问题：局部最优



4.3.2 训练方法：梯度下降

Momentum: 避免随机梯度下降陷入局部最优

$$\beta_{t+1} = \mu\beta_t - \eta\nabla_{\theta}L(\theta_t), \quad \theta_{t+1} = \theta_t + \beta_{t+1}$$



4.3.2 训练方法：梯度下降

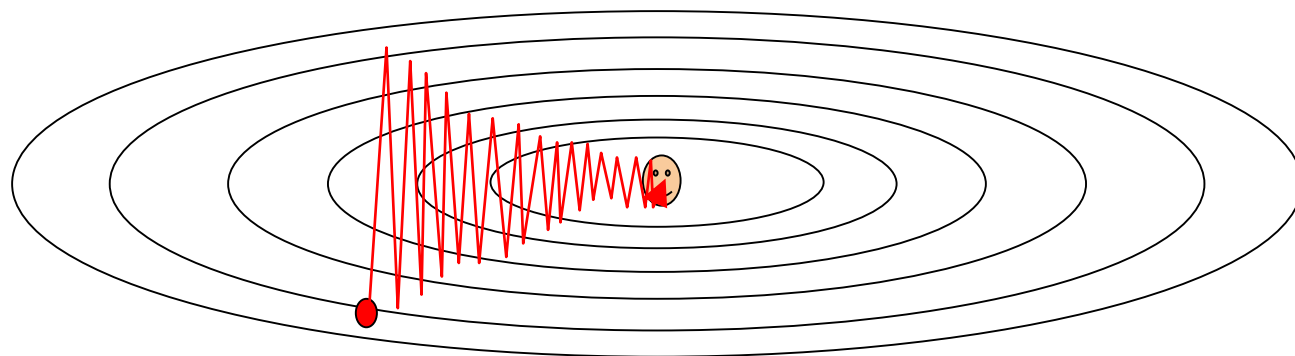
Weight-decay: 在目标函数上增加对较大系数的惩罚项，基于权重向量的L2或L1范数，避免过拟合

$$\tilde{L}(w) = L(w) + \frac{\lambda}{2} \sum_i w_i^2$$
$$\Delta w_i \propto -\frac{\partial L}{\partial w_i} - \lambda w_i$$

λ 一般设置较小，
例如0.0005

4.3.3 优化方法：SGD-M

➤ 传统SGD在沟壑时容易出现震荡：



➤ Momentum 更新值：

$$v_{t+1} = \mu v_t + \eta \nabla_{\theta} L(\theta_t), \quad \theta_{t+1} = \theta_t - v_{t+1}$$

其中 v_t 是momentum, $\nabla_{\theta} L(\theta_t)$ 是minibatch 梯度, $\mu \in [0,1]$ 是momentum对应的常系数; η 是学习率。

参数中梯度方向不大的维度加速更新，同时减少在梯度方向变化大的维度上的更新幅度。

4.3.3 优化方法：ADAGRAD (Duchi et al., 2011)

➤ ADAGRAD: Adaptive Gradient

- ◆ 利用梯度平方累加和(**cumulative sum** of squared gradients)的平方根

$$c_t = \sum_{j=1}^t \left(\nabla_{\theta} L(\theta_j) \right)^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\nabla_{\theta} L(\theta_t)}{\sqrt{c_t + \varepsilon}}$$

- ◆ 对低频参数做较大的更新，对高频参数做较小的更新
- ◆ 提升了SGD的鲁棒性，稀疏数据表现好
- ◆ Note: $c^{(t)}$ tends to grow linearly with time t , so ADAGRAD decreases its effective learning rate over time as $1/\sqrt{t}$

4.3.3 优化方法：RMSprop (Hinton et al., 2012)

➤ RMSprop: Root-Mean-Square Prop

- ◆ 问题：不同变量的梯度幅度不一致
- ◆ 思路：利用梯度平方加权取倒数对梯度进行加权
- ◆ 定义RMS (Root-Mean-Squared) 梯度

$$s_t = \gamma s_{t-1} + (1 - \gamma) (\nabla_{\theta} L(\theta_t))^2$$

Mean-squared

$$\theta_{t+1} = \theta_t - \eta \frac{\nabla_{\theta} L(\theta_t)}{\sqrt{s_t} + \varepsilon}$$

Root

- ◆ 保证各维度导数在一个量级，减少摆动
- ◆ Hinton建议： $\gamma = 0.9$, $\eta = 0.001$.

4.3.3 优化方法：ADAM (Kingma and Ba, 2014)

➤ ADAM: Adaptive Momentum Optimization

- ◆ 结合Momentum和RMSProp
- ◆ 计算梯度和梯度平方的平滑平均

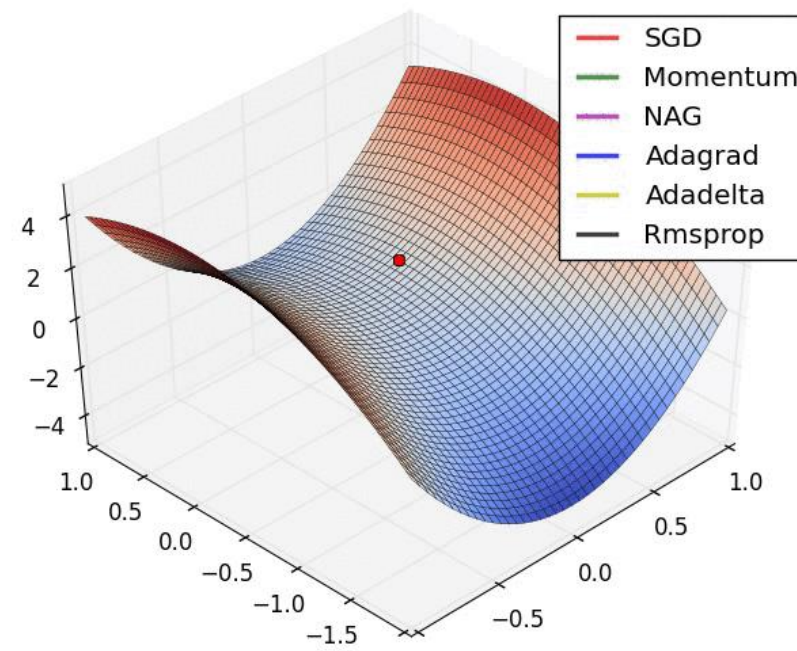
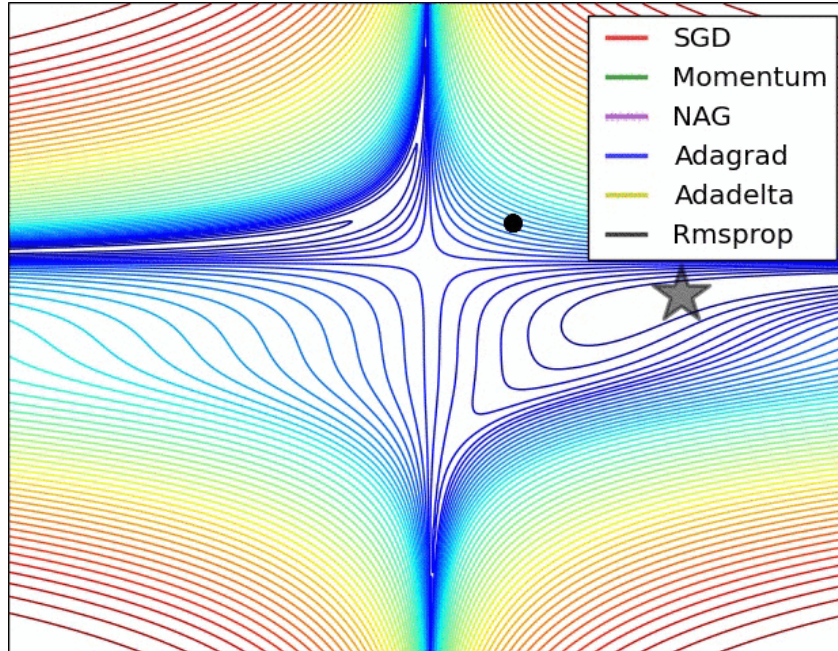
$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) \nabla_{\theta} L(\theta_t)$$

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) (\nabla_{\theta} L(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{v_t}{\sqrt{s_t + \epsilon}}$$

- ◆ 参数设置: $\beta_1=0.9$, β_2 接近于1, 例如 0.9999

4.3.3 优化方法比较



From A. Karpathy's cs231n notes.

4.3.4 参数初始化及学习率选取

- 深度神经网络的参数初始化方法：

- 一般而言，偏置初始化为0；
- 重点考虑权重初始化方法

思考：为什么权重不采用常数/零值初始化？

随机初始化：权重初始化为0，标准差为 σ $w_i^l \sim \mathcal{N}(0, \sigma^2)$

Xavier初始化：

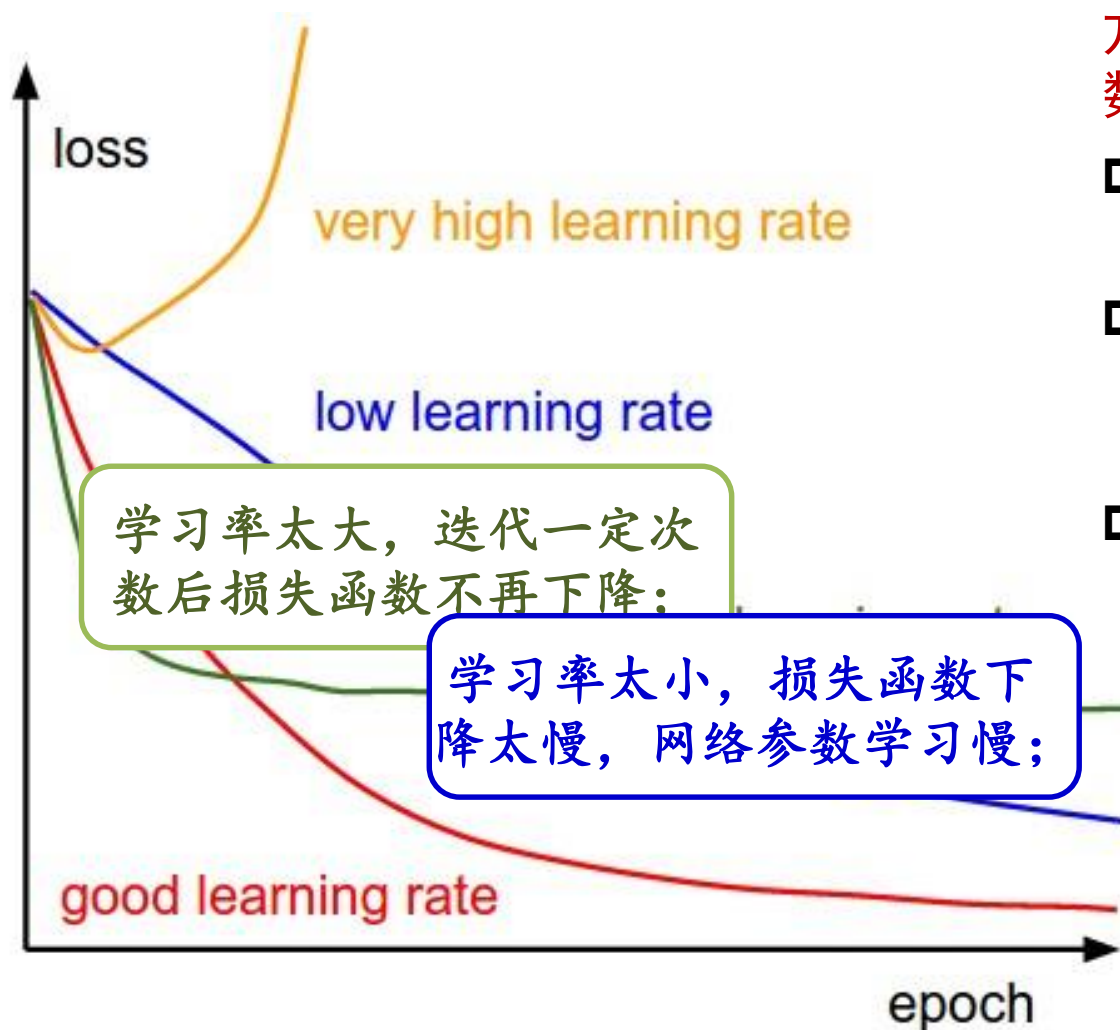
$$w_i^l \sim \mathcal{N}\left(0, \frac{2}{n_l + n_{l-1}}\right) \quad w_i^l \sim U\left[-\sqrt{\frac{6}{n_l + n_{l-1}}}, \sqrt{\frac{6}{n_l + n_{l-1}}}\right]$$

正态分布 均匀分布

Kaiming初始化：

$$w_i^l \sim \mathcal{N}\left(0, \frac{2}{n_l}\right) \quad w_i^l \sim U\left[-\sqrt{\frac{6}{n_l}}, \sqrt{\frac{6}{n_l}}\right]$$

4.3.4 学习率选取



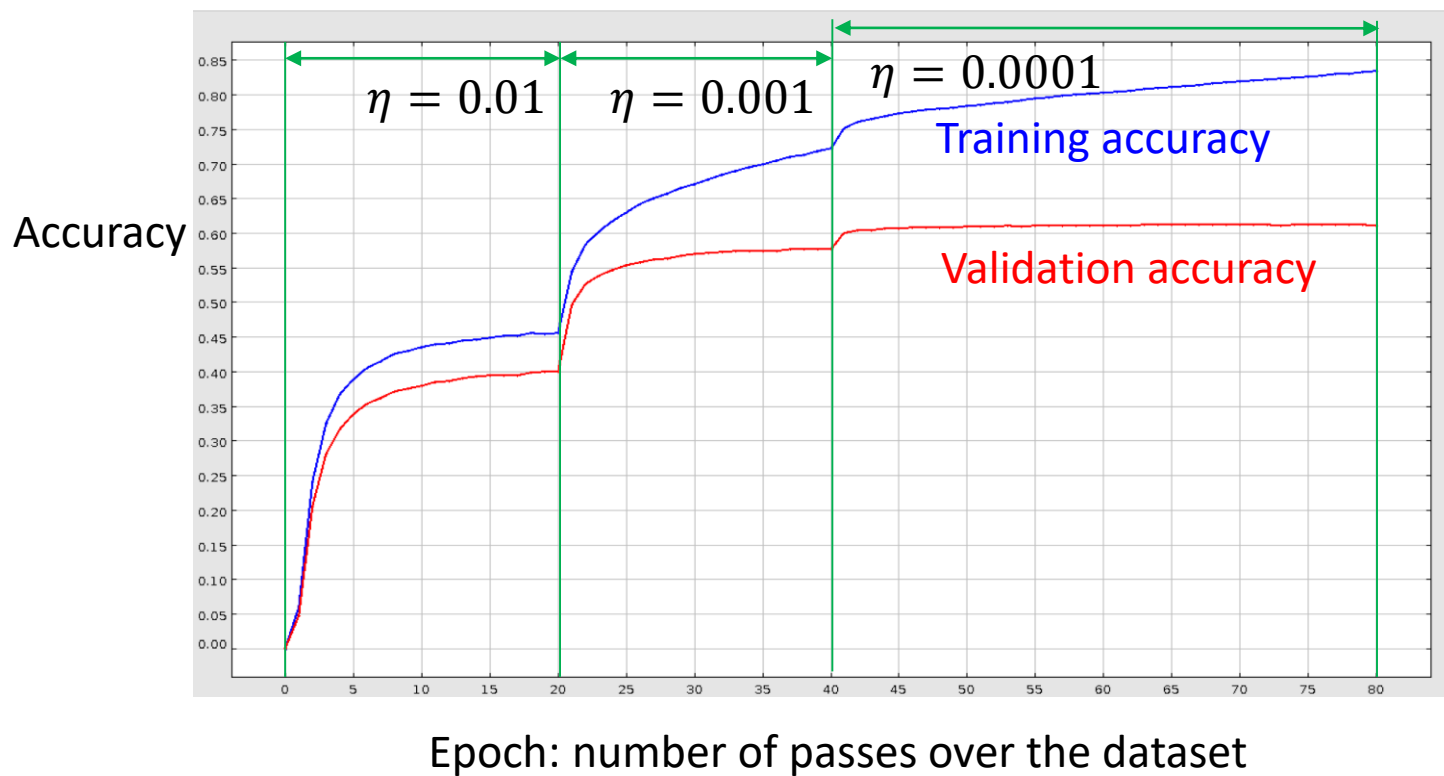
方案：在训练过程中根据迭代次数逐步调整学习率

- ❑ 在训练初期，离目标损失远，设置较大学习率
- ❑ 训练一段时间后，已到达目标损失附近，距离最优点近，选择较小学习率
- ❑ 例如：1/t decay: $\eta^t = \eta / \sqrt{t + 1}$

学习率设置与多因素有关，例如初始值，需要在实验中进行调整

4.3.4 学习率选取

利用ImageNet数据集训练Alexnet. η = learning rate.



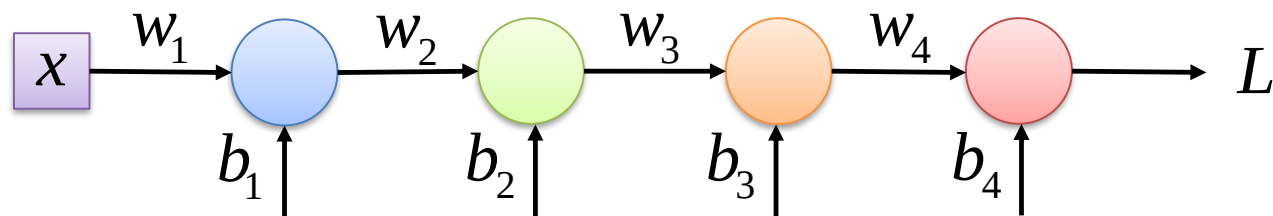
4.3.5 训练中可能遇到的问题：梯度消失 Vanishing Gradient Problem

□ 以基于Sigmoid激活函数的深度神经网络为例：

□ Sigmoid激活函数梯度计算

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)(1-\sigma(z)) \in \left(0, \frac{1}{4}\right]$$

□ 多隐含层深度神经网络：Sigmoid激活函数



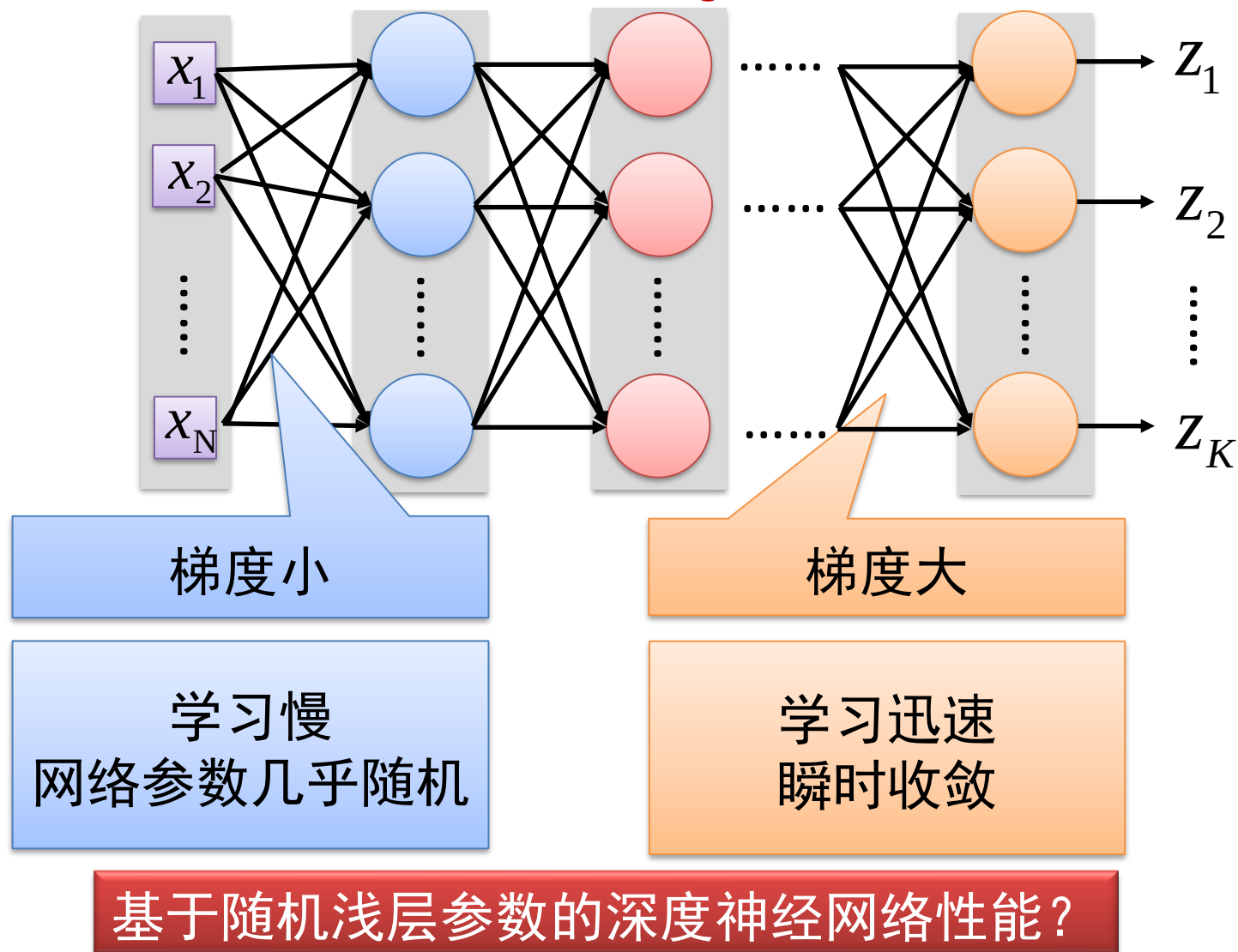
$$\frac{\partial L}{\partial b_1} = \sigma'(b_1)w_2\sigma'(b_2)w_3\sigma'(b_3)w_4\frac{\partial L}{\partial b_4}$$

$$\frac{\partial L}{\partial b_1} \leq \left(\frac{1}{4}\right)^4 w_2 w_3 w_4 \frac{\partial L}{\partial b_4}$$

随着网络深度的加深，
梯度衰减更加严重

4.3.5 训练中可能遇到的问题：梯度消失 Vanishing Gradient Problem

□ 深度神经网络，激活函数为Sigmoid:



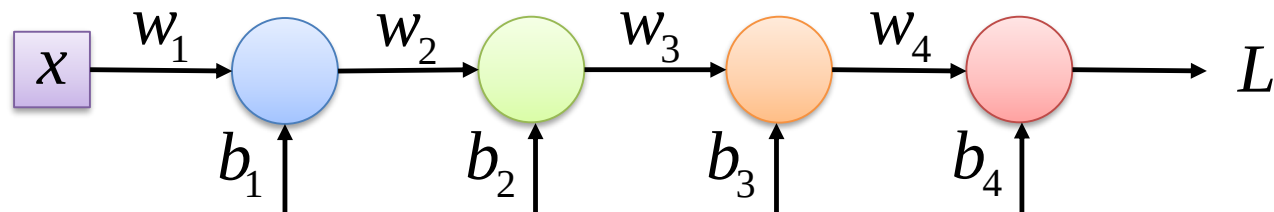
4.3.5 训练中可能遇到的问题：梯度消失 Vanishing Gradient Problem

➤ 如何解决梯度消失的问题？

- ◆ 分层预训练：Hinton于2006年提出，利用无监督数据进行分层预训练，再利用有监督数据调整网络参数；
- ◆ ReLU激活函数：Sigmoid和tanh激活函数存在较严重的梯度消失问题；
- ◆ Batch Normalization: 逐层对数据进行尺度归一化；
- ◆ 辅助损失函数：对浅层神经元输出建立辅助损失函数，直接传递梯度，GoogleNet中使用；

4.3.5 训练中可能遇到的问题：梯度爆炸 Exploding Gradient Problem

➤ 以基于Sigmoid激活函数的深度神经网络为例：



$$\frac{\partial L}{\partial b_1} = \sigma'(b_1)w_2\sigma'(b_2)w_3\sigma'(b_3)w_4\frac{\partial L}{\partial b_4}, |w_j\sigma'(b_j)| > 1$$

◆ 损失函数值出现大幅度震荡或者变成NaN

➤ 如何解决梯度爆炸的问题？

- ◆ 重新设计网络模型/更换激活函数
- ◆ RNN中使用ReLU激活函数可减少梯度爆炸
- ◆ 使用梯度截断(gradient clipping)
- ◆ 权重正则化，增加L1/L2正则惩罚项

以下不属于ReLU激活函数优点的是

- ☐ A 计算效率高
- ☐ B 相比Sigmoid, 可避免梯度消失
- ☐ C 可产生稀疏的特征表达
- ☒ D 存在dead neurons

试分析双曲正切 $\text{Tan H}(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ 的导数形式为 ()

☒ A $\frac{4e^{-2z}}{(1+e^{-2z})^2}$

☐ B $\frac{e^{-z}}{(1+e^{-z})}$

☐ C $\frac{4e^{-2z}}{(1+e^{-2z})}$

☐ D $\frac{e^{-2z}}{(1+e^{-2z})}$

提交

在深度神经网络中，经常使用ReLU激活函数，其形式为 $f(z) = \max(0, z)$

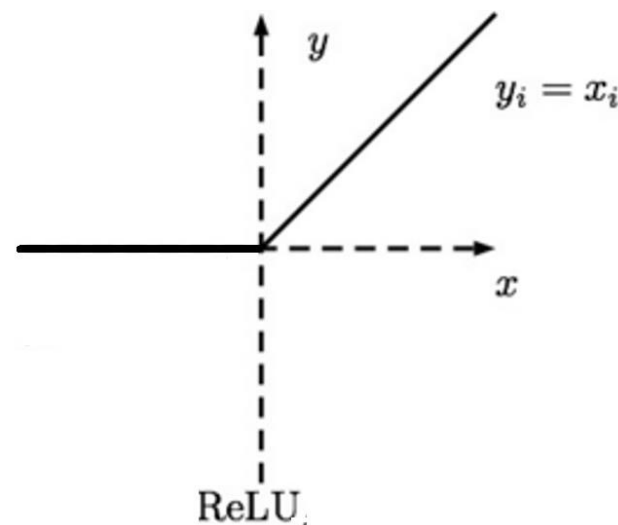
试问下面()可作为其在实际编程中的导数。

☒ A $f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$

☐ B $f'(z) = 1$

☐ C $f'(z) = 0$

☐ D 不存在



参考文献

1. <http://deeplearning.net>
2. Raul Rojas. Neural networks: a systematic introduction. Springer, 1996.
3. S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural computation, vol. 9, no. 8, pp. 1735–1780, 1997.
4. A. Graves et al., Supervised sequence labelling with recurrent neural networks. Springer, 2012, vol. 385.
5. A. Graves, S. Fern´andez, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in Proceedings of the International Conference on Machine learning, 2006, pp. 369–376.
6. K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” arXiv preprint arXiv:1406.1078, 2014.
7. J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” arXiv preprint arXiv:1412.3555, 2014.
8. M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” tensorflow.org, 2015.
9. M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” IEEE Transactions on Signal Processing, vol. 45, no. 11, pp. 2673–2681, 1997.
10. N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” Journal of Machine Learning Research, vol. 15, no. 1, pp. 1929–1958, 2014.
11. K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
12. Wu Y, Schuster M, Chen Z, et al. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation[J]. 2016.
13. S. Han, J. Pool, S. Narang, H. Mao, S. Tang, E. Elsen, B. Catanzaro, J. Tran, and W. J. Dally, “DSD: Regularizing deep neural networks with dense-sparse-dense training flow,” arXiv preprint arXiv: 1607.04381, 2016.

谢谢大家！

欢迎提出宝贵意见！

BP算法：数学基础

➤ 向量求导：

- ◆ 设向量 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, $f(\mathbf{x})$ 是关于向量 $\mathbf{x}$ 的标量函数，则实值标量函数 $f(\mathbf{x})$ 关于 $\mathbf{x}$ 的偏导可表示为如下形式：

$$\frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

列向量偏导算子

$$D_{\mathbf{x}} f \stackrel{\text{def}}{=} \frac{\partial f}{\partial \mathbf{x}^T} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \dots & \frac{\partial f}{\partial x_n} \end{bmatrix}$$

行向量偏导

列向量偏导算子/梯度算子

$$\nabla_{\mathbf{x}} \stackrel{\text{def}}{=} \frac{\partial}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial}{\partial x_1} & \frac{\partial}{\partial x_2} & \dots & \frac{\partial}{\partial x_n} \end{bmatrix}^T$$

行向量偏导算子

$$D_{\mathbf{x}} \stackrel{\text{def}}{=} \frac{\partial}{\partial \mathbf{x}^T} = \begin{bmatrix} \frac{\partial}{\partial x_1} & \frac{\partial}{\partial x_2} & \dots & \frac{\partial}{\partial x_n} \end{bmatrix}$$

BP算法：数学基础

➤ Jacobian矩阵：

- ◆ 设向量 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, $F(\mathbf{x})$ 是关于向量 $\mathbf{x}$ 的向量函数, $F(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})]^T$ 则向量函数 $F(\mathbf{x})$ 关于 $\mathbf{x}$ 的行向量偏导可表示为

$$D_{\mathbf{x}} \mathbf{F}(\mathbf{x}) \stackrel{\text{def}}{=} \frac{\partial \mathbf{F}}{\partial \mathbf{x}^T} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Jacobian矩阵

BP算法：数学基础

➤ Jacobian矩阵：

◆ 设 $X = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix} = [x_{ij}]_{i=1,j=1}^{mn}$

◆ $f(X)$ 是关于矩阵 X 的标量函数

◆ $f(X)$ 关于 X 的偏导可表示为

$$D_X f(X) \stackrel{\text{def}}{=} \frac{\partial f}{\partial X^T} = \begin{bmatrix} \frac{\partial f}{\partial x_{11}} & \frac{\partial f}{\partial x_{21}} & \cdots & \frac{\partial f}{\partial x_{m1}} \\ \frac{\partial f}{\partial x_{12}} & \frac{\partial f}{\partial x_{22}} & \cdots & \frac{\partial f}{\partial x_{m1}} \\ \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f}{\partial x_{1n}} & \frac{\partial f}{\partial x_{2n}} & \cdots & \frac{\partial f}{\partial x_{mn}} \end{bmatrix} \in \mathfrak{R}^{n \times m}$$

Jacob i an矩阵

BP算法：数学基础

➤ 矩阵列向量化：

◆ 设 $X = [x_{ij}]_{i=1,j=1}^{mn}$

◆ $F(X)$ 是关于矩阵 X 的矩阵函数， $F(X) = [f_{kl}(X)]_{k=1,l=1}^{pq}$

◆ 记： $vec(X)$ 为 **列向量化标记**，即

$$vec(X) \stackrel{def}{=} [x_{11}, x_{21}, \dots, x_{m1}, x_{12}, \dots, x_{mn}]^T$$

$$vec(F(X)) \stackrel{def}{=} [f_{11}(X), f_{21}(X), \dots, f_{p1}(X), f_{12}(X), \dots, f_{pq}(X)]^T$$

BP算法：数学基础

◆ 设 $X = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix} = [x_{ij}]_{i=1,j=1}^{mn}$

◆ $F(X)$ 是关于矩阵 X 的矩阵函数, $F(X) = [f_{kl}(X)]_{k=1,l=1}^{pq}$

◆ $F(X)$ 关于 X 的 **Jacobian** 矩阵可表示为

$$D_X \mathbf{F} \stackrel{\text{def}}{=} \frac{\partial \text{vec}(F(X))}{\partial \text{vec}(X)^T} = \begin{bmatrix} \frac{\partial f_{11}}{\partial x_{11}} & \cdots & \frac{\partial f_{11}}{\partial x_{m1}} & \cdots & \frac{\partial f_{11}}{\partial x_{1n}} & \cdots & \frac{\partial f_{11}}{\partial x_{mn}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_{p1}}{\partial x_{11}} & \cdots & \frac{\partial f_{p1}}{\partial x_{m1}} & \cdots & \frac{\partial f_{p1}}{\partial x_{1n}} & \cdots & \frac{\partial f_{p1}}{\partial x_{mn}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_{1q}}{\partial x_{11}} & \cdots & \frac{\partial f_{1q}}{\partial x_{m1}} & \cdots & \frac{\partial f_{1q}}{\partial x_{1n}} & \cdots & \frac{\partial f_{1q}}{\partial x_{mn}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_{pq}}{\partial x_{11}} & \cdots & \frac{\partial f_{pq}}{\partial x_{m1}} & \cdots & \frac{\partial f_{pq}}{\partial x_{1n}} & \cdots & \frac{\partial f_{pq}}{\partial x_{mn}} \end{bmatrix} \in \mathfrak{R}^{pq \times mn}$$

Jacobian
矩阵

$$\nabla_X \mathbf{F} \stackrel{\text{def}}{=} (D_X \mathbf{F})^T$$

梯度矩阵为
Jacobian 矩阵
的转置

BP算法：数学基础

◆ 例1: $F(X) = AXB$, 其中 $A \in \mathcal{R}^{p \times m}, X \in \mathcal{R}^{m \times n}, B \in \mathcal{R}^{n \times q}$,

$F(X) = [f_{kl}(X)]_{k=1,l=1}^{pq}$, 其Jacobian矩阵为?

◆ 解答:

$$\frac{\partial f_{kl}}{\partial x_{ij}} = \frac{\partial \left(\sum_{u=1}^m \sum_{v=1}^n a_{ku} x_{uv} b_{vl} \right)}{\partial x_{ij}} = b_{jl} a_{ki}$$

注: Kronecker 积

$$A \otimes B \stackrel{\text{def}}{=} \begin{bmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{bmatrix}$$

$$b_{11}a_{11}, b_{11}a_{12}, \dots, b_{11}a_{1m}, b_{21}a_{11}, b_{21}a_{12}, \dots, b_{21}a_{1m}, \dots$$

$$D_X \mathbf{F} = \begin{bmatrix} \frac{\partial f_{11}}{\partial x_{11}} & \cdots & \frac{\partial f_{11}}{\partial x_{m1}} & \cdots & \frac{\partial f_{11}}{\partial x_{1n}} & \cdots & \frac{\partial f_{11}}{\partial x_{mn}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_{p1}}{\partial x_{11}} & \cdots & \frac{\partial f_{p1}}{\partial x_{m1}} & \cdots & \frac{\partial f_{p1}}{\partial x_{1n}} & \cdots & \frac{\partial f_{p1}}{\partial x_{mn}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_{1q}}{\partial x_{11}} & \cdots & \frac{\partial f_{1q}}{\partial x_{m1}} & \cdots & \frac{\partial f_{1q}}{\partial x_{1n}} & \cdots & \frac{\partial f_{1q}}{\partial x_{mn}} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_{pq}}{\partial x_{11}} & \cdots & \frac{\partial f_{pq}}{\partial x_{m1}} & \cdots & \frac{\partial f_{pq}}{\partial x_{1n}} & \cdots & \frac{\partial f_{pq}}{\partial x_{mn}} \end{bmatrix}$$

$$= B^T \otimes A \in \mathcal{R}^{pq \times mn}$$

BP算法：数学基础

- ◆ 例1: $F(X) = AXB$, 其中 $A \in \mathcal{R}^{p \times m}, X \in \mathcal{R}^{m \times n}, B \in \mathcal{R}^{n \times q}$,
 $F(X) = [f_{kl}(X)]_{k=1,l=1}^{pq}$, 其Jacobian矩阵为?

- ◆ 解答(续):

$$\frac{\partial f_{kl}}{\partial x_{ij}} = \frac{\partial \left(\sum_{u=1}^m \sum_{v=1}^n a_{ku} x_{uv} b_{vl} \right)}{\partial x_{ij}} = b_{jl} a_{ki}$$

$$\mathbf{F}(X) = AXB \Rightarrow D_X \mathbf{F} \stackrel{\text{def}}{=} B^T \otimes A, \nabla_X \mathbf{F} \stackrel{\text{def}}{=} B \otimes A^T$$

$$\mathbf{F}(X) = AX \Rightarrow D_X \mathbf{F} \stackrel{\text{def}}{=} I_n \otimes A, \nabla_X \mathbf{F} \stackrel{\text{def}}{=} I_n \otimes A^T$$

$$\mathbf{F}(X) = XB \Rightarrow D_X \mathbf{F} \stackrel{\text{def}}{=} B^T \otimes I_m, \nabla_X \mathbf{F} \stackrel{\text{def}}{=} B^T \otimes I_m$$

BP算法：数学基础

来源：张贤达《矩阵分析及应用》
(第2版), pp 151.

表 3.1.2 矩阵函数 $F(X)$ 的偏导与 Jacobian 矩阵、梯度矩阵的关系

矩阵函数	$\partial f_{kl} / \partial x_{ij}$	Jacobian 矩阵 $D_X F(X)$	梯度矩阵 $\nabla_X F(X)$
AXB	$b_{jl} a_{ki}$	$B^T \otimes A$	$B \otimes A^T$
$A^T X B$	$b_{jl} a_{ik}$	$B^T \otimes A^T$	$B \otimes A$
AXB^T	$b_{lj} a_{ki}$	$B \otimes A$	$B^T \otimes A^T$
$A^T X B^T$	$b_{lj} a_{ik}$	$B \otimes A^T$	$B^T \otimes A$
$AX^T B$	$b_{il} a_{kj}$	$(B^T \otimes A) K_{mn}$	$K_{nm} (B \otimes A^T)$
$A^T X^T B$	$b_{il} a_{jk}$	$(B^T \otimes A^T) K_{mn}$	$K_{nm} (B \otimes A)$
$AX^T B^T$	$b_{li} a_{kj}$	$(B \otimes A) K_{mn}$	$K_{nm} (B^T \otimes A^T)$
$A^T X^T B^T$	$b_{li} a_{jk}$	$(B \otimes A^T) K_{mn}$	$K_{nm} (B^T \otimes A)$
XX^T	$\delta_{li} x_{kj} + x_{lj} \delta_{ki}$	$(K_{mm} + I_{m^2})(X \otimes I_m)$	$(X^T \otimes I_m)(K_{mm} + I_{m^2})$
$X^T X$	$x_{il} \delta_{kj} + \delta_{lj} x_{ik}$	$(K_{nn} + I_{n^2})(I_n \otimes X^T)$	$(I_n \otimes X)(K_{nn} + I_{n^2})$
$X^T B X$	$(BX)_{il} \delta_{kj} + \delta_{lj} (X^T B)_{ki}$	$((BX)^T \otimes I_n) K_{mn}$ $+ (I_n \otimes (X^T B))$	$K_{nm} ((BX) \otimes I_n)$ $+ (I_n \otimes (B^T X))$
$XB X^T$	$(BX^T)_{jl} \delta_{ki} + \delta_{li} (XB)_{kj}$	$(XB^T) \otimes I_m$ $+ (I_m \otimes (XB)) K_{mn}$	$(BX^T) \otimes I_m$ $+ K_{nm} (I_m \otimes (XB)^T)$

BP算法：数学基础

- ◆ 例2: $F(X) = [\sigma(x_{ij})]_{i=1,j=1}^{mn}$, 为逐元素运算, 其Jacobian矩阵为?

- ◆ 解答:

$$\frac{\partial f_{kl}}{\partial x_{ij}} = \frac{\partial \sigma(x_{kl})}{\partial x_{ij}} = \begin{cases} \sigma'(x_{ij}) & k=i, l=j \\ 0 & otherwise \end{cases}$$

$$D_X \mathbf{F} \stackrel{def}{=} diag[\sigma'(x_{11}), \sigma'(x_{21}), \dots, \sigma'(x_{mn})] \in \mathbb{R}^{mn \times mn}$$

为对角阵

BP算法：数学基础

➤ 矩阵微分：

◆ 设 $X = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix} = [x_{ij}]_{i=1,j=1}^{mn}$

◆ 定义矩阵微分 $dX = [dx_{ij}]_{i=1,j=1}^{mn}$

◆ 1: 矩阵迹的微分 $dtr(X) = tr(dX)$

◆ 2: 常数值与矩阵 X 的微分 $d(\alpha X) = \alpha dX$

◆ 3: 常数值与矩阵 X 的微分 $d(AXB) = A(dX)B$

◆ 4: 矩阵函数 U, V, W 乘积的微分矩阵为

◆ $d(UV) = d(U)V + UdV$

◆ $d(UVW) = d(U)VW + U(dV)W + UVdW$

BP算法：数学基础

➤ 矩阵微分与Jacobian矩阵：

- ◆ 设 $X = [x_{ij}]_{i=1,j=1}^{mn}$
- ◆ $F(X)$ 是关于矩阵 X 的矩阵函数， $F(X) = [f_{kl}(X)]_{k=1,l=1}^{pq}$
- ◆ 记： $vec(X)$ 为列向量化标记，即

$$vec(X) \stackrel{def}{=} [x_{11}, x_{21}, \dots, x_{m1}, x_{12}, \dots, x_{mn}]^T$$

$$vec(F(X)) \stackrel{def}{=} [f_{11}(X), f_{21}(X), \dots, f_{p1}(X), f_{12}(X), \dots, f_{pq}(X)]^T$$

- ◆ 则 $dvec(F(X)) = J \cdot dvec(X)$

Jacobian矩阵 $D_X \mathbf{F}$

$\nabla_X \mathbf{F} \stackrel{def}{=} (D_X \mathbf{F})^T$ 梯度矩阵为Jacobian矩阵的转置

BP算法：数学基础

➤ 矩阵求导与Jacobian矩阵：

- ◆ 设 $X = [x_{ij}]_{i=1,j=1}^{mn}$
- ◆ $F(X)$ 是关于矩阵 X 的矩阵函数， $F(X) = [f_{kl}(X)]_{k=1,l=1}^{pq}$
- ◆ 记： $vec(X)$ 为 **列向量化标记**，即

$$vec(X) \stackrel{def}{=} [x_{11}, x_{21}, \dots, x_{m1}, x_{12}, \dots, x_{mn}]^T$$

$$vec(F(X)) \stackrel{def}{=} [f_{11}(X), f_{21}(X), \dots, f_{p1}(X), f_{12}(X), \dots, f_{pq}(X)]^T$$

- ◆ 则将矩阵 $F(X)$ 对矩阵 X 的导数定义为

$$\frac{\partial F(X)}{\partial X} \stackrel{def}{=} \frac{\partial vec(F(X))}{\partial vec(X)}$$

- ◆ 与矩阵微分的关联

$$dvec(F(X)) = \left(\frac{\partial F(X)}{\partial X} \right)^T \cdot dvec(X)$$

BP算法：数学基础

- ◆ 例3(链式法则): 已知 $Y = F(X) = AXB$, 其中 $A \in \mathcal{R}^{p \times m}, X \in \mathcal{R}^{m \times n}, B \in \mathcal{R}^{n \times q}, Y \in \mathcal{R}^{p \times q}$, $f(Y)$ 是关于 Y 的标量函数, 则 $f(Y)$ 关于 X 的偏导数是?

- ◆ 解答:

$$\begin{aligned} df &= \text{trace} \left(\left(\frac{\partial f}{\partial Y} \right)^T dY \right) \\ &= \text{trace} \left(\left(\frac{\partial f}{\partial Y} \right)^T A dX B \right) \\ &= \text{trace} \left(B \left(\frac{\partial f}{\partial Y} \right)^T A dX \right) \\ &= \text{trace} \left(\left(A^T \frac{\partial f}{\partial Y} B^T \right)^T dX \right) \end{aligned}$$


$$\frac{\partial f}{\partial X} = A^T \frac{\partial f}{\partial Y} B^T$$

BP算法：数学基础

- ◆ 例4(链式法则): 已知 $Y = \sigma(AXB)$, 其中 $A \in \mathcal{R}^{p \times m}, X \in \mathcal{R}^{m \times n}, B \in \mathcal{R}^{n \times q}, Y \in \mathcal{R}^{p \times q}$, σ 是逐元素运算函数, $f(Y)$ 是关于 Y 的标量函数, 则 $f(Y)$ 关于 X 的偏导是?

- ◆ 解答:

$$\begin{aligned} df &= \text{trace} \left(\left(\frac{\partial f}{\partial Y} \right)^T dY \right) \\ &= \text{trace} \left(\left(\frac{\partial f}{\partial Y} \right)^T (\sigma'(AXB) \odot (AdXB)) \right) \\ &= \text{trace} \left(\left(\frac{\partial f}{\partial Y} \odot \sigma'(AXB) \right)^T (AdXB) \right) \\ &= \text{trace} \left(\left(A^T \left(\frac{\partial f}{\partial Y} \odot \sigma'(AXB) \right) B^T \right)^T dX \right) \end{aligned}$$


$$\frac{\partial f}{\partial X} = A^T \left(\frac{\partial f}{\partial Y} \odot \sigma'(AXB) \right) B^T$$

参考文献

1. 张贤达 《矩阵分析及应用》 (第2版), 清华大学出版社

2. 矩阵运算

- <https://docs.scipy.org/doc/numpy-1.15.0/reference/routines.linalg.html?highlight=linalg>
- <https://docs.scipy.org/doc/numpy-1.15.0/reference/generated/numpy.einsum.html#numpy.einsum>

3. 矩阵求导

- <https://zhuanlan.zhihu.com/p/24709748> (标量函数对矩阵求导)
- <https://zhuanlan.zhihu.com/p/24863977> (矩阵对矩阵求导)